

# Entorno de simulación para el estudio de la locomoción de robots modulares

Autor: Rafael Treviño Menéndez    Matrícula: 020438

Tutor Interno: Darío Maravall Gómez-Allende  
Tutor Externo: Juan González Gómez (UAM)



# Índice general

---

|                                                      |               |
|------------------------------------------------------|---------------|
| <b>1. Introducción</b>                               | <b>1</b>      |
| 1.1. Encuadre del proyecto . . . . .                 | 1             |
| 1.1.1. Robótica y tipos de robots . . . . .          | 1             |
| 1.1.2. Robots modulares . . . . .                    | 9             |
| 1.2. Estado del arte . . . . .                       | 12            |
| 1.2.1. Microsoft Robotics Studio . . . . .           | 12            |
| 1.2.2. Webots . . . . .                              | 13            |
| 1.2.3. Darwin2K . . . . .                            | 13            |
| 1.2.4. Pyro . . . . .                                | 14            |
| 1.2.5. Adam . . . . .                                | 15            |
| 1.2.6. Simulador PolyBot . . . . .                   | 16            |
| 1.2.7. Simulador M-Tran . . . . .                    | 16            |
| 1.3. Motivación . . . . .                            | 17            |
| 1.4. Objetivos . . . . .                             | 18            |
| 1.5. Organización de la memoria . . . . .            | 19            |
| <br><b>I Designer: Robótica modular y topologías</b> | <br><b>21</b> |
| <b>2. Estructuras modulares</b>                      | <b>23</b>     |
| 2.1. Introducción . . . . .                          | 23            |
| 2.2. Módulo Y1 . . . . .                             | 23            |
| 2.3. Topologías . . . . .                            | 25            |
| 2.4. Conexiones entre módulos . . . . .              | 27            |
| 2.4.1. Conexión en Topologías 1D . . . . .           | 27            |
| 2.4.2. Conexión en topologías 2D . . . . .           | 28            |
| 2.5. Representación de estructuras . . . . .         | 29            |
| 2.6. Resumen . . . . .                               | 31            |

|                                                                 |               |
|-----------------------------------------------------------------|---------------|
| <b>3. Designer</b>                                              | <b>33</b>     |
| 3.1. Introducción . . . . .                                     | 33            |
| 3.2. Definición del problema . . . . .                          | 34            |
| 3.3. Diseño de la solución . . . . .                            | 34            |
| 3.3.1. Formato <i>.rbt</i> . . . . .                            | 34            |
| 3.4. Especificación de requisitos . . . . .                     | 35            |
| 3.5. Diseño de alto nivel . . . . .                             | 36            |
| 3.6. Descripción de interfaces entre módulos . . . . .          | 37            |
| 3.6.1. Interfaz <i>design</i> . . . . .                         | 37            |
| 3.6.2. Interfaz <i>drawStuff</i> . . . . .                      | 38            |
| 3.6.3. Interfaz <i>glSelector</i> . . . . .                     | 38            |
| 3.7. Diseño de bajo nivel . . . . .                             | 39            |
| 3.7.1. Módulo <i>design</i> . . . . .                           | 39            |
| 3.7.2. Módulo <i>glSelector</i> . . . . .                       | 39            |
| 3.8. Descripción de interfaces de usuario . . . . .             | 40            |
| 3.9. Pruebas . . . . .                                          | 42            |
| 3.9.1. Pruebas unitarias . . . . .                              | 42            |
| 3.9.2. Pruebas de integración . . . . .                         | 43            |
| 3.9.3. Pruebas de sistema . . . . .                             | 45            |
| 3.10. Resumen . . . . .                                         | 48            |
| <br><b>II Generator: Generación de secuencias de movimiento</b> | <br><b>49</b> |
| <b>4. Secuencias de movimiento</b>                              | <b>51</b>     |
| 4.1. Introducción . . . . .                                     | 51            |
| 4.2. Inspiración biológica: CPG's . . . . .                     | 51            |
| 4.3. Funciones generadoras y matrices de movimiento . . . . .   | 53            |
| 4.4. Funciones generadoras sinusoidales . . . . .               | 53            |
| 4.5. Resumen . . . . .                                          | 56            |
| <br><b>5. Generator</b>                                         | <br><b>57</b> |
| 5.1. Introducción . . . . .                                     | 57            |
| 5.2. Definición del problema . . . . .                          | 57            |
| 5.3. Diseño de la solución . . . . .                            | 58            |
| 5.3.1. Formato <i>.cpg</i> . . . . .                            | 58            |
| 5.3.2. Formato <i>.mvm</i> . . . . .                            | 58            |
| 5.4. Especificación de requisitos . . . . .                     | 59            |
| 5.5. Diseño de alto nivel . . . . .                             | 60            |
| 5.6. Descripción de interfaces entre módulos . . . . .          | 61            |
| 5.6.1. Interfaz <i>step</i> . . . . .                           | 61            |
| 5.6.2. Interfaz <i>bashcolor</i> . . . . .                      | 62            |
| 5.7. Diseño de bajo nivel . . . . .                             | 63            |
| 5.7.1. Módulo <i>step</i> . . . . .                             | 63            |
| 5.7.2. Módulo <i>bashcolor</i> . . . . .                        | 64            |

---

|        |                                  |    |
|--------|----------------------------------|----|
| 5.8.   | Pruebas . . . . .                | 65 |
| 5.8.1. | Ficheros de prueba . . . . .     | 65 |
| 5.8.2. | Pruebas unitarias . . . . .      | 67 |
| 5.8.3. | Pruebas de integración . . . . . | 68 |
| 5.8.4. | Pruebas de sistema . . . . .     | 69 |
| 5.9.   | Resumen . . . . .                | 73 |

### III Simulator: Simulación física 75

|        |                                                   |    |
|--------|---------------------------------------------------|----|
| 6.     | Simulación y evaluación                           | 77 |
| 6.1.   | Introducción . . . . .                            | 77 |
| 6.2.   | Motor de simulación . . . . .                     | 77 |
| 6.3.   | Motor físico ODE . . . . .                        | 78 |
| 6.4.   | Módulos Y1 . . . . .                              | 78 |
| 6.4.1. | Modelado . . . . .                                | 78 |
| 6.4.2. | Simulación . . . . .                              | 80 |
| 6.5.   | Simulación de robots modulares . . . . .          | 81 |
| 6.6.   | Movimiento de robots reales . . . . .             | 82 |
| 6.7.   | Resumen . . . . .                                 | 84 |
| 7.     | Simulator                                         | 85 |
| 7.1.   | Introducción . . . . .                            | 85 |
| 7.2.   | Definición del problema . . . . .                 | 85 |
| 7.3.   | Especificación de requisitos . . . . .            | 86 |
| 7.4.   | Diseño de la solución . . . . .                   | 88 |
| 7.4.1. | Protocolo serie . . . . .                         | 88 |
| 7.5.   | Diseño de alto nivel . . . . .                    | 88 |
| 7.6.   | Descripción de interfaces entre módulos . . . . . | 90 |
| 7.6.1. | Interfaz <i>simulation</i> . . . . .              | 90 |
| 7.6.2. | Interfaz <i>robot</i> . . . . .                   | 90 |
| 7.6.3. | Interfaz <i>sequence</i> . . . . .                | 91 |
| 7.6.4. | Interfaz <i>serial</i> . . . . .                  | 91 |
| 7.6.5. | Interfaz <i>drawStuff</i> . . . . .               | 92 |
| 7.6.6. | Interfaz <i>bashcolor</i> . . . . .               | 92 |
| 7.7.   | Diseño de bajo nivel . . . . .                    | 92 |
| 7.7.1. | Módulo <i>simulation</i> . . . . .                | 92 |
| 7.7.2. | Módulo <i>robot</i> . . . . .                     | 94 |
| 7.7.3. | Módulo <i>sequence</i> . . . . .                  | 95 |
| 7.7.4. | Módulo <i>serial</i> . . . . .                    | 95 |
| 7.7.5. | Módulo <i>bashcolor</i> . . . . .                 | 96 |
| 7.8.   | Descripción de interfaces de usuario . . . . .    | 96 |
| 7.9.   | Pruebas . . . . .                                 | 96 |
| 7.9.1. | Ficheros de prueba . . . . .                      | 96 |
| 7.9.2. | Pruebas unitarias . . . . .                       | 97 |

|                                                         |            |
|---------------------------------------------------------|------------|
| 7.9.3. Pruebas de integración . . . . .                 | 99         |
| 7.9.4. Pruebas de sistema . . . . .                     | 100        |
| 7.10. Resumen . . . . .                                 | 103        |
| <b>8. Extensión del simulador: Algoritmos genéticos</b> | <b>105</b> |
| 8.1. Introducción . . . . .                             | 105        |
| 8.2. El problema de la coordinación . . . . .           | 105        |
| 8.3. Espacio de soluciones . . . . .                    | 106        |
| 8.4. Algoritmos genéticos. . . . .                      | 106        |
| 8.5. La extensión optimizer . . . . .                   | 107        |
| 8.5.1. Pseudocódigo . . . . .                           | 107        |
| 8.5.2. Experimento de prueba . . . . .                  | 107        |
| 8.6. Resultados . . . . .                               | 108        |
| 8.7. Resumen . . . . .                                  | 111        |
| <b>IV MRSuite: Entorno completo de simulación</b>       | <b>113</b> |
| <b>9. El entorno MRSuite</b>                            | <b>115</b> |
| 9.1. Introducción . . . . .                             | 115        |
| 9.2. Arquitectura . . . . .                             | 115        |
| 9.3. Pruebas de aceptación . . . . .                    | 116        |
| 9.4. Validación del simulador . . . . .                 | 118        |
| 9.4.1. Pruebas con el robot PP . . . . .                | 118        |
| 9.4.2. Pruebas con el robot Hypercube . . . . .         | 122        |
| 9.5. Manual de usuario . . . . .                        | 127        |
| 9.5.1. Instalación . . . . .                            | 127        |
| 9.5.2. Manejo . . . . .                                 | 128        |
| 9.6. Resumen . . . . .                                  | 130        |
| <b>10.Resultados y conclusiones</b>                     | <b>131</b> |
| <b>11.Líneas futuras</b>                                | <b>133</b> |
| <b>Bibliografía</b>                                     | <b>134</b> |

# Índice de figuras

---

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1.1. Un robot industrial realizando tareas de soldadura . . . . . | 2  |
| 1.2. Kits MindStorm RIS y MindStorm NXT . . . . .                 | 2  |
| 1.3. Robot Tritt . . . . .                                        | 3  |
| 1.4. Robot SkyBot . . . . .                                       | 4  |
| 1.5. Robot TarriBot . . . . .                                     | 4  |
| 1.6. Robot PI . . . . .                                           | 5  |
| 1.7. Prueba de sumo y laberinto . . . . .                         | 5  |
| 1.8. Prueba de alcafutbol y funny golf . . . . .                  | 6  |
| 1.9. Robot Mars Rover . . . . .                                   | 7  |
| 1.10. El robot Asimo, de Honda . . . . .                          | 8  |
| 1.11. La idea de Mark Yim . . . . .                               | 8  |
| 1.12. Robot serpiente . . . . .                                   | 9  |
| 1.13. PolyBot, historia de módulos . . . . .                      | 10 |
| 1.14. Modelo bípedo PolyBot . . . . .                             | 10 |
| 1.15. Modelo cuadrúpedo M-Tran II . . . . .                       | 11 |
| 1.16. Modelo en anillo de Cube Revolutions . . . . .              | 11 |
| 1.17. Microsoft Robotics Studio . . . . .                         | 12 |
| 1.18. Simulador Webots . . . . .                                  | 13 |
| 1.19. Simulador Darwin2K . . . . .                                | 14 |
| 1.20. Simulador Pyro, mediante Gazebo . . . . .                   | 14 |
| 1.21. Simulador Adam . . . . .                                    | 15 |
| 1.22. Simulador PolyBot . . . . .                                 | 16 |
| 1.23. Simulador M-Tran . . . . .                                  | 17 |
| 1.24. Versión inicial del simulador . . . . .                     | 17 |
| 2.1. Un módulo Y1 y sus diferentes partes . . . . .               | 24 |
| 2.2. Cotas de un módulo Y1 . . . . .                              | 24 |
| 2.3. Plano de un módulo Y1 . . . . .                              | 25 |
| 2.4. Topologías 1D . . . . .                                      | 25 |
| 2.5. Topologías 2D . . . . .                                      | 26 |
| 2.6. Topologías 3D . . . . .                                      | 26 |
| 2.7. Dos módulos conectados en fase . . . . .                     | 27 |
| 2.8. Dos módulos conectados en desfase . . . . .                  | 27 |
| 2.9. Robot en estrella real . . . . .                             | 28 |

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| 2.10. Tres configuraciones con conexión en triángulo, cuadrado y pentágono | 28  |
| 2.11. Ejemplo de conexonado con su grafo estructural asociado (I)          | 29  |
| 2.12. Ejemplo de conexonado con su grafo estructural asociado (II)         | 30  |
| 2.13. Ejemplo de conexonado con su grafo estructural asociado (III)        | 30  |
| 3.1. Designer                                                              | 33  |
| 3.2. Esquema de funcionamiento del diseñador                               | 34  |
| 3.3. Arquitectura de alto nivel del diseñador                              | 37  |
| 3.4. Interfaz del diseñador                                                | 40  |
| 3.5. Interfaz del diseñador con otro punto de vista                        | 41  |
| 3.6. Diferentes orientaciones de los módulos                               | 42  |
| 3.7. Nodos con múltiples aristas                                           | 42  |
| 3.8. Estructura creada para la prueba 1                                    | 46  |
| 3.9. Estructura creada para la prueba 2                                    | 46  |
| 3.10. Estructura creada para la prueba 3                                   | 47  |
| 4.1. Esquema de la espina dorsal de un animal                              | 52  |
| 4.2. Esquema de aplicación de CPG's a un robot modular                     | 52  |
| 4.3. Parámetros de una función generadora sinusoidal                       | 54  |
| 5.1. Esquema de funcionamiento del generador                               | 57  |
| 5.2. Arquitectura de alto nivel del generador                              | 61  |
| 6.1. Comparativa de la simulación en el plano físico y gráfico             | 80  |
| 6.2. Conexonado y dimensiones de un servo Futaba S3003                     | 81  |
| 6.3. Comunicaciones con el robot                                           | 82  |
| 6.4. Señales PWM                                                           | 83  |
| 7.1. Esquema de funcionamiento del simulador                               | 86  |
| 7.2. Arquitectura de alto nivel del simulador                              | 89  |
| 7.3. Interfaz del simulador                                                | 96  |
| 8.1. Gráfico de evolución de los algoritmos genéticos (I)                  | 109 |
| 8.2. Gráfico de evolución de los algoritmos genéticos (II)                 | 109 |
| 8.3. Gráfico de evolución de los algoritmos genéticos (III)                | 110 |
| 9.1. Diagrama de bloques del MRSuite                                       | 116 |
| 9.2. Simulación de la configuración lineal de 8 módulos                    | 117 |
| 9.3. Simulación de la configuración cuadrúpeda                             | 117 |
| 9.4. Simulación de la configuración en estrella                            | 118 |
| 9.5. Simulación del robot PP (I)                                           | 119 |
| 9.6. Simulación del robot PP (II)                                          | 120 |
| 9.7. Simulación del robot PP (III)                                         | 120 |
| 9.8. Simulación del robot PP (IV)                                          | 121 |
| 9.9. Simulación del robot PP (V)                                           | 122 |
| 9.10. Simulación del robot hypercube (I)                                   | 123 |

---

|                                                      |     |
|------------------------------------------------------|-----|
| 9.11. Simulación del robot hypercube (II) . . . . .  | 124 |
| 9.12. Simulación del robot hypercube (III) . . . . . | 125 |
| 9.13. Simulación del robot hypercube (IV) . . . . .  | 126 |
| 9.14. Simulación del robot hypercube (V) . . . . .   | 127 |



# Índice de pseudocódigos

---

|       |                                                             |     |
|-------|-------------------------------------------------------------|-----|
| 3.1.  | design: appendNode (nodes, sel, type)                       | 39  |
| 3.2.  | design: writeRobot (nodes, modelFile)                       | 39  |
| 3.3.  | glSelector: init (width, height)                            | 39  |
| 3.4.  | glSelector: start (x, y)                                    | 40  |
| 3.5.  | glSelector: nodes = end ()                                  | 40  |
| 5.1.  | step: cpgs = ParseFile (file)                               | 63  |
| 5.2.  | step: seq = generateMatrix (functions, s)                   | 63  |
| 5.3.  | step: mrsOutput (seq, fd)                                   | 64  |
| 5.4.  | step: cOutput (seq, fd)                                     | 64  |
| 5.5.  | step: octaveOutput (seq, fd)                                | 64  |
| 5.6.  | bashcolor: msg (L)                                          | 64  |
| 6.1.  | simulator: Algoritmo de simulación                          | 78  |
| 6.2.  | simulator: Algoritmo de posicionamiento de servos           | 80  |
| 6.3.  | simulator: Algoritmo de actualización de estímulos          | 82  |
| 7.1.  | simulation: initSim (robots, seqs, dModule, wire, serial)   | 92  |
| 7.2.  | simulation: simulationLoop (args, endTime, renderEnabled)   | 93  |
| 7.3.  | simulation: position = getRobotInitPosition (index)         | 93  |
| 7.4.  | simulation: position = getRobotLastPosition (index)         | 93  |
| 7.5.  | simulation: time = getSimulationTime ()                     | 93  |
| 7.6.  | robot: init (source, world, space, pos, dModule, wire, ser) | 94  |
| 7.7.  | robot: createRotbot ()                                      | 94  |
| 7.8.  | robot: drawRobot ()                                         | 94  |
| 7.9.  | robot: setServos (seq)                                      | 94  |
| 7.10. | sequence: init (seq)                                        | 95  |
| 7.11. | sequence: forward ()                                        | 95  |
| 7.12. | sequence: position = desiredPos ()                          | 95  |
| 7.13. | sequence: writeGait (file)                                  | 95  |
| 7.14. | serial: init (term)                                         | 95  |
| 7.15. | serial: setServos (servos)                                  | 95  |
| 7.16. | serial: setMask (mask)                                      | 95  |
| 8.1.  | optimizer: optimizer <fichero de estructura>                | 107 |



---

# Capítulo 1

## Introducción

---

### 1.1. Encuadre del proyecto

#### 1.1.1. Robótica y tipos de robots

Este proyecto se encuadra dentro del ámbito de los robots modulares y su simulación.

El termino *Robot* fue acuñado por Karel Kapek y proviene de la palabra checa *robota* que significa *trabajo*. Creó esta palabra para denominar a unas máquinas con forma humana que realizaban el trabajo del hombre. Esta idea de robots humanoides tiene mucho sentido. Permite que las herramientas diseñadas para hombre puedan ser directamente usadas por los robots. En la actualidad el término robot[Rob] tiene un significado más amplio, incluyendo no sólo a los robots con forma humana.

Para comprender mejor el ámbito del proyecto se va a establecer una clasificación en cuatro grupos: robots industriales, de educación, de exploración y de investigación.

#### Robots industriales

Los **robots industriales** realizan tareas repetitivas en las cadenas de montaje de las fábricas, así como operaciones que pueden resultar peligrosas para los operarios. Con ellos se consigue incrementar la productividad y la fiabilidad obteniéndose mayores beneficios económicos. El típico robot industrial es un brazo mecánico anclado a una base, como el mostrado en la figura 1.1. Pueden realizar diferentes tareas según el manipulador que se les instale, como por ejemplo pintar, soldar o ensamblar, haciéndolos más versátiles. La locomoción no es una característica importante, ya que son las diferentes piezas las que llegan al robot a través de una cinta transportadora. Se puede encontrar más información en [PC83], [MPG89], [Ang86].



Figura 1.1: Un robot industrial realizando tareas de soldadura

## Robots de educación

La robótica es un área multidisciplinar compuesta por mecánica, electrónica y programación. Por un lado, esta combinación hace que la robótica sea apasionante. Pero, en el otro lado, la curva de aprendizaje para los neófitos es mayor que en el caso de otras disciplinas. Para solucionar este problema aparecen los **robots de educación**. Están pensados para facilitar la iniciación en la robótica, así como enseñar conceptos de mecánica, electrónica y programación. En general, los robots educativos utilizan ruedas u orugas como medio de locomoción.

Los **Legó MindStorm** entran dentro de esta clasificación, figura 1.2. Proviene del mundo de los juguetes. Las estructuras mecánicas se construyen utilizando sus populares piezas, lo que hace muy sencilla su construcción y modificación. Como electrónica emplean los que denominan “el ladrillo”. Una pieza de Legó especializada que funciona como un cerebro al que se le conectan los sensores y actuadores, que también tienen una apariencia tipo Legó. El entorno de programación es visual y muy intuitivo, siguiendo la misma idea de modularidad que para la mecánica. Las instrucciones son “piezas” que se unen en la pantalla unas con otras “construyéndose” los algoritmos que controlan el robot.

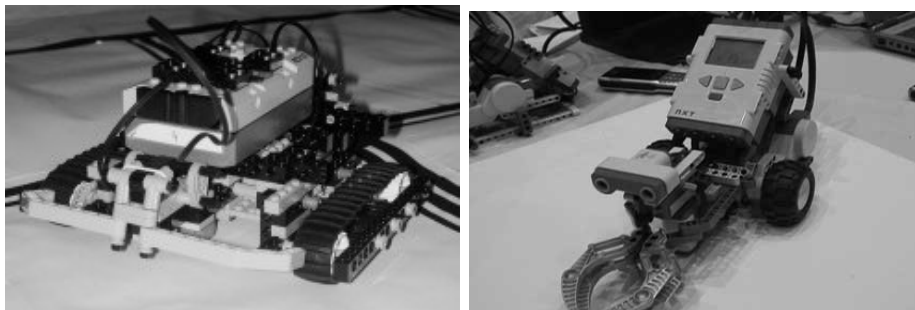


Figura 1.2: Kits MindStorm RIS y MindStorm NXT

El inconveniente que presentan es que toda la electrónica y piezas empleadas tienen que ser de Lego. No es posible extender los robots con sensores o componentes electrónicos comerciales, ni tampoco se pueden clonar los prototipos creados si no es comprando más kits de Lego. Se trata de robots cerrados.

Frente a esta rigidez se han desarrollado en las universidades españolas otros robots educativos más abiertos y que utilizan componentes y estructuras comerciales, de manera que pueden ser fácilmente clonables y ampliables.

Uno de estos kits fue el robot **Tritt**[Tri], creado en 1997 por cuatro estudiantes de la UPM y que luego comercializaron con la empresa Microbótica S.L. Su aspecto se puede ver en la figura 1.3. Como estructura mecánica utiliza piezas de Lego. Para la electrónica usa el microprocesador 6811 de Motorola y la programación se realizaba originalmente en ensamblador y posteriormente en C. Con este robot, los estudiantes aprendían arquitectura de computadores programando a bajo nivel el procesador de Motorola además de introducirse en la robótica. Incluso podían poner en práctica sus conocimientos de electrónica añadiendo nuevos sensores. La aplicación típica de Tritt es la de seguir una línea negra sobre fondo blanco. Para ello utiliza dos sensores de infrarrojos del tipo CNY70.

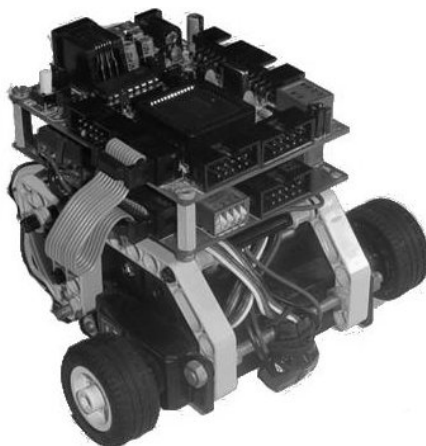


Figura 1.3: Robot Tritt

La evolución de Tritt es el robot **Skybot**[Skya], mostrado en la figura 1.4. En vez de piezas de Lego utiliza unas piezas en plástico de 3mm cortadas por láser. El robot se diseñó de tal forma que los estudiantes se pudiesen construir las piezas a partir de láminas de PVC o metacrilato, cortándolas y pegándolas. Como electrónica se emplea la tarjeta Skypic[Skyb], basada en el microcontrolador PIC16F876A de Microchip. Este es un microcontrolador muy usado, barato y fácil de encontrar. Para la programación del robot se pueden usar cualquiera de las herramientas disponibles para esta familia de micros. Existe tanto software comercial como alternativas libres.



Figura 1.4: Robot SkyBot

Muchos de los involucrados en la robótica vienen del mundo de la electrónica o el software, para ellos la mecánica es un problema. Han aparecido soluciones muy originales a este problema, como por ejemplo el robot **TarriBot**[Tar], que es extremadamente fácil de construir a partir de una tarrina de CD's y usando como ruedas dos CD's. En la figura 1.5 se muestra una foto.



Figura 1.5: Robot TarriBot

Las ventajas de robots como Tritt, SkyBot y TarriBot entre otros es que son robots libres. Todos los planos de la mecánica y la electrónica están disponibles para que cualquiera los pueda estudiar, modificar, fabricar o distribuir. Además se usa una electrónica comercial, lo que permite aplicar lo aprendido para el desarrollo de proyectos profesionales.

Existen otros kits de robótica educacional más especializados y avanzados, que incorporan mayor cantidad de sensores. Uno de ellos es el robot Pi[PI], creado también por estudiantes de la UPM y que actualmente se comercializa por la empresa

RBZ design. Tiene una forma de cilindro, como se muestra en la figura 1.6. Este robot fue diseñado con la idea de explorar los comportamientos distribuidos y colaborativos de colonias de robots.

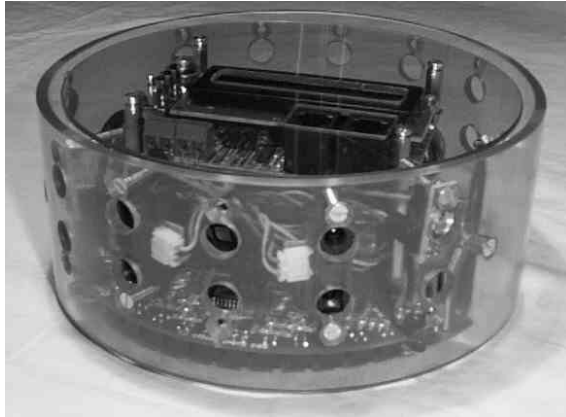


Figura 1.6: Robot PI

Gracias a los robots de educación, la robótica se está extendiendo entre las universidades españolas. Se organizan diferentes competiciones y concursos que ponen a prueba la creatividad y el ingenio de los alumnos. Además, ofrecen la oportunidad de practicar todos los conocimientos teóricos adquiridos. Cada año el nivel de las pruebas aumenta, lo que provoca que los robots diseñados sean cada vez más sofisticados.

Uno de los eventos más importantes a nivel nacional es Alcabot[Alc]. En 2006 se realizaron las pruebas de sumo, laberinto, alcafutbol y funny golf, figuras 1.7 y 1.8. Esta última servía de clasificación para Eurobot, una competición europea.

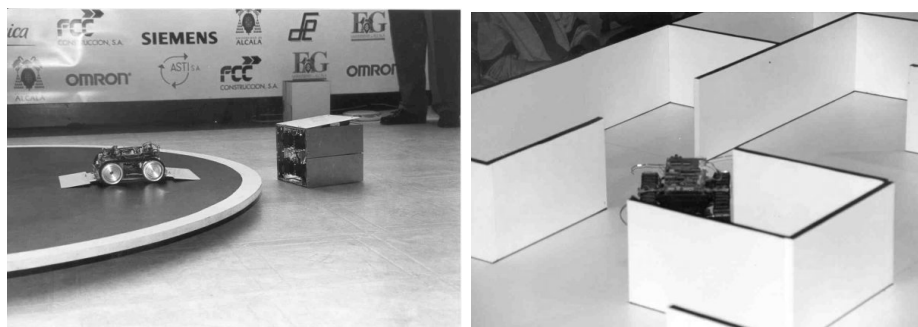


Figura 1.7: Prueba de sumo y laberinto

En la prueba de sumo, dos robots situados en el interior de un tatami circular luchan por expulsar al oponente. Mediante sensores de infrarrojos o ultrasonidos detectan al adversario y realizan la estrategia programada.

La prueba de laberinto es la más algorítmica. En la última edición se situaba un robot en el interior del laberinto y tenía que lograr salir en el menor tiempo posible. La forma del laberinto era conocida, pero no la posición inicial del robot. El primer objetivo, por tanto, era el de localizarse y luego calcular la trayectoria de salida.

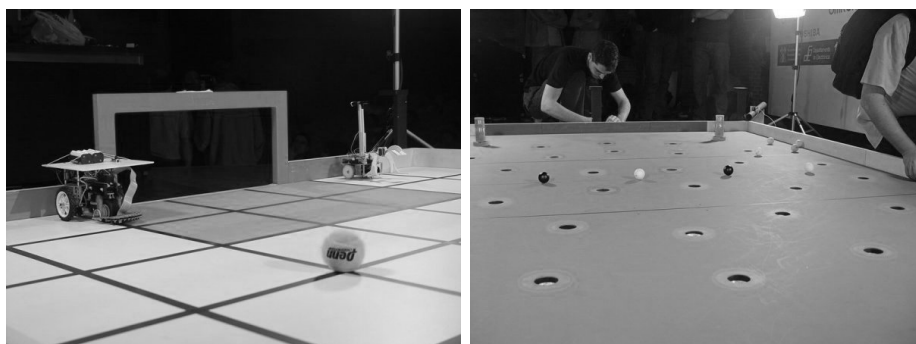


Figura 1.8: Prueba de alcafutbol y funny golf

Otras pruebas están inspiradas en los deportes. En la prueba de fútbol, cada grupo compuesto por dos robots debe empujar una pelota para introducirla en la portería contraria. El campo está formado por cuadrados de colores para permitir a los robots situarse.

En la prueba denominada “Funny Golf” existen dos tipos diferentes de agujeros y pelotas. Cada equipo debe meter las pelotas de su color en sus agujeros mientras dificulta la tarea del equipo contrario.

## Robots de exploración

Los **robots de exploración** son empleados para el reconocimiento de entornos hostiles o de complicado acceso para el hombre. Algunos ejemplos son el **Dante II** que descendió al interior del cráter del monte Spurr en Alaska o el **Mars Rover**, figura 1.9, que la NASA envió para la exploración de Marte. El primero usa seis patas para la locomoción mientras que el segundo utiliza ruedas.

El gran problema a resolver en este tipo de robots es el de la locomoción: conseguir un robot lo más versátil posible que sea capaz de moverse por la mayor cantidad posible de terrenos. La solución clásica consiste en estudiar las condiciones del terreno por el que se tiene que desplazar y diseñar un robot adaptado a esos requisitos. El inconveniente es que es necesario conocer a priori la zona.

La mayoría de los robots de exploración comienzan como proyectos de investigación. Se desarrollan diferentes prototipos y finalmente se selecciona el más adecuado. La propia NASA lleva varios años trabajando en este problema de la exploración en Marte. Uno de los prototipos que desarrollaron fue el robot Ambler, con seis patas de tres grados de libertad cada una. Sin embargo, con sus 3 metros y medio de altura y 3 de anchura nunca fue viable su utilización para enviarlo a Marte.

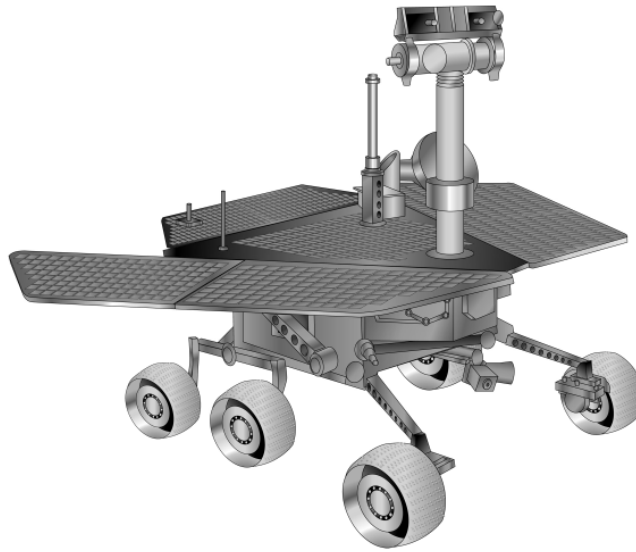


Figura 1.9: Robot Mars Rover

De momento la solución que está empleando la NASA para la exploración consiste en robots telecontrolados con ruedas. Pero esta manera de desplazarse no está exenta de problemas. Los terrenos no pueden ser muy escarpados y tienen que ser cuidadosamente seleccionados.

### Robots de investigación

Los **robots de investigación** se usan para desarrollar nuevas ideas. Con ellos se prueban nuevos conceptos, se diseñan mejores algoritmos o se crean estructuras más perfeccionadas. No existe un prototipo de robot de investigación. Hay tantos modelos diferentes como investigadores en robótica. Unos experimentan con nuevas formas de locomoción, otros imitando animales como peces, salamandras o gusanos, e incluso otros en robots capaces de autorreplicarse.

Un tipo de robots de investigación que está teniendo mucha repercusión a nivel mediático recientemente son los robots humanoides. Todavía son prototipos y no existen versiones comerciales. La utilidad de estos robots es la que se comentaba al comienzo de esta introducción: la creación de robots que puedan utilizar las herramientas que han sido diseñadas para los humanos. Uno de los robots humanoides más avanzados es Asimo[Asi], desarrollado por la empresa nipona Honda. La evolución del robot se puede ver en la figura 1.10.



Figura 1.10: El robot Asimo, de Honda

Otro grupo de robots en los que se está investigando muy activamente son los llamados robots modulares reconfigurables. Esta idea la propuso Mark Yim en su tesis doctoral en 1994 como posible solución al problema de la locomoción enunciado en el apartado anterior. Frente a la idea de diseñar robots específicos a partir de las características de un terreno, propuso crear robots que se adaptasen al terreno, cambiando tanto su forma como su manera de desplazarse.

Para ilustrar esta idea describió un escenario muy cotidiano: el porche de uno de los edificios de su universidad. Para atravesar ese porche y llegar al jardín, el robot primero tendría que moverse por el propio suelo llano de la terraza. Para desplazarse por él podría adoptar la forma de una rueda y rodar sobre sí mismo, un movimiento rápido y energéticamente eficiente. Al final del porche tendría que pasar por debajo de la barandilla. El robot se transformaría en un gusano, pasaría por debajo y bajaría el escalón para llegar al jardín. Una vez allí, se convertiría en un robot cuadrúpedo. Al tener 4 patas el movimiento es más estable y puede sortear mejor las irregularidades del terreno.



Figura 1.11: La idea de Mark Yim

Este escenario, representado en la figura 1.11, lo propuso para visualizar su idea: que sean los robots los que se transformen para adaptarse al medio. Aparecieron así los robots modulares.

### 1.1.2. Robots modulares

Los robots modulares están formados a partir de unos módulos básicos. El diseño se centra en el módulo y mediante la unión de ellos se crean las estructuras de los robots. Según la manera en cómo se conectan y separan los módulos se puede establecer una división, entre robots autoconfigurables, capaces de cambiar su forma automáticamente, y robots manualmente configurables en los que es el diseñador el que junta los módulos para crear estructuras.

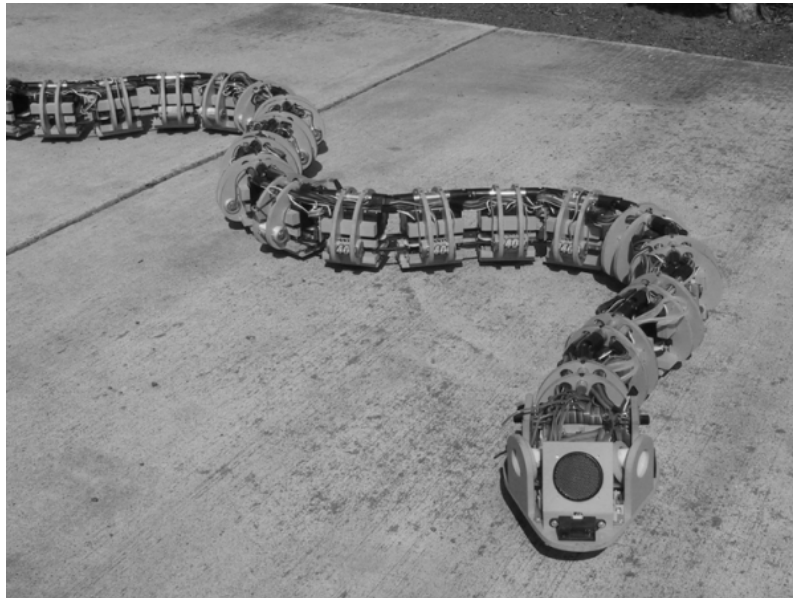


Figura 1.12: Robot serpiente

Las estructuras modulares más sencillas son las cadenas de módulos. Los robots tipo serpiente se pueden considerar como los primeros robots modulares, figura 1.12. Sin embargo, fue Mark Yim en 1994 en su robot **Polypod** el que introdujo la idea de unos módulos de propósito general.

Esta idea fue evolucionando hasta que en 1997 creó **Polybot**[Pol] y demostró con un experimento la viabilidad de la autoconfiguración mediante la transformación de gusano a rueda. En la figura 1.13 se muestran las diferentes generaciones de los módulos desarrollados. La generación I se corresponde con robots manualmente configurables. Dentro de esta generación hay diferentes versiones.

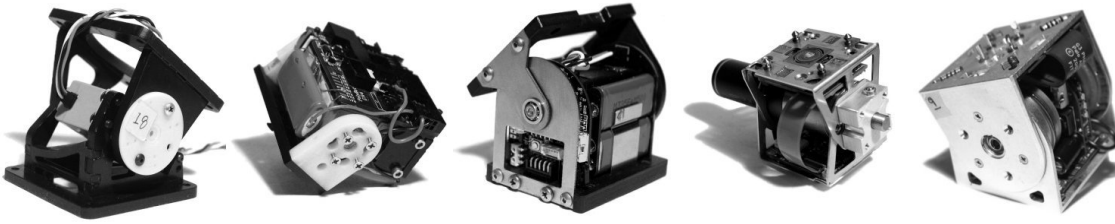


Figura 1.13: PolyBot, historia de módulos

La última, la versión 5, permite crear estructuras modulares más complejas, como el robot bípedo mostrado en la figura 1.14. En la generación II los módulos pueden unirse y soltarse automáticamente y los robots, por tanto, son autoconfigurables. La generación III es una mejora de la anterior, con unos módulos de dimensiones más reducidas, ya que caben en un cubo de 5cm de lado.



Figura 1.14: Modelo bípedo PolyBot

Uno de los robots modulares autoconfigurables más avanzados es el **M-Tran**[MTr], desarrollado en el Instituto de Ciencia y Tecnología Avanzada (AIST), Japón. Cada módulo tiene dos grados de libertad y las uniones entre ellos se pueden realizar de diferentes maneras.



Figura 1.15: Modelo cuadrúpedo M-Tran II

En la figura 1.15 se muestra el robot con dos configuraciones diferentes, una como cuadrúpedo y otra como cadena de módulos. Adicionalmente se muestra un módulo.

En España también se están realizando investigaciones en el campo de los robots modulares. **Juan González** en su tesis doctoral en desarrollo está trabajando en la locomoción y coordinación de robots modulares con topologías de unas dimensión. Ha diseñado los **módulos Y1**, inspirados en la generación I de Polybot. Con ellos se pueden crear robots modulares manualmente configurables. Se ha construido la saga de robots gusanos **Cube[Cubb]: Cube Reloaded, Cube Revolutions** e **Hypercube**.

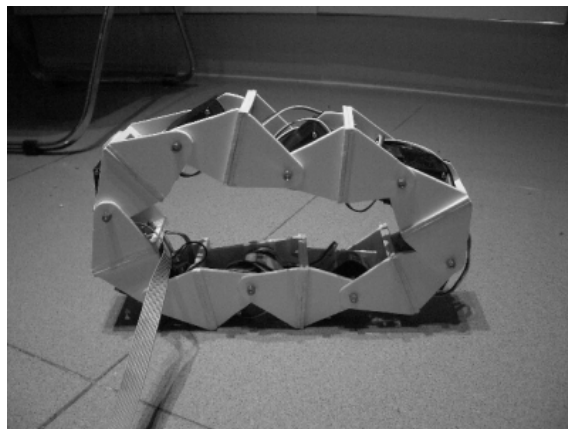


Figura 1.16: Modelo en anillo de Cube Revolutions

En la figura 1.16 se muestra a Cube Revolutions en una configuración de rueda. Uno de los objetivos de este proyecto fin de carrera es aportar un entorno de

simulación para los robots contruidos con estos módulos y que sirva de ayuda a los investigadores de este área.

Los robots modulares además pueden ser utilizados como robots de aprendizaje, permitiendo a los alumnos crear sus propias configuraciones e investigando las maneras de hacerlas moverse.

## 1.2. Estado del arte

La simulación en el campo de la robótica modular es muy importante. Gracias a ella no sólo es posible conocer a priori cómo se mueve una determinada configuración, sino que permite implementar funciones de evaluación para utilizar algoritmos de búsqueda que encuentren soluciones a diferentes problemas. Por un lado se podrían realizar búsquedas sobre qué configuraciones son las más óptimas para desplazarse por un determinado terreno. Y por otro lado, dada una configuración fija de  $M$  módulos, se puede buscar cuáles son los movimientos más óptimos. Esto sólo es posible mediante la simulación.

En este apartado se describen algunos de los simuladores más importantes para robótica, indicando cuáles son sus principales características y en qué difieren del simulador creado en este proyecto, el MRSuite.

### 1.2.1. Microsoft Robotics Studio

Este proyecto representa la incursión de Microsoft en el mundo de la robótica[Mic]. Incorpora varios modelos de robots comerciales que es posible manejar y/o simular. Además permite la definición de nuevos robots, creados por el usuario. El control se realiza mediante un lenguaje de programación visual propio.

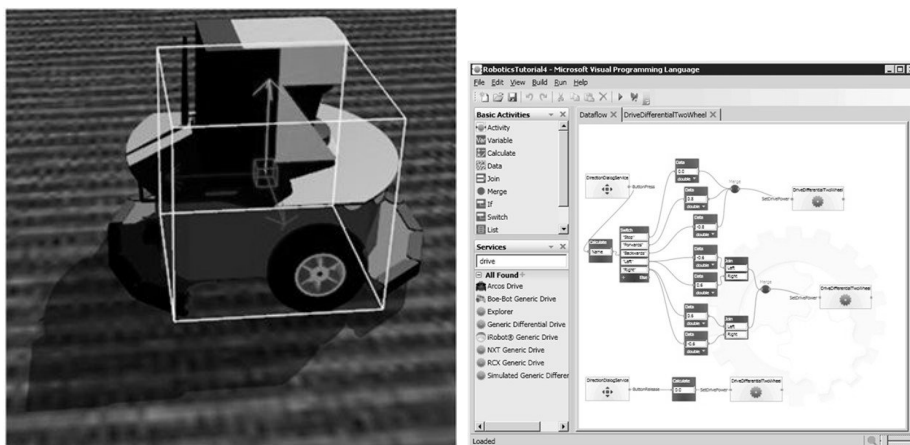


Figura 1.17: Microsoft Robotics Studio

En la figura 1.17 se muestran dos pantallazos de este programa en acción. En la parte de la izquierda aparece la simulación de un robot. En la derecha se ve el aspecto del lenguaje de programación visual: diferentes cajas interconectadas.

El Microsoft Robotics Studio es un entorno muy completo y profesional. Sin embargo entre sus desventajas están el que no es específico para robots modulares, utiliza un formato cerrado para la representación de la información, lo que ata al usuario a esta aplicación y sólo está disponible para plataformas Windows.

### 1.2.2. Webots

Webots[Web] es una aplicación de desarrollo y simulación de robots. Permite crear estructuras y generar las secuencias de movimiento para su locomoción. Además incluye soporte para varios robots comerciales, como el Aibo de Sony entre otros. Es un programa multiplataforma, disponible para Linux, Windows y Mac. Está basado en el motor físico ODE.

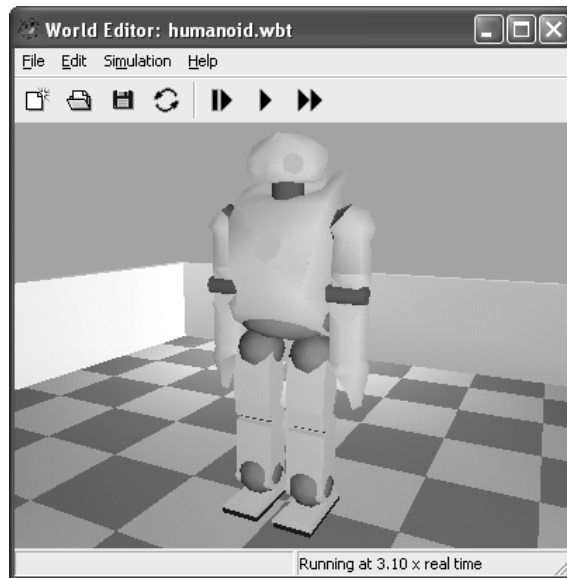


Figura 1.18: Simulador Webots

En la figura 1.18 se muestra la simulación de un robot bípedo que ha sido creado con Webots.

El principal inconveniente de este simulador es que no es libre y la licencia cuesta alrededor de 2300euros.

### 1.2.3. Darwin2K

Darwin2K[Dar] es el software desarrollado por Chris Leger en su tesis doctoral sobre optimización y síntesis automática de estructuras de robots. Además de simu-

lar cualquier tipo de robot, lo que hace a esta herramienta diferente es que incluye la generación de estructuras robóticas mediante evolución por algoritmos genéticos. Está programada en C++ en entorno Unix y es software libre. La visualización se hace a través de OpenGL.



Figura 1.19: Simulador Darwin2K

En la figura 1.19 se muestra un robot hexápoco simulado en Darwin2K.

La principal desventaja reside en su dificultad de uso. El diseño de los robots se hace a través de comandos en ficheros de texto. Es una aplicación muy especializada que requiere una gran curva de aprendizaje. En comparación, el MRSuite es más amigable y permite el diseño de los robots modulares utilizando una interfaz gráfica.

#### 1.2.4. Pyro

Pyro[Py] es un entorno de software libre que proporciona una arquitectura de control y simulación de robots. Está descrito en lenguaje Python. Permite interactuar con diferentes robots mediante el uso de drivers. Es muy extensible, dando la posibilidad de que los usuarios escriban sus programas en Python para interactuar con los robots.

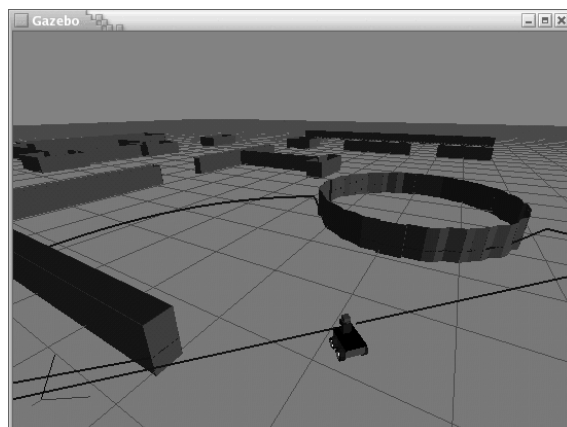


Figura 1.20: Simulador Pyro, mediante Gazebo

En la figura 1.20 se muestra la salida de una simulación a través de Gazebo, un visualizador en 3D.

Debido a su fácil extensibilidad, con Pyro se podrían simular también robots modulares. Pero carece de una herramienta visual para el diseño de las estructuras y el software para la generación de las secuencias. No obstante, dado que el MRSuite está también escrito en Python y es muy modular, se podría integrar con Pyro para dotarlo de soporte para la simulación de robots modulares.

### 1.2.5. Adam

El simulador Adam[Ada] es un software específico para la simulación de robots modulares compuesto por módulos simples con un único grado de libertad. Para la generación de las secuencias se utiliza como función generadora un tono sinusoidal con sus parámetros de amplitud, frecuencia y fase. Mediante unos scripts de texto se definen los robots. La herramienta calcula las secuencias de movimiento a partir de las funciones generadoras y aplicando técnicas evolutivas.



Figura 1.21: Simulador Adam

En la figura 1.21 se muestra la simulación de un robot complejo.

La ventaja de esta herramienta es la mayor cantidad de estructuras modulares que se pueden definir y simular. En el caso de MRSuite está restringida, por el momento, a topologías de una y dos dimensiones. No obstante, el MRSuite es un entorno más amigable. Permite definir las estructuras mediante una interfaz gráfica. Además, se pueden usar como funciones generadoras cualquier función matemática y no sólo senos. Por último, el MRSuite puede controlar robots modulares reales conectados a través del puerto serie.

### 1.2.6. Simulador PolyBot

Polybot, el robot modular creado por Mark Yim tiene su propio entorno de simulación y de creación de robots modulares, pero adaptado a sus módulos. Está desarrollado en Java y consta de un servidor al que se conectan los *applets* y una versión para su ejecución en solitario.

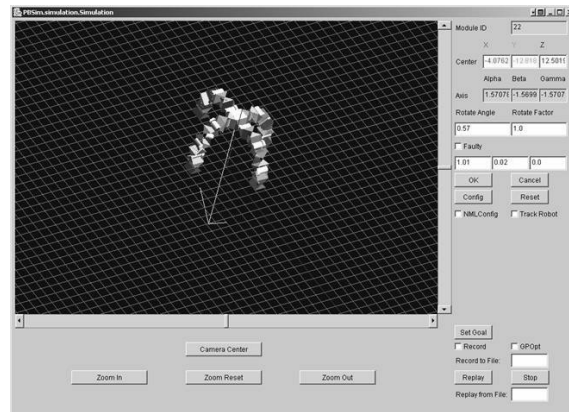


Figura 1.22: Simulador PolyBot

En la figura 1.22 se muestra un pantallazo de la simulación de un robot cuadrúpedo.

Una característica muy interesante es la posibilidad de almacenar los resultados de la simulación para luego poderlos reproducir. El sistema de generación de secuencias de movimiento está basado en autómatas finitos. Como formato de datos se emplea el XML.

El principal inconveniente de este entorno es que está especializado en los módulos Polybot. Y al no ser software libre no se puede adaptar para trabajar con otros módulos diferentes. Otra diferencia con el MRSuite es en la manera de sintetizar las secuencias de movimiento. Para el MRSuite se eligió un modelo bioinspirado basado en funciones generadoras.

### 1.2.7. Simulador M-Tran

El robot M-Tran también tiene su propio simulador[MTr], que permite la generación de secuencias de movimiento mediante algoritmos genéticos y la realización automática de las autoconfiguraciones del robot.

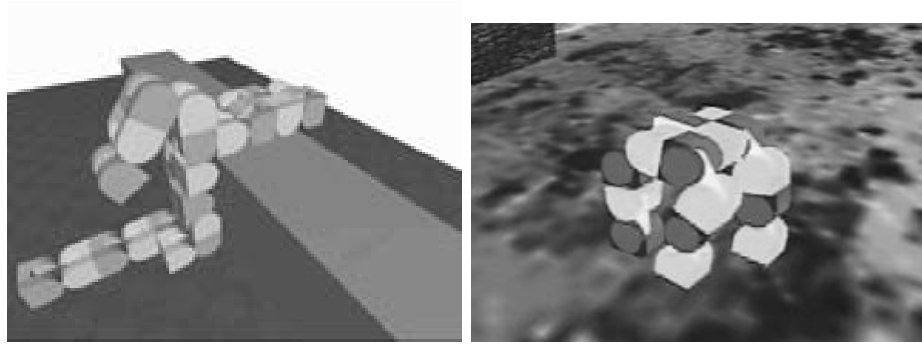


Figura 1.23: Simulador M-Tran

En la figura 1.23 se ven dos capturas de pantalla del simulador. En la izquierda se está simulando una autoreconfiguración. El robot está cambiando su forma para poder superar un obstáculo. En la derecha ha adoptado la forma de cuadrúpedo y se está moviendo.

Las desventajas de este simulador son las mismas que las de Polybot. Se trata de simuladores muy específicos que sólo son válidos con sus módulos. Y al no ser libres no se pueden adaptar para simular otros robots.

### 1.3. Motivación

La robótica es uno de esos campos en los que los curiosos no se aburren nunca, y fué eso lo que me llevó a interesarme por este área: la curiosidad. Por ser un área multidisciplinar, la robótica necesita de muchos profesionales de diferentes especialidades para funcionar, pero sin duda la informática es el “cerebro” de todo robot. Sus funciones van desde el simple control de los sensores y actuadores hasta los algoritmos más complejos de visión o navegación. Por todo esto, es la disciplina ideal, además de ser muy vistoso, para realizar el proyecto fin de carrera.

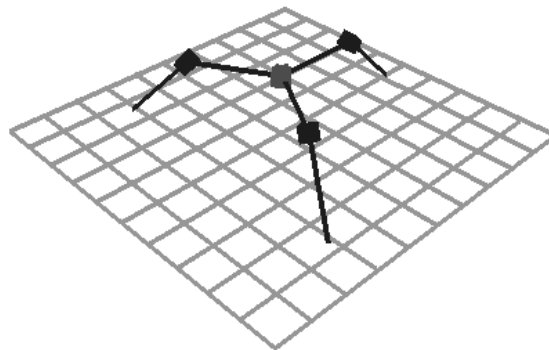


Figura 1.24: Versión inicial del simulador

A pesar de lo atractivo que puede ser la construcción de un robot real, las complicaciones que acarrea, sobre todo para alguien con escasos conocimientos mecánicos y electrónicos, además del presupuesto necesario, lo hacen poco propicio. Por eso la idea de un simulador cobra más fuerza como proyecto a desarrollar.

Fue hace 3 años que conocí a **Juan González** de la **UAM** en **SICFIMA**. La idea que presentó entonces era la de un robot ápedo llamado **Cube Revolutions** formado por módulos idénticos acoplados unos a otros, un **robot modular**. Esa idea me fascinó desde el principio. Pasado el tiempo se me ocurrió la idea de crear un simulador para esta fascinante clase de robots, esta fue la semilla del actual proyecto y que puede verse en la figura 1.24. A Juan la idea le entusiasmó y comencé a trabajar en el proyecto bajo su tutela, a pesar de que por entonces ni se me pasaba por la cabeza en que llegara a ser un proyecto fin de carrera.

Como curiosidad del proyecto comentar que comenzó siendo un proyecto escrito en lenguaje C. Más tarde comenzó mi fiebre de Python[Pyt], ya que utilizaba este lenguaje con casi cualquier pretexto, ya fuera para proyectos personales o académicos. No tardé mucho en modificar el proyecto a Python y los resultados no pudieron ser más satisfactorios. Además de incorporar la capacidad multiplataforma, el uso de las diferentes librerías y la extensibilidad de la que dotaron al sistema me dieron la razón del cambio.

## 1.4. Objetivos

El objetivo principal de este proyecto es el **desarrollo de un entorno de simulación y prototipado de robots modulares**, tiene los siguientes objetivos menores:

- 1.- **Software de diseño de estructuras modulares** basadas en los módulos Y1.
- 2.- **Software de diseño de secuencias de movimiento** a partir de fórmulas matemáticas.
- 3.- **Software de simulación** de robots modulares, que permita mejorar y evolucionar los diseños y movimientos diseñados.

Todas las partes que componen el entorno deben contar con las siguientes **características**:

- 1.- **Facilidad de uso**: Debe ser un entorno cómodo de utilizar para personas con conocimientos de manejo de un PC.
- 2.- **Multiplataforma**: Válido para que se puede utilizar bajo cualquier sistema operativo.

**3.- Extensible:** Cualquiera con conocimientos del lenguaje de programación debe ser capaz de crear sus propias aplicaciones basado en las librerías desarrolladas.

Gracias a los puntos 1 y 2 se asegura que cualquiera que desee utilizar estas herramientas sea capaz de hacerlo. El punto 3 permite aprovechar todo lo que se ha desarrollado en este proyecto para utilizarlo en otros sistemas.

Todo este entorno se denomina genéricamente **MRSuite**<sup>1</sup>.

## 1.5. Organización de la memoria

El tomo presentado se va a dividir en cuatro grandes partes. Tres de ellas se corresponden con las tres herramientas desarrolladas y la cuarta está dedicada a la integración de todas ellas.

Las partes que tratan sobre las herramientas se han dividido a su vez en dos capítulos. Uno donde se explican los conceptos teóricos involucrados y otro con la documentación técnica del software.

Las cuatro partes son:

**Designer:** En esta parte se tratará la teoría relacionada con la generación y diseño de estructuras modulares.

**Generator:** La segunda parte concierne principalmente a la generación de ondas para el movimiento de los robots.

**Simulator:** El eje central del proyecto es el simulador, que se encarga de la representación de los robots en un mundo tridimensional.

**MRSuite:** En la última parte se tratará la integración de las herramientas en el entorno completo que representa este proyecto, además de diferentes pruebas de aceptación y validación del proyecto completo. También se tratarán las líneas futuras del proyecto y las conclusiones finales.

Hay que resaltar que dentro de cada capítulo se dedica una sección a la introducción del mismo para que el lector pueda familiarizarse brevemente con los conceptos y definiciones que se van a tratar. Al final también existe por cada capítulo una sección de resumen que sintetiza todo lo visto.

El lector puede hacerse una idea general de los contenidos de cada capítulo leyendo la introducción y resumen del mismo.

---

<sup>1</sup>Modular Robotics Suite



---

Parte I

# Designer: Robótica modular y topologías

---



---

## Capítulo 2

# Estructuras modulares

---

### 2.1. Introducción

Como se vió en el apartado 1.1.2, los robots modulares están formados por la unión de unos módulos básicos. Dada su naturaleza modular, las estructuras creadas se denominarán genericamente **estructuras modulares**. El término **configuración** se utilizará como sinónimo de una estructura modular particular. En su acepción general, hace referencia a las distintas formas que puede adoptar un robot reconfigurable. Las estructuras modulares con características comunes se agrupan en **topologías**. Por ejemplo, se hablará de topologías de una dimensión cuando se trate de configuraciones formadas por módulos encadenados.

En este capítulo se introducirá el módulo Y1, que es en el que está basado el simulador, y todo el proyecto. Se mostrarán los tipos de conexionado posibles entre módulos adyacentes y las diferentes topologías que se pueden construir con ellos. Para terminar se darán ejemplos de diferentes estructuras modulares y cómo se pueden representar mediante grafos acíclicos.

### 2.2. Módulo Y1

Los módulos utilizados en este proyecto son los módulos Y1[Y1].

Son unos módulos muy sencillos. Con un único grado de libertad (GDL) actuando por un servo de tipo Futaba S3003 o compatible. Tiene dos partes principales: la **cabeza** y el **cuerpo**, como se muestra en la figura 2.1.

El servo está atornillado al cuerpo. El eje del servo está enganchado a la corona y ésta a la cabeza. De esta manera, las dos partes, cuerpo y cabeza, rotan una con respecto a la otra.

La parte superior se denomina **panza**. Nos servirá como referencia para especificar la orientación del módulo, panza arriba como en la figura 2.1 o panza abajo.

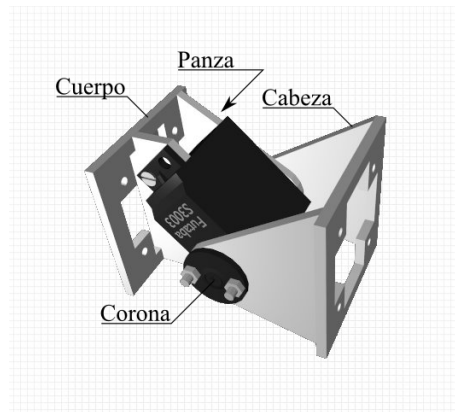


Figura 2.1: Un módulo Y1 y sus diferentes partes

Las dimensiones físicas del módulo cuando está en su posición inicial son:  $52 \times 52 \times 72$  mm como se muestra en la figura 2.2.

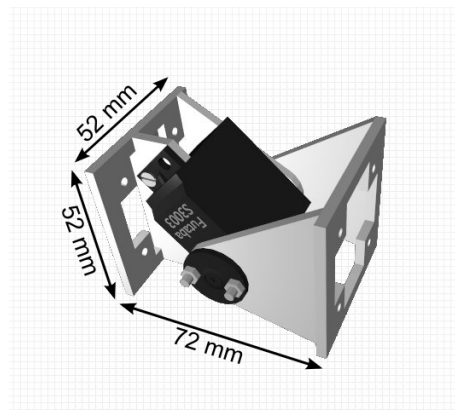


Figura 2.2: Cotas de un módulo Y1

El rango de giro es de 180 grados, que es el máximo permitido por el servo. El origen se toma en 0 grados y las posiciones extremas se corresponden con -90 y 90 grados.

Una de las principales características del Módulo Y1 es que es libre. Los planos están publicados y se conceden permisos para que cualquiera lo pueda estudiar, modificar, fabricar y distribuir. Además, su construcción es sencilla. Cada módulo está compuesto de seis piezas de plástico de 3mm de grosor que se unen utilizando un pegamento de contacto. En la figura 2.3 se muestra la planta, el alzado y una sección.

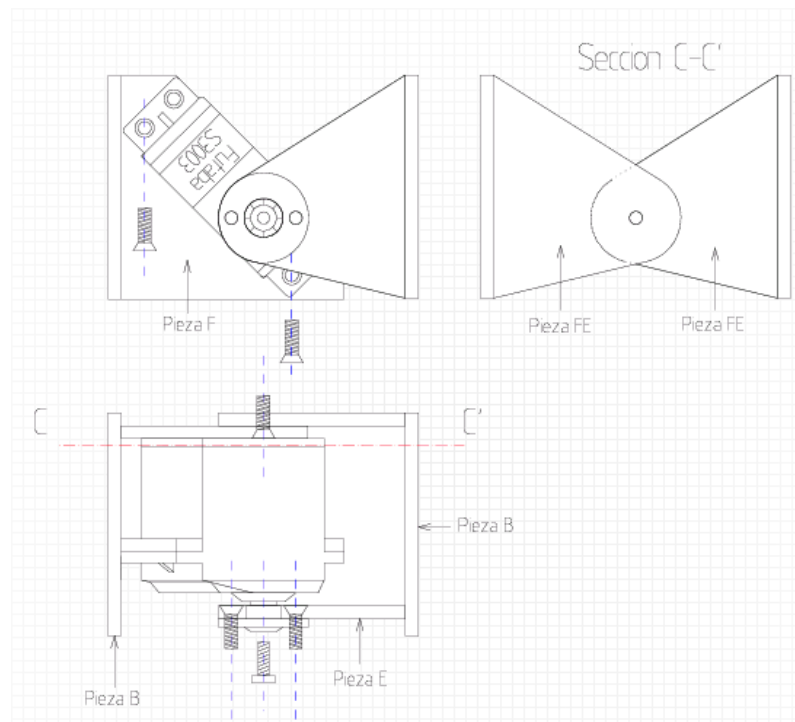


Figura 2.3: Plano de un módulo Y1

## 2.3. Topologías

Las infinitas estructuras modulares que se pueden construir se agrupan en tres grandes familias o topologías según sus dimensiones en el estado inicial. Así aparecen las topologías de una, dos y tres dimensiones.

- Topologías 1D:

Son aquellas constituidas por la unión en cadena de varios módulos. En su estado inicial todos los módulos pertenecen a la misma recta.

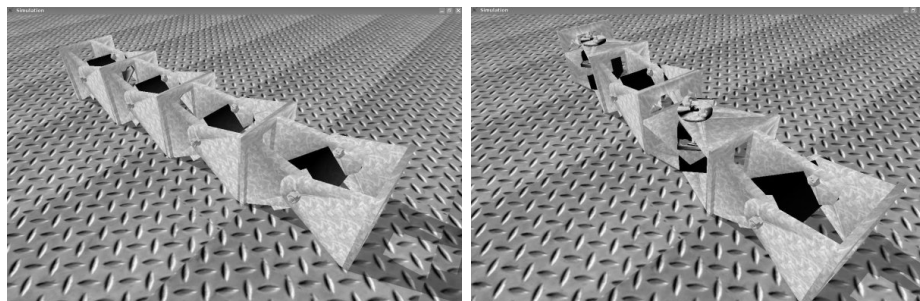


Figura 2.4: Topologías 1D

En la figura 2.4 se pueden ver dos configuraciones de cuatro módulos con una topología 1D.

- Topologías 2D:

Se caracterizan porque todos los módulos pertenecen al mismo plano cuando están en su estado inicial.

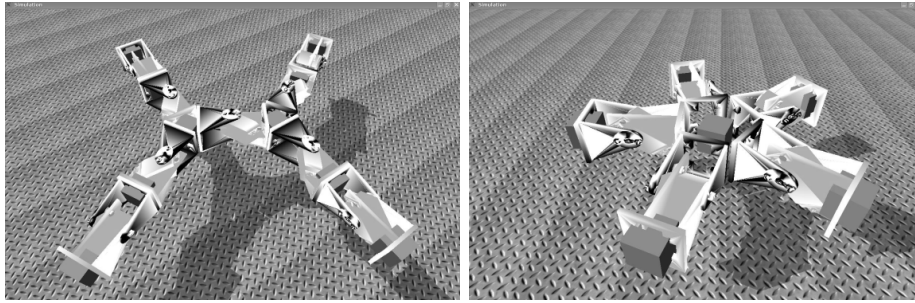


Figura 2.5: Topologías 2D

La figura 2.5 muestra dos ejemplos de topologías 2D. La configuración de la izquierda es un cuadrúpedo de nueve módulos. La de la derecha es una estrella de cinco.

- Topologías 3D:

Resto de estructuras. En su estado inicial, los módulos no están en el mismo plano.

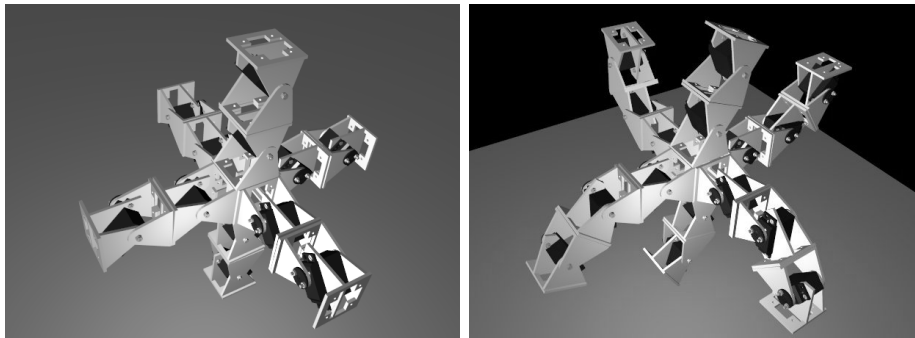


Figura 2.6: Topologías 3D

En la figura 2.6 se pueden ver dos configuraciones de 16 y 22 módulos con topología 3D. Están formadas por seis “brazos” en diferentes direcciones.

El entorno MRSuite desarrollado en este proyecto permite diseñar y simular robots modulares con topologías de 1 y 2 dimensiones.

## 2.4. Conexiones entre módulos

### 2.4.1. Conexión en Topologías 1D

Los módulos Y1 fueron inicialmente desarrollados para estudiar la locomoción de las topologías 1D. Por ello es muy fácil crear cadenas de módulos.

Dos módulos pueden conectarse entre sí de dos maneras: conexión en fase y en desfase.

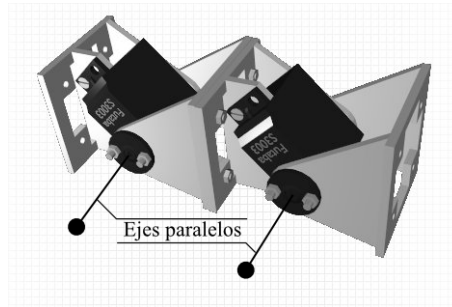


Figura 2.7: Dos módulos conectados en fase

La **conexión en fase** de muestra en la figura 2.7. Ambos módulos permanecen con la misma orientación y los ejes de giro son paralelos. Ésta es la conexión que permite estudiar la locomoción en 1D de robots de tipo gusano. Un ejemplo es el robot **Cube Revolutions**[Cuba].

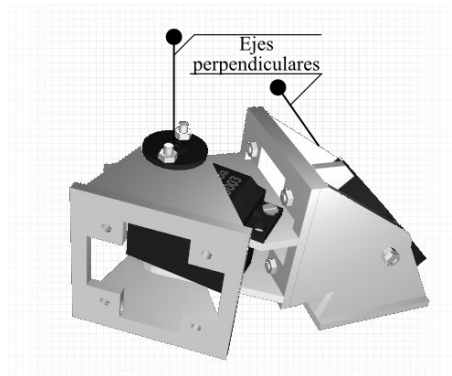


Figura 2.8: Dos módulos conectados en desfase

La **conexión en desfase** se muestra en la figura 2.8. Ahora los ejes de giro están rotados 90 grados. Uno de los módulos gira perpendicularmente al suelo y el otro lo hace paralelamente. Con esta conexión se pueden crear robots de tipo gusano capaces de moverse en un plano. Por ejemplo el robot **Hypercube**[Hyp], constituido

por 8 módulos desfasados, que es capaz de realizar al menos 5 tipos de movimientos diferentes.

### 2.4.2. Conexión en topologías 2D

Es posible utilizar los módulos Y1 para construir robots con topologías de dos dimensiones, aunque no han sido diseñados explícitamente para ello. Se puede conseguirlo bien construyendo nuevas piezas a las que se atornillarán los módulos Y1 o bien uniéndolos directamente mediante unas “bridas”. Un ejemplo es el robots de tres módulos con configuración en estrella que se observa en la figura 2.9.

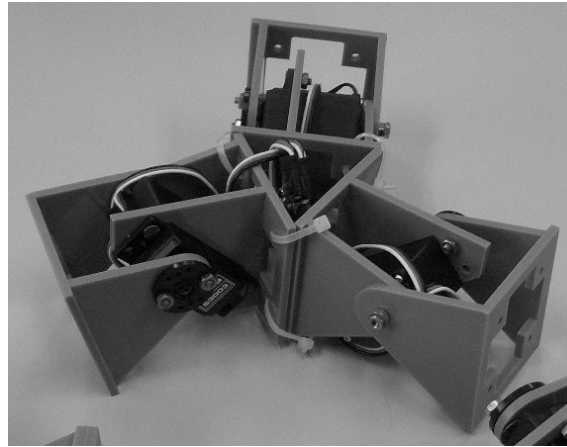


Figura 2.9: Robot en estrella real

Este tipo de conexionado en 2D se denomina **conexión en polígono**. Cada uno de los módulos se sitúa en uno de los lados de un polígono regular. Así se tienen conexiones poligonales en triángulo, cuadrado, pentágono, hexágono, ... En la figura 2.10 se muestran tres configuraciones simples con conexión en triángulo, cuadrado y pentágono.

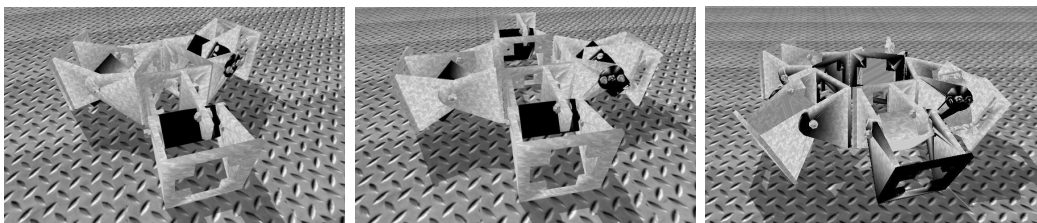


Figura 2.10: Tres configuraciones con conexión en triángulo, cuadrado y pentágono

## 2.5. Representación de estructuras

Las estructuras modulares que se pueden construir a partir de los módulos Y1 son aquellas con topologías de 1 y 2 dimensiones limitadas a los tipos de conexionado indicados en el apartado 2.4. Estas son las mismas estructuras soportadas por la aplicación MRSuite.

Para representarlas se utilizan grafos acíclicos simples, donde las aristas representan los módulos y los nodos son las conexiones entre ellos. Este tipo de grafos se denominan **grafos estructurales** del robot.

Cada arista tiene 3 propiedades que definen la orientación del módulo al que representa. Dichas propiedades tienen 2 valores cada una y la notación que se ha seguido se muestra en el cuadro 2.1. Todas las informaciones responden al estado inicial del módulo.

|                   |            |                                             |
|-------------------|------------|---------------------------------------------|
| Tipo de giro      | Vertical   | El eje es paralelo plano del suelo.         |
|                   | Horizontal | El eje es perpendicular al plano del suelo. |
| Izquierda-derecha | Derecha    | La cabeza del módulo apunta a la derecha.   |
|                   | Izquierda  | La cabeza del módulo apunta a la izquierda. |
| Arriba-abajo      | Arriba     | El módulo está panza arriba.                |
|                   | Abajo      | El módulo está panza abajo.                 |

Cuadro 2.1: Tabla de notación

Para ilustrar estos conceptos se pueden ver las figuras 2.11, 2.12 y 2.13, tres ejemplos de conexionado con sus respectivos grafos estructurales y los valores dados a las aristas.

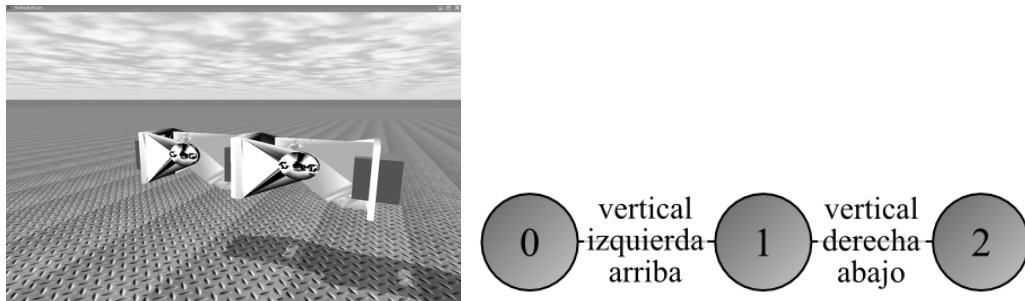


Figura 2.11: Ejemplo de conexionado con su grafo estructural asociado (I)

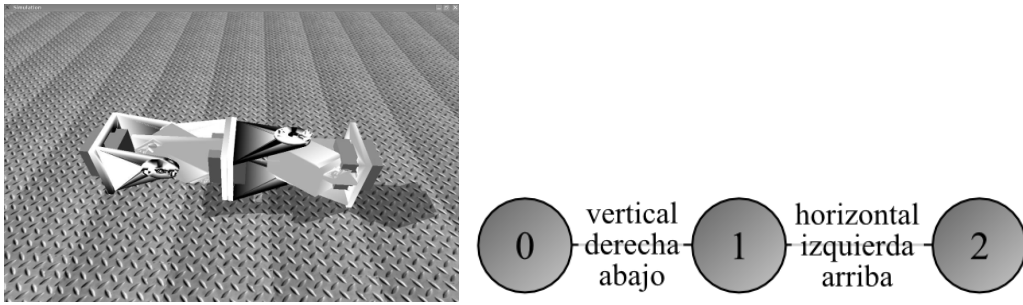


Figura 2.12: Ejemplo de conexionado con su grafo estructural asociado (II)

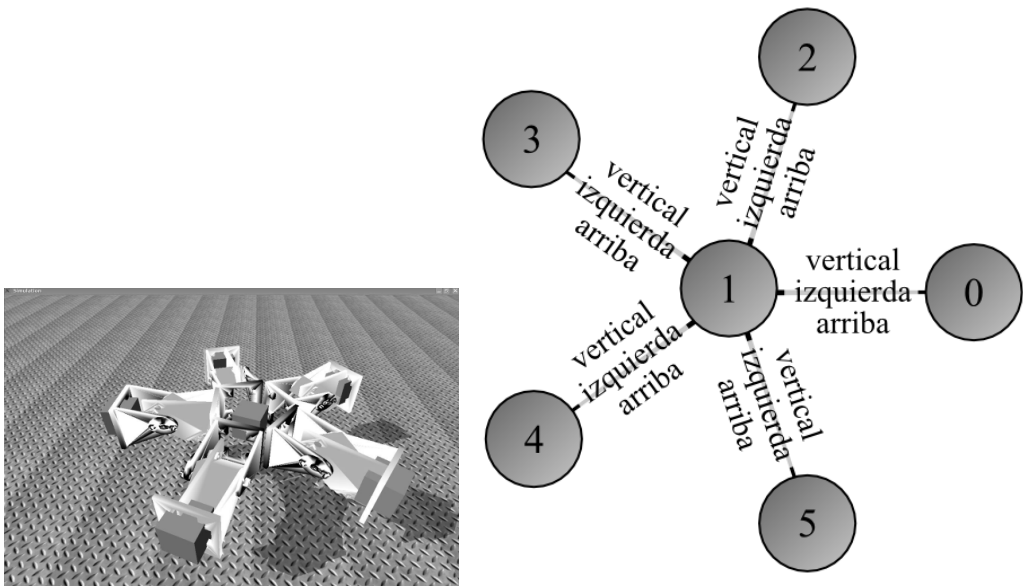


Figura 2.13: Ejemplo de conexionado con su grafo estructural asociado (III)

Hay que puntualizar que, a pesar de que en la representación interna los módulos tienen diferentes orientaciones, en la visualización tienen la misma apariencia.

## 2.6. Resumen

Los **robots modulares** están formados por elementos simples e indivisibles: los módulos. Este proyecto se basa en los **módulos Y1** que tienen un grado de libertad actuado por un servo.

Los robots modulares se pueden clasificar en **topologías** de una, dos y tres dimensiones. Las topologías 1D comprenden configuraciones que son “cadenas de módulos”. En las topologías 2D todos los módulos en su estado inicial se encuentran en el mismo plano. Finalmente las topologías 3D comprenden el resto de configuraciones.

Según la topología del robot, se pueden emplear diferentes **tipos de conexionado**. Para topologías 1D, dos módulos adyacentes se pueden conectar de dos maneras: en fase si ambos módulos giran en el mismo plano y desfase si lo hacen en planos perpendiculares. Para las topologías en 2D, el conexionado también puede ser en forma de polígono regular. Se pueden tener conexiones en triángulo, cuadrado, pentágono, hexágono, . . . .

Las **estructuras modulares** se pueden representar mediante grafos acíclicos denominados grafos estructurales, donde las aristas se corresponden con los módulos Y1 y los nodos son los puntos de conexión entre ellos.

La **herramienta desarrollada** en este proyecto se basa en los grafos estructurales para el diseño y simulación de robots modulares con topologías de 1 y 2 dimensiones. Basándose en los tres tipos de conexionado mencionados anteriormente: fase, desfase y polígono.



---

## Capítulo 3

# Designer

---

### 3.1. Introducción

*Designer* es una aplicación de diseño de **robots modulares**. Permite crear **estructuras modulares** a partir de **módulos Y1**. Como salida del diseño se obtiene un fichero con el **grafo estructural** con  $N$  nodos y  $A$  aristas. Este fichero puede ser importado posteriormente en el simulador para su visualización estática o dinámica.

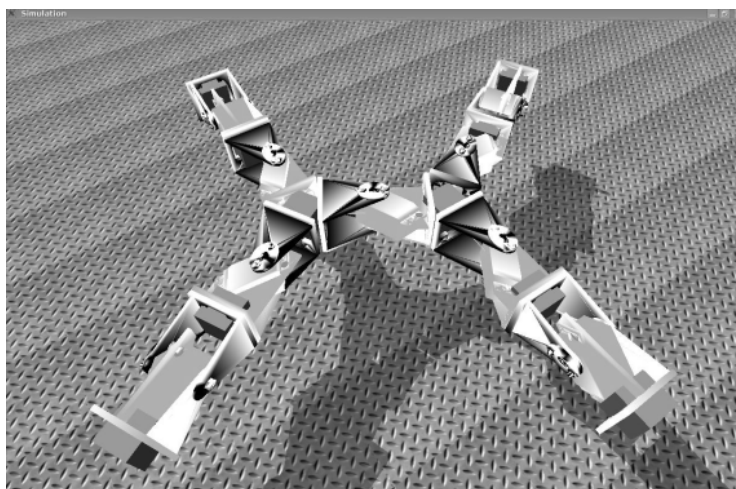


Figura 3.1: Designer

En la figura 3.1 se muestra el designer en funcionamiento para que el lector se pueda hacer una idea. Se puede ver un robot con estructura de tipo cuadrúpedo compuesta por 9 módulos y 10 nodos. El interfaz se describe con más detalle en el apartado 3.8 de la página 40.

## 3.2. Definición del problema

El objetivo es diseñar una aplicación que permita al usuario la creación de **estructuras modulares** mediante una interfaz gráfica.

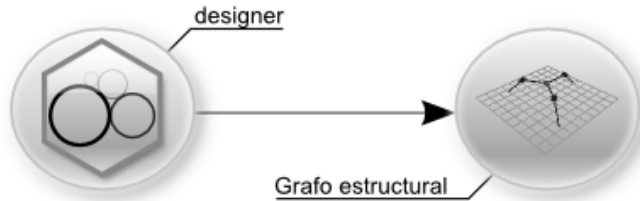


Figura 3.2: Esquema de funcionamiento del diseñador

En la figura 3.2 se muestra el esquema de funcionamiento. El usuario especifica la estructura del robot y el *designer* crea el **grafo estructural** y lo guarda en un fichero.

Para el almacenamiento de los grafos en ficheros se ha diseñado un formato específico, el .rbt, descrito en el apartado 3.3.1.

## 3.3. Diseño de la solución

### 3.3.1. Formato .rbt

El formato .rbt permite especificar los **grafos estructurales** en forma de lista de aristas. Cada arista se define con su nodo origen, su nodo destino y su orientación.

La estructura del formato es la siguiente:

- 1: # Definición de aristas
- 2: <Nodo origen > <Orientación 1 > <Nodo destino>
- 3: <Nodo origen > <Orientación 2 > <Nodo destino>
- 4: ...
- 5: <Nodo origen > <Orientación  $K$  > <Nodo destino>

Hay una arista por cada línea del fichero. Los tres campos están separados por espacios en blanco. El nodo origen y destino son números naturales, comenzando por el 0. Se pueden introducir comentarios utilizando el símbolo #. La orientación se compone de tres caracteres, cada uno de ellos, con dos valores:

#### 1.- Primer carácter: Tipo de giro

Indica si la articulación se mueve verticalmente ( $V$ ) u horizontalmente ( $H$ ).

**2.- Segundo carácter: Orientación izquierda-derecha**

Indica si la cabeza del módulo apunta a la derecha (*N*) o a la izquierda (*I*).

**3.- Tercer carácter: Orientación arriba-abajo**

Indica si la panza del módulo apunta al suelo (*N*) o hacia arriba (*I*).

En el apartado 3.9 de la página 42 se muestran varios ejemplos de este formato.

La importancia de este formato es la gran flexibilidad para la especificación de estructuras robóticas modulares.

### 3.4. Especificación de requisitos

**1.- RF *Capacidad de diseño* (Esencial)**

**1.1.- Configuraciones**

Se podrán diseñar **estructuras modulares** con **topologías** en 1D y 2D, compuestas por módulos que giren vertical u horizontalmente.

**1.2.- Módulos**

Se utilizarán los **módulos Y1**.

**2.- RF *Comandos* (Esencial)**

**2.1.- Guardado de configuraciones**

Almacenar la **estructura modular** en un fichero de tipo .rbt.

**2.2.- Adición de módulos**

Añadir nuevos módulos, bien de tipo vertical (VNN) u horizontal (HNN).

**3.- RF *Formato de salida* (Esencial)**

Se utilizará el formato .rbt, descrito en el apartado 3.3.1.

**4.- RU *Interfaz de línea de comandos* (Esencial)**

**4.1.- Argumentos**

La interfaz con la línea de comando tendrá los siguientes argumentos en ese orden:

**4.1.1.- Fichero de salida (Opcional)**

Se proporciona un nombre de fichero donde almacenar los datos del **grafo estructural**.

**5.- RU *Interfaz gráfica* (Esencial)**

**5.1.- Apariencia general**

Consistirá en una ventana para la visualización tridimensional de la estructura del robot.

#### 5.2.- Controles

Los comandos se realizarán bien por teclado o por ratón.

#### 5.3.- Movimiento de cámara

Todos los movimientos se realizarán con el ratón. Serán:

##### 5.3.1.- Movimiento de cabeceo y rotación sobre si misma

Mediante el movimiento del ratón y de la pulsación del botón izquierdo se realizarán estos movimientos.

##### 5.3.2.- Desplazamiento en la dirección de abscisas y de ordenadas

Mediante el movimiento del ratón y de la pulsación del botón derecho se realizarán estos movimientos.

##### 5.3.3.- Desplazamiento en altura

Mediante el movimiento del ratón y de la pulsación de ambos botones se realizarán estos movimientos.

#### 5.4.- Nodos

Los nodos se representarán mediante cubos. Al seleccionarlos deberán cambiar de color. Se utilizarán como puntos de referencia para añadir más módulos al diseño.

#### 5.5.- Selección de uniones

Las uniones entre módulos se podrán seleccionar con el ratón. Esto permitirá la adición de módulos en esos puntos.

### 6.- RI *Interfaz para programadores* (Esencial)

#### 6.1.- Utilización

La librería tendrá una API para poder ser utilizada en otras aplicaciones. La interfaz gráfica no será más que una *fachada* para su manejo.

#### 6.2.- API

Los diferentes servicios que deberá ofrecer serán:

##### 6.2.1.- appendNode (nodes, sel, type)

Agregar módulos a la **estructura modular**.

##### 6.2.2.- writeRobot (nodes, modelFile)

Escribir en el fichero suministrado el **grafo estructural**, en formato .rbt.

## 3.5. Diseño de alto nivel

Esta aplicación consta de varios componentes, mostrados en la figura 3.3:

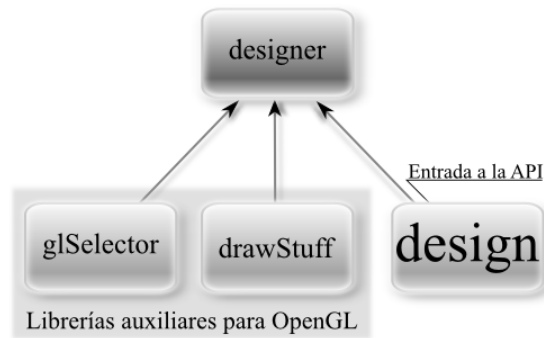


Figura 3.3: Arquitectura de alto nivel del diseñador

Las responsabilidades se reparten de la siguiente manera:

**design:** Módulo central. Es la librería que implementa la API descrita en el apartado 3.4. Permite al usuario construir aplicaciones basadas en el diseñador.

**designer:** Es la fachada del sistema. Módulo principal encargado de la presentación y del uso de las demás librerías para integrar la aplicación. Se ocupa de la interacción con el usuario.

**drawStuff:** Módulo responsable del trabajo de visualización a través de OpenGL. Librería incluida junto con el motor físico ODE que se utiliza en la simulación. Se ha adaptado de forma que pueda ser usada desde Python. Copyright (C) 2001-2003 Russell L. Smith. Bajo licencia GPL.

**glSelector:** Selección de **módulos Y1** en el mundo tridimensional.

Este diseño permite una fuerte reutilización de componentes. La librería *design* es un componente reutilizable para otros proyectos. La fachada puede sustituirse por otra en forma de línea de comandos. Y las librerías auxiliares de OpenGL pueden ser utilizadas en otros proyectos con características de visualización e interacción similares a éste.

## 3.6. Descripción de interfaces entre módulos

### 3.6.1. Interfaz *design*

#### 1.- *appendNode (nodes, sel, type)*

Función encargada agregar un nuevo módulo al nodo seleccionado del **grafo estructural**. En *nodes* se almacena el grafo. *sel* es el nodo seleccionado. *type* determina el tipo de **módulo Y1** a agregar.

**2.- writeRobot (nodes, modelFile)**

Escribe el **grafo estructural** en el formato visto en el apartado 3.3.1. En *nodes* se almacena el grafo. *modelFile* es el nombre del fichero que se va a utilizar.

**3.6.2. Interfaz drawStuff**

Para la utilización de la API en lenguaje C que proporciona la librería drawstuff ha sido necesario realizar un recubrimiento en forma de extensión Python. La interfaz Python es el subconjunto de funciones de la interfaz C necesarias con los mismos nombres y parámetros.

**1.- dsSimulationLoop (width, height, funcs, render)**

Función principal de visualización. Se inicia la librería OpenGL y se configura para la visualización del robot. Se debe suministrar el tamaño de pantalla en *width* y *height*, las funciones de callback en *funcs* y el modo de renderizado requerido en *render*.

**2.- dsSetShadows (enable)**

Habilitar / Deshabilitar el renderizado de sombras.

**3.- dsSetTexture (texture\_number)**

Habilitar / Deshabilitar el renderizado de texturas.

**4.- dsDrawY1 (pos, R, type, wire)**

Esta función se encarga de dibujar los **módulos Y1** a partir de un modelo. Los parámetros son: posición 3D en *pos*, la matriz de rotación en *R*, el tipo de **módulo Y1** en *type* y el tipo de renderizado en *wire*. Este último determina si se superpone un modelo alámbrico.

**5.- dsStop ()**

Detener la visualización. Usada para liberar recursos de la simulación.

**3.6.3. Interfaz glSelector****1.- init (width, height)**

Esta función inicializa la selección de nodos en OpenGL. Se suministran las propiedades de la pantalla, *width* y *height*.

**2.- start (x, y)**

Activa el modo de selección. Se proporcionan las coordenadas, *x* e *y*, donde pulsó el usuario.

**3.- nodes = end ()**

Termina el modo de selección. Devuelve en *nodes* los nodos seleccionados.

## 3.7. Diseño de bajo nivel

En esta sección no se han suministrado los diseños de bajo nivel del módulo drawstuff por no haber sido desarrollados dentro del marco de este proyecto.

### 3.7.1. Módulo *design*

---

**Pseudocódigo 3.1** design: appendNode (nodes, sel, type)

---

```
1: if nodes =  $\Phi$  then    \ * El grafo estructural está vacío *\
2:   nodes = {V, N, N, 1}
3: end if
4: if type = 0 then    \ * Tipo de módulo vertical *\
5:   nodes = nodes  $\cup$  {V, N, N, longitud(nodes)}
6: else if type = 1 then    \ * Tipo de módulo horizontal *\
7:   nodes = nodes  $\cup$  {H, N, N, longitud(nodes)}
8: end if
```

---

---

**Pseudocódigo 3.2** design: writeRobot (nodes, modelFile)

---

```
1: if modelFile =  $\Phi$  then    \ * No hay nombre de fichero *\
2:   modelFile = "robot.rbt"    \ * Establece el nombre por defecto *\
3: end if
4: Abre el fichero
5: for i, modules in nodes do    \ * Recorre cada nodo del grafo estructural *\
6:   for module in modules do    \ * Recorre cada módulo en ese nodo *\
7:     Escribe la salida del módulo Y1 con el formato especificado
8:   end for
9: end for
10: Cierra el fichero
```

---

### 3.7.2. Módulo *glSelector*

---

**Pseudocódigo 3.3** glSelector: init (width, height)

---

```
1: self.width = width    \ * Guarda el valor en el módulo *\
2: self.height = height    \ * Guarda el valor en el módulo *\
```

---

---

**Pseudocódigo 3.4** glSelector: start (x, y)

---

- 1: Dispone a OpenGL en modo selección
  - 2: Establece la proyección en base al punto dado
  - 3: **for** Perspectiva **do**    \\* *Modifica la perspectiva para ver los puntos que hay en el campo de visión* \*\
  - 4:    Busca un punto seleccionado
  - 5: **end for**
  - 6: Almacena en el estado interno de OpenGL el valor de los puntos encontrados
- 

---

**Pseudocódigo 3.5** glSelector: nodes = end ()

---

- 1: Restablece las matrices de proyección
  - 2: Limpia el estado interno de OpenGL
  - 3: **return** Puntos encontrados en la búsqueda    \\* *Obtiene los puntos del estado interno de OpenGL, donde se dejaron* \*\
- 

## 3.8. Descripción de interfaces de usuario

Al arrancar el *designer* aparecen los elementos principales: un Módulo Y1 y un nodo virtual, como se muestra en la figura 3.4.

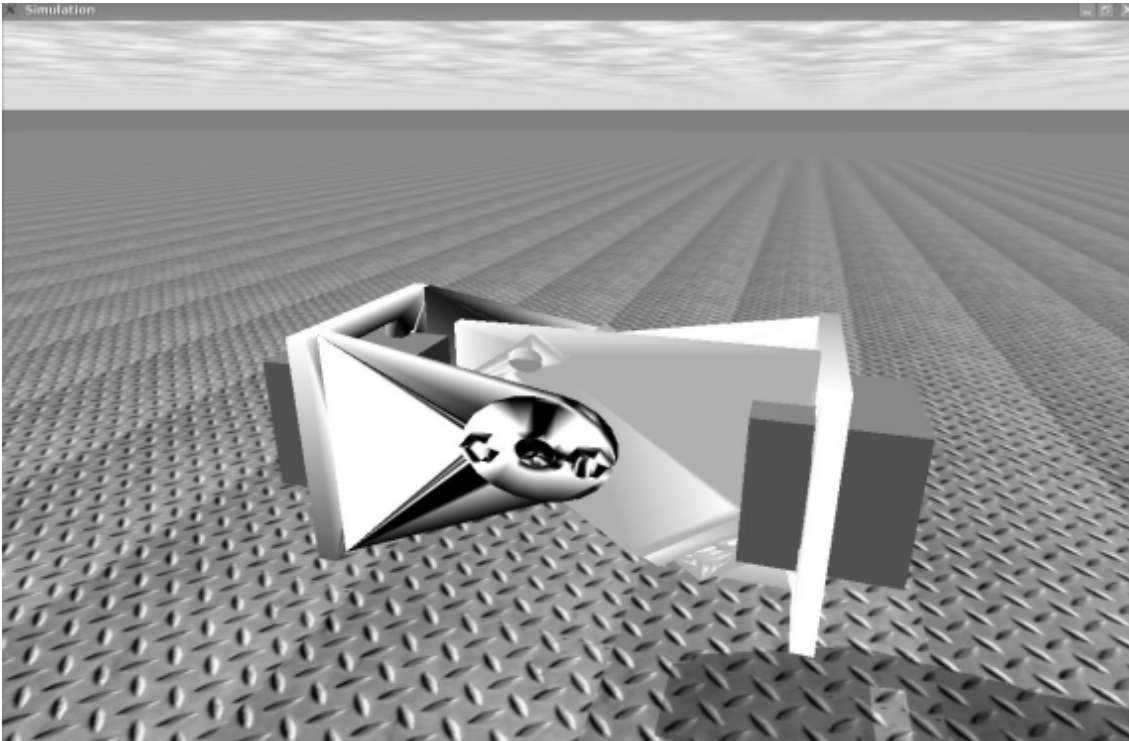


Figura 3.4: Interfaz del diseñador

- **Módulo Y1** renderizado:

El renderizado del **módulo Y1** se ha obtenido a partir de su diseño con la aplicación Blender<sup>1</sup>, exportando a OpenGL.

- Nodo virtual:

Son las “herramientas” del diseñador. Cada **módulo Y1** tiene uno a cada lado y permite añadir más **módulos Y1**.

Utilizando el ratón, el usuario puede cambiar la posición de la cámara. En la figura 3.5 se ve lo mismo que en la 3.4 pero desde otro punto de vista.

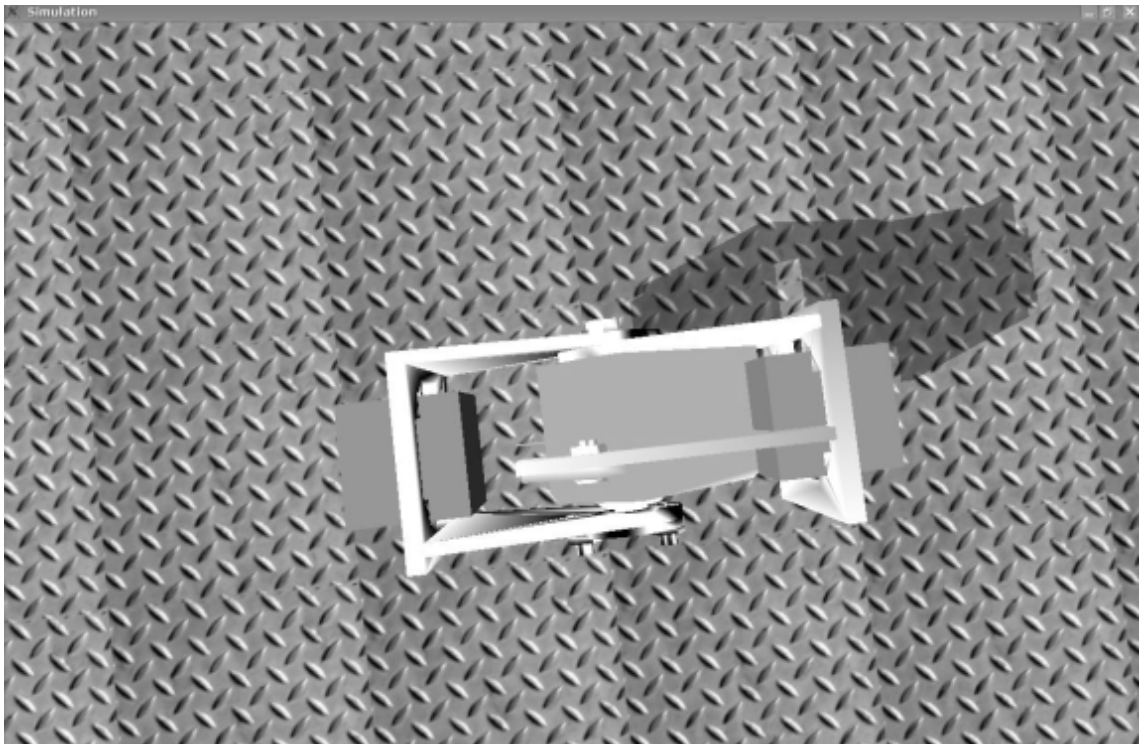


Figura 3.5: Interfaz del diseñador con otro punto de vista

Al pasar el cursor encima de los nodos se seleccionan y cambian su color. El usuario puede añadir bien el módulo vertical o bien uno horizontal. En la figura 3.6 se muestran dos estructuras. La de la izquierda por dos **módulos Y1** con la misma orientación. La de la derecha con orientaciones diferentes.

---

<sup>1</sup>Software de diseño tridimensional libre.

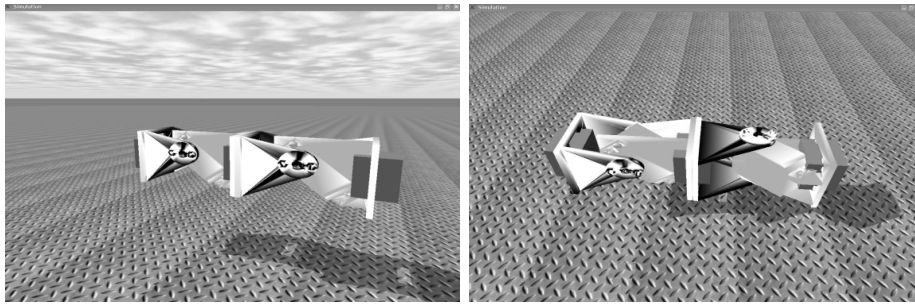


Figura 3.6: Diferentes orientaciones de los módulos

Al ir añadiendo **módulos Y1** a los diferentes nodos, se van obteniendo las configuraciones finales. En la figura 3.7 se observan nodos con 4 y 5 aristas.

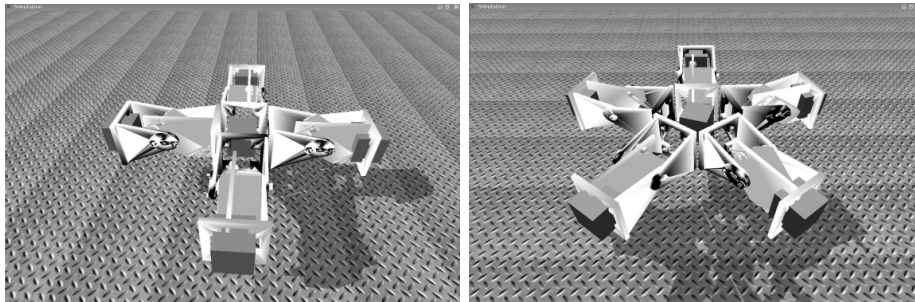


Figura 3.7: Nodos con múltiples aristas

Para finalizar, el teclado ofrece los siguientes comandos:

- *w*: Guardado de configuraciones.
- *a*: Adición de un módulo vertical.
- *b*: Adición de un módulo horizontal.

## 3.9. Pruebas

### 3.9.1. Pruebas unitarias

#### 1.- Módulo design



1.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

1.2.- **Llamada:** `./design.py`

1.3.- Entrada:  $\Phi$

1.4.- Salida esperada:

■ Salida estándar:

```

1 Autotesting...
2 Module added
3 Module type 0 added at node 0
4 Module type 1 added at node 0
5 Module type 1 added at node 1
6 Module type 1 added at node 1
7 Module type 1 added at node 2
8 Module type 0 added at node 3
9 Module type 1 added at node 1
10 Config. escrita en "test.rbt"
11 Robot writed at "test.rbt"
```

■ *robot.rbt*:

```

1 0 VNN 1
2 0 VNN 2
3 0 HNN 3
4 1 HNN 4
5 1 HNN 5
6 2 HNN 6
7 3 VNN 7
8 5 VNN 8
```

### 3.9.2. Pruebas de integración

La estrategia que se va a utilizar para probar la integración de los diferentes módulos será de arriba a abajo. Es decir, partiendo del módulo principal, se irán integrando los demás módulos. Para los módulos aún no integrados se utilizarán *stubs* que los sustituyan.

1.- Interfaz drawStuff



1.1.- **Propósito:** Integración entre *designer* y *drawstuff*. Las funciones a integrar serán:

1.1.1.- *dsSimulationLoop* (*width*, *height*, *funcs*, *render*)



1.1.2.- *dsSetShadows* (*enable*)



1.1.3.- *dsSetTexture* (*texture\_number*)



1.1.4.- *dsDrawY1* (*pos*, *R*, *type*, *wire*)



1.1.5.- *dsStop* ()



1.2.- **Llamada:** *./designer*

1.3.- **Entrada:**  $\Phi$

1.4.- **Salida esperada:** En esta prueba la única salida esperada es la visualización correcta de todos los módulos. Para ello se realiza una prueba añadiendo módulos sobre la interfaz y se comprueba visualmente que todo es correcto.

2.- Interfaz glSelector



2.1.- **Propósito:** Integración entre *designer* y *glSelector*. Las funciones a integrar serán:

2.1.1.- *init (width, height)*



2.1.2.- *start (x, y)*



2.1.3.- *end ()*



2.2.- **Llamada:** *./designer*

2.3.- **Entrada:**  $\Phi$

2.4.- **Salida esperada:** En este caso, además de esperar lo mismo que en la prueba anterior, se debe comprobar que la selección de nodos funciona correctamente.

3.- Interfaz design



3.1.- **Propósito:** Integración entre *designer* y *design*. Las funciones a integrar serán:

3.1.1.- *appendNode (nodes, sel, type)*



3.1.2.- *writeRobot (nodes, modelFile)*



3.2.- **Llamada:** *./designer*

3.3.- **Entrada:**  $\Phi$

3.4.- **Salida esperada:**

■ **Salida estándar:**

1 Config. escrita en ‘‘robot.rbt’’

■ **robot.rbt:**

1 0 VNN 1

2 0 VNN 2

3 0 HNN 3

4 1 HNN 4

5 1 HNN 5

6 2 HNN 6

7 3 VNN 7

8 5 VNN 8

### 3.9.3. Pruebas de sistema

En este apartado se presentan las pruebas de **caja negra** utilizadas para cubrir los requisitos especificados. Por la interrelación de los módulos, en todas las pruebas se van a probar todos los requisitos:

1.- RF *Capacidad de diseño* (Esencial)



2.- RF *Comandos* (Esencial)



3.- RF *Formato de salida* (Esencial)



4.- RU *Interfaz de línea de comandos* (Esencial)



5.- RU *Interfaz visual* (Esencial)



6.- RI *Interfaz para programadores* (Esencial)



1.- Configuración lineal de 8 segmentos

1.1.- **Propósito:** Comprobar los requisitos

1.2.- **Llamada:** `./designer.py hypercube.rbt`

1.3.- **Entrada:**  $\Phi$

1.4.- **Salida esperada:**

▪ **Salida estándar:**

1 Config. escrita en ‘‘hypercube.rbt’’

▪ ***hypercube.rbt:***

1 0 VNN 1

2 1 HNN 2

3 2 VNN 3

4 3 HNN 4

5 4 VNN 5

6 5 HNN 6

7 6 VNN 7

8 7 HNN 8

1.5.- **Pantallazo:**

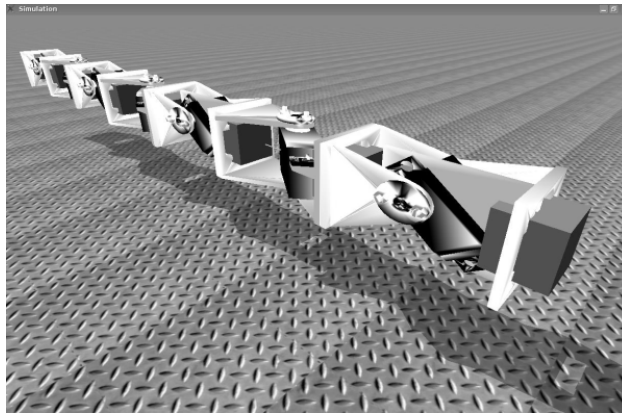


Figura 3.8: Estructura creada para la prueba 1

## 2.- Configuración estrella de 5 segmentos

2.1.- **Propósito:** Comprobar los requisitos

2.2.- **Llamada:** `./designer.py star.rbt`

2.3.- **Entrada:**  $\Phi$

2.4.- **Salida esperada:**

- **Salida estándar:**

- 1 Config. escrita en `“star.rbt”`

- ***star.rbt*:**

- 1 0 VNN 1

- 2 1 VNN 2

- 3 1 VNN 3

- 4 1 VNN 4

- 5 1 VNN 5

2.5.- **Pantallazo:**

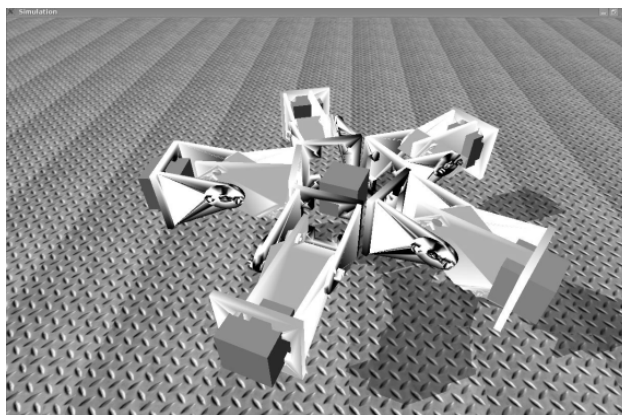


Figura 3.9: Estructura creada para la prueba 2

### 3.- Configuración cuadrúpeda

3.1.- **Propósito:** Comprobar los requisitos

3.2.- **Llamada:** `./designer.py quad.rbt`

3.3.- **Entrada:**  $\Phi$

3.4.- **Salida esperada:**

- **Salida estándar:**

1 Config. escrita en ‘‘quad.rbt’’

- ***quad.rbt:***

1 0 VNN 1

2 1 HNN 2

3 2 HNN 3

4 2 HNN 4

5 3 HNN 6

6 3 HNN 7

7 4 VNN 5

8 6 VNN 9

9 7 VNN 8

3.5.- **Pantallazo:**

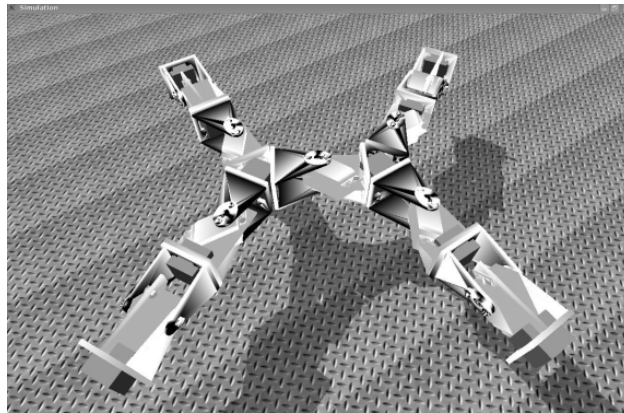


Figura 3.10: Estructura creada para la prueba 3

### 3.10. Resumen

La herramienta *designer* realiza la función de asistente de diseño para poder generar las **estructuras modulares** de los robots en forma de **grafo estructural**.

La entrada del sistema es únicamente la interacción con el usuario, que a través de una interfaz gráfica, va introduciendo sus ideas y diseños a la **estructura modular**.

La salida del sistema es un fichero de texto con el formato descrito en el apartado 3.3.1.

Gracias a su diseño modular, se pueden reutilizar sus componentes. La librería principal, *design*, puede ser incorporada a otros proyectos, de forma que las salidas siguen siendo compatibles con el simulador.

Se ha utilizado la librería *drawstuff* y se ha creado la librería *glSelection*. Ambas, librerías de soporte para los gráficos OpenGL que se utilizan en la visualización. Con estos elementos se crea la fachada del sistema que permite su uso de forma gráfica.

---

Parte II

## Generator: Generación de secuencias de movimiento

---



# Secuencias de movimiento

---

## 4.1. Introducción

En este capítulo se aborda el problema de la generación automática de secuencias de movimiento para conseguir la locomoción de los robots modulares. Se trata de un problema muy amplio. Sin embargo, este estudio se restringe a movimientos periódicos, de régimen permanente y estáticamente estables.

La locomoción estáticamente estable es aquella en la que la proyección del centro de gravedad se encuentra en todo momento en el interior del polígono formado por los puntos de apoyo.

Existen diferentes alternativas para la síntesis de movimientos. Un enfoque interesante es el control bioinspirado. Consistente en imitar los modelos animales. En este proyecto se utilizará un modelo muy simplificado de los generadores centrales de patrones (CPG's) que se encuentran en el sistema nervioso de los animales.

Dentro del capítulo se introducirá el modelo bioinspirado y la idea de funciones generadoras. Después se verá cómo a partir de estas funciones se generan las secuencias de movimiento y cómo quedan representadas mediante matrices. Finalmente se presentarán un tipo simplificado de funciones generadoras: las sinusoidales. Con ellas se han obtenido muy buenos resultados en la locomoción de topologías de una dimensión.

## 4.2. Inspiración biológica: CPG's

En el sistema nervioso de los animales existen unas cadenas de neuronas denominadas generadores centrales de patrones (CPG's), encargadas de la generación de actividades rítmicas. Los CPG's controlan el movimiento de las alas de insectos y pájaros, el nado de los peces o los movimientos peristálticos del esófago<sup>1</sup>, entre otros. Los ritmos pueden ser iniciados, modificados o modulados mediante estímulos.

Un ejemplo típico es el nado de las lampreas[lam]. A lo largo de la espina dorsal se sitúan los CPG's, cada uno conectado con el siguiente y el anterior. Cada CPG se

---

<sup>1</sup>Movimientos oscilantes que ayudan al bolo alimenticio a llegar al estómago

encarga de controlar las oscilaciones de un grupo de músculos. El efecto global es la aparición de ondas que recorren el cuerpo de la lamprea consiguiendo su propulsión.

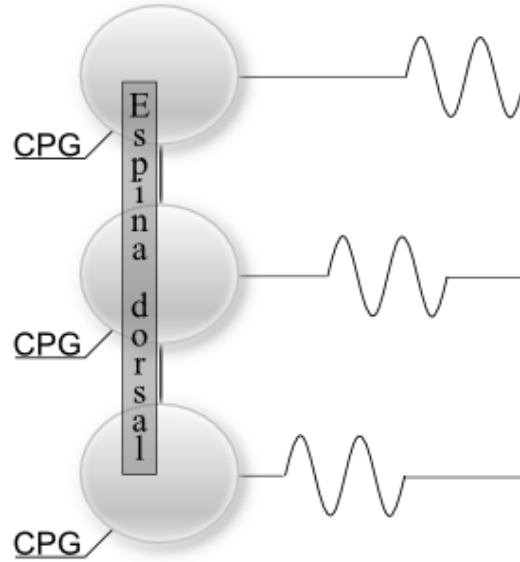


Figura 4.1: Esquema de la espina dorsal de un animal

En la figura 4.1 se puede apreciar esquemáticamente como cada neurona de la espina dorsal genera una salida coordinada con las demás, pero independiente.

El modelo de control utilizado en este proyecto está basado en generadores que controlan la posición de cada uno de los módulos del robot. Si el robot tiene  $M$  módulos, habrá  $M$  generadores, cada uno actuando sobre un servo. Consiguiendo así una mayor flexibilidad. Este concepto se ilustra en la figura 4.2.

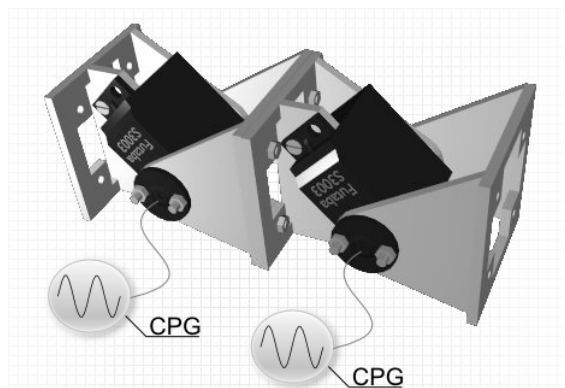


Figura 4.2: Esquema de aplicación de CPG's a un robot modular

Cada uno de los generadores se describirán matemáticamente mediante una

función periódica  $g_i(t)$ . Estas funciones indican la posición del servo a lo largo del tiempo. Se denominan funciones generadoras.

El problema de cómo generar una secuencia de movimiento para que un robot de  $M$  módulos se pueda desplazar se reduce a encontrar las  $M$  funciones generadoras y sus parámetros (amplitud, periodo, ...). Ahora se trata de un problema de búsqueda en un espacio de soluciones. Como se muestra en el capítulo 8, una de las maneras de realizar estas búsquedas es por medio de algoritmos genéticos. El MRSuite ha sido diseñado para poderse ampliar fácilmente para implementar estas u otras técnicas de búsqueda.

### 4.3. Funciones generadoras y matrices de movimiento

Las funciones generadoras son funciones periódicas que determinan la posición del servo para cada instante  $t$ . Dado que el rango de movimiento del servo es entre  $-90$  y  $90$  grados, estas funciones se definen como  $g_i : \mathbb{R} \rightarrow [-90, 90]$ . Cada función representa el modelo simplificado del CPG.

Las secuencias de movimiento se consiguen muestreando las funciones generadoras en un periodo. Sea  $S$  el número de muestras a emplear, cada función  $g_i$  se puede representar mediante una lista de  $S$  elementos:  $[g_i(0), g_i(1), \dots, g_i(S-1)]$ . Esta lista indica las posiciones del servo  $i$  para esos  $S$  instantes de tiempo.

Dado que en total hay  $M$  funciones generadoras y cada una de ellas contiene  $S$  muestras, las secuencias de movimiento las representaremos por medio de una matriz de movimiento, de  $M$  filas  $\times$   $S$  columnas.

En la matriz de movimiento, la fila  $i$ -ésima está asociada a la función generadora  $g_i$ , que a su vez es la que controla el servo  $i$ -ésimo. Por tanto, esta fila contiene las posiciones que tiene que alcanzar el servo  $i$ -ésimo durante un periodo. La columna  $j$ -ésima se asocia al instante de tiempo  $j$ -ésimo e indica la posición en la que se encuentran los  $M$  servos en ese instante.

### 4.4. Funciones generadoras sinusoidales

Las funciones generadoras más simples son las sinusoidales. Están descritas por la ecuación 4.1. Al discretizarlas se obtiene la ecuación 4.2.

$$g_i(t) = A_i \sin(2\pi t/T_i + \Phi_i) + O_i \quad (4.1)$$

$$g_i(t) = A_i \sin(2\pi n/S_i + \Phi_i) + O_i \quad (4.2)$$

En la figura 4.3 se aprecian algunos de los parámetros más importantes de las funciones generadoras sinusoidales:

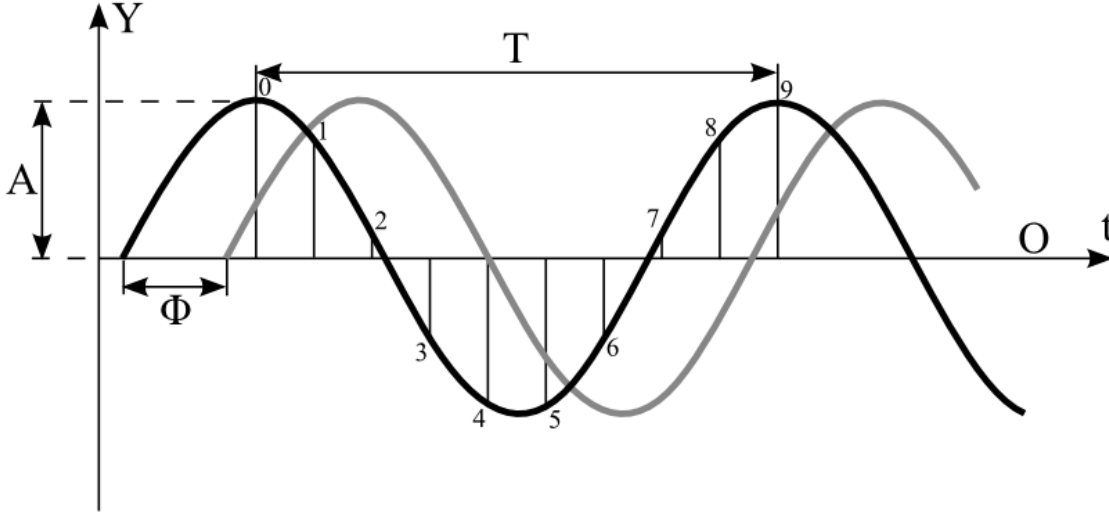


Figura 4.3: Parámetros de una función generadora sinusoidal

Los parámetros presentados son:

- $A_i$ : Amplitud máxima, en valor absoluto, de la función generadora. Dicho parámetro determina los movimientos máximos que efectúa la función y está limitada por las características mecánicas del servo utilizado. Para el caso de los módulos Y1,  $A_i$  no puede superar el valor de 90.
- $T_i$ : Representa el periodo o distancia entre dos repeticiones del patrón. Esta longitud es la que se fracciona a la hora de muestrear la función para obtener los valores finales de las **secuencias de movimiento**. En la figura 4.3 se observan los números del 0 al 9 que se corresponden con las 10 muestras de un periodo.
- $\Phi_i$ : Desfase entre funciones generadoras. Este es uno de los parámetros claves para conseguir una buena coordinación entre los módulos.
- $O_i$ : Valor medio de la función  $g_i$ .

De esta forma, mediante funciones matemáticas a modo de patrones, es posible generar las **secuencias de movimiento** simplemente muestreando estas funciones, del mismo modo que lo haría un CPG en el cuerpo de un ser vivo.

Si ahora se retorna al mundo de la robótica, aplicando esta teoría de patrones a todas las articulaciones de un robot, se obtiene como resultado una serie de  $M$  **secuencias de movimiento**, una para cada una de las  $M$  articulaciones. Cada

una de estas series está formada por las  $S$  muestras de la **función generadora**. Por lo tanto, como resultado se tiene una matriz de datos, en la que las  $M$  filas son cada una de las **secuencias de movimiento** y cada una de las  $S$  columnas son la rotación de las articulaciones para cada uno de los momentos.

## 4.5. Resumen

Los CPG's son cadenas de neuronas que en los seres vivos generan patrones de movimiento para activar los músculos. De la misma manera, se desea generar secuencias de movimiento para los robots modulares. La forma que tienen las secuencias de movimiento en los robots son funciones generadoras. Un tipo muy utilizado de estas funciones son las funciones generadoras sinusoidales.

Muestreando dichas funciones generadoras se obtienen las **secuencias de movimiento del robot**. Como resultado del muestreo se obtienen  $M$  **secuencias de movimiento**, una por articulación, y cada una de ellas formada por los  $S$  valores muestreados.

Para organizar toda la información se construye una matriz  $M \times S$ , donde las filas son **secuencias de movimiento** y las columnas son valores del muestreo de la **función generadora**.

### 5.1. Introducción

*Generator* es una aplicación de síntesis de **secuencias de movimiento** a partir de **funciones generadoras** definidas por el usuario. Como resultado de la síntesis se obtiene una **matriz de movimiento** de dimensiones  $M \times S$ . Siendo  $M$  el número de articulaciones del robot y  $S$  el total de muestras. Dicha **matriz de movimiento** será leída bien por el simulador o bien por las aplicaciones del usuario.

### 5.2. Definición del problema

Se debe diseñar una aplicación que sintetice la **matriz de movimiento** a partir de las **funciones generadoras** y sus parámetros específicos.



Figura 5.1: Esquema de funcionamiento del generador

En la figura 5.1 se muestra el esquema de funcionamiento del generador. La entrada son las **funciones generadoras** con sus parámetros y la salida las **secuencias de movimiento** en forma de matriz.

Uno de los problemas es la especificación de las **funciones generadoras** de la forma más genérica posible. El objetivo es que el usuario pueda utilizar cualquier función matemática y definir sus propios parámetros. Para lograrlo se ha diseñado el formato .cpg, descrito en el apartado 5.3.1.

En el fichero constarán diferentes fórmulas matemáticas con sus parámetros. Después, por cada articulación se especificará la fórmula a usar y los parámetros concretos que se van a emplear.

## 5.3. Diseño de la solución

Es necesario definir dos formatos de fichero: Uno para las funciones generadoras con sus parámetros (formato .cpg) y el otro para la **matriz de movimiento** (formato .mvm).

### 5.3.1. Formato .cpg

Este formato permite la especificación de las **funciones generadoras** utilizando cualquier fórmula matemática y con cualquier número de parámetros. Primero se definen las funciones genéricas y luego se instancian dando valores a sus parámetros.

El formato es el siguiente:

- 1: #<Comentario>
- 2: # Definición de las funciones genéricas
- 3: <Nombre de la **función generadora**>: <Lista de  $N$  parámetros>: <Fórmula matemática>
- 4:
- 5: # Instanciación de las funciones
- 6: <Nombre de la **función generadora**> <Valor del parámetro 1 >  $\dots$  <Valor del parámetro  $N$  >
- 7: <Nombre de la **función generadora**> <Valor del parámetro 1 >  $\dots$  <Valor del parámetro  $N$  >
- 8:  $\dots$

Además de los parámetros definidos por el usuario, la aplicación proporciona la variable  $t$  y la constante *samples*. La variable  $t$  hace referencia a la variable independiente. La constante *samples* representa el número de muestras.

La ventaja de este formato es su flexibilidad para la definición de las **funciones generadoras**.

### 5.3.2. Formato .mvm

El formato .mvm permite especificar las matrices de movimiento, de  $M$  filas y  $S$  columnas. Cada línea del fichero se corresponde con una fila de la matriz.

- 1: <Fila 1 Columna 1 > <Fila 1 Columna 2 >  $\dots$  <Fila 1 Columna  $S$  >
- 2: <Fila 2 Columna 1 > <Fila 2 Columna 2 >  $\dots$  <Fila 2 Columna  $S$  >
- 3:  $\dots$
- 4: <Fila  $M$  Columna 1 > <Fila  $M$  Columna 2 >  $\dots$  <Fila  $M$  Columna  $S$  >

Los valores escritos en cada fila deben estar separados por espacios en blanco.

## 5.4. Especificación de requisitos

### 1.- RF *Formato de entrada* (Esencial)

La entrada a la aplicación se efectuará mediante el uso de ficheros de texto. El formato de fichero será el formato .cpg definido en el apartado 5.3.1.

### 2.- RF *Generación de secuencias* (Esencial)

El generador creará cada una de las filas de la **matriz de movimiento** partiendo de las **funciones generadoras**, dando valores a la variable  $t$  desde 0 hasta  $samples - 1$ .

### 3.- RF *Formato de salida* (Esencial)

#### 3.1.- Ficheros de texto

El fichero de salida del generador serán **matrices de movimiento** que podrán ser leídas directamente por el simulador, por lo que el formato deberá ser compatible entre ambas aplicaciones.

#### 3.2.- Contenido

Cada línea del fichero contendrá una fila de la **matriz de movimiento**. Cada fila estará compuesta de  $samples$  valores enteros en el rango  $[-180, 180]$  separados por espacios.

#### 3.3.- Representación

La **matriz de movimiento** se podrá representar mediante tres formatos: 3.3.1.- nativo del simulador, .mvm definido en el apartado 5.3.2 3.3.2.- *script* para Octave 3.3.3.- lenguaje C .

### 4.- RU *Interfaz de línea de comandos* (Esencial)

#### 4.1.- Argumentos

La interfaz con la línea de comando tendrá los siguientes argumentos en ese orden:

##### 4.1.1.- Fichero de entrada (Obligatorio)

##### 4.1.2.- Fichero de salida (Opcional)

##### 4.1.3.- Opciones

- $-s<samples>$ : Determina las muestras que se van a tomar. Si no se especifica serán 10 muestras.
- $-c$ : Activa el formato de salida para el lenguaje C.
- $-octave$ : Activa el formato de salida para Octave.
- $-mrs$ : Activa el formato de salida para el simulador, definido en el apartado 5.3.2.

#### 4.2.- Mensajes

Los mensajes serán de diferentes tipos:

##### 4.2.1.- Notas (Note)

En caso de información.

##### 4.2.2.- Avisos (Warning)

En caso de un problema por el que no haya que detener la ejecución.

##### 4.2.3.- Errores (Error)

Por problemas que no puedan ser resueltos.

### 5.- RI *Interfaz para programadores* (Esencial)

#### 5.1.- Utilización

La librería tendrá una API para permitir su uso desde otras aplicaciones.

La interfaz de línea de comandos será sólo una *fachada* para la librería.

#### 5.2.- API

Los diferentes servicios que deberá ofrecer serán:

##### 5.2.1.- ParseFile (file)

Función encargada de leer el fichero de entrada, será el inicio del proceso.

##### 5.2.2.- generateMatrix (cpgs, s)

El motor sintetizador de las **secuencias de movimiento**. Se alimentará de lo que lea el parser. Generará la **matriz de movimiento**.

##### 5.2.3.- mrsOutput (seq, fd)

Generador de la secuencia al formato del simulador.

##### 5.2.4.- cOutput (seq, fd)

Generador de la secuencia al formato del lenguaje C.

##### 5.2.5.- octaveOutput (seq, fd)

Generador de la secuencia al formato de Octave.

## 5.5. Diseño de alto nivel

La arquitectura diseñada se muestra en la figura 5.2. Consta de tres módulos:

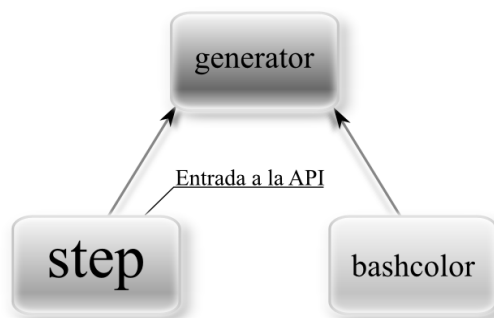


Figura 5.2: Arquitectura de alto nivel del generador

Se puede apreciar la composición modular tal y como se ha descrito en los párrafos anteriores. Módulo a módulo las responsabilidades quedarían repartidas de la siguiente forma:

**step:** Módulo principal que implementa toda la funcionalidad del generador. Se trata de una librería que se puede reutilizar para construir aplicaciones basadas en el generador. Para utilizarlo sólo es necesario importarlo en el proyecto y utilizar la interfaz descrita en el apartado 5.4 de requisitos.

**generator:** Módulo principal encargado de la lectura de parámetros y la generación de los ficheros de salida. Es la fachada del sistema. Realiza llamadas a los demás módulos.

**bashcolor:** Módulo encargado de realizar la mensajería coloreada.

Este diseño permite la reutilización de componentes. El módulo *step* puede ser utilizado en las aplicaciones de usuario y la fachada, *generator*, sustituida por otra como una interfaz gráfica.

## 5.6. Descripción de interfaces entre módulos

### 5.6.1. Interfaz *step*

- $cpgs = ParseFile(file)$

Lee el fichero *file* y obtiene las **funciones generadoras** y los parámetros específicos. Devuelve una lista de funciones, *cpgs*. Cada una de estas funciones estará compuesta por la fórmula en formato nativo Python y sus parámetros en formato numérico.

- $seq = generateMatrix(cpgs, s)$

Crea las **secuencias de movimiento** a partir de las **funciones generadoras** y sus parámetros, *cpgs*. El parámetro *cpgs* contiene una lista de **funciones generadoras**. El parámetro *s* representa el número de muestras, la variable *samples*. Devuelve la **matriz de movimiento**, *seq*.

- *mrsOutput (seq, fd)*

Escribe la **matriz de movimiento** en fomato nativo del simulador, *.mvm*. Las entradas son el parámetro *seq* de las **secuencias de movimiento** y el descriptor de fichero *fd*.

- *cOutput (seq, fd)*

Escribe la **matriz de movimiento** usando el fomato de tabla del lenguaje C. Las entradas son el parámetro *seq* de las **secuencias de movimiento** y el descriptor de fichero *fd*.

- *octaveOutput (seq, fd)*

Escribe la **matriz de movimiento** en fomato de *script* para Octave. Las entradas son el parámetro *seq* de las **secuencias de movimiento** y el descriptor de fichero *fd*.

## 5.6.2. Interfaz *bashcolor*

### 1.- *msg (L)*

Imprime mensajes coloreados por pantalla. La lista *L* estará compuesta de tuplas con el color y la cadena a imprimir. La lista de colores disponibles es rojo, azul, cyan, gris, verde, morado, blanco, negro, marron, amarillo. Para todos excepto los cuatro últimos existe una versión en mayúculas con un color más claro.

## 5.7. Diseño de bajo nivel

### 5.7.1. Módulo *step*

---

**Pseudocódigo 5.1** step: cpgs = ParseFile (file)

---

```
1: Se abre el fichero file y se leen todas sus líneas en la variable lines
2: funcs =  $\Phi$ 
3: l =  $\Phi$ 
4: for line in lines do
5:   if line no contiene # then   \* No es un comentario *\
6:     if line contiene : then   \* Función *\
7:       Separar la línea mediante :   \* Se separa el nombre de los parámetros y de la
                                         función en sí *\
8:       Limpiar de espacios y tabulaciones cada parte   \* Quitar todo lo que sobre
                                                         *\
9:       Generar una función compilada de la información leída   \* A partir de la
                                         cadena de caracteres, se crea un objeto de código *\
10:      funcs = funcs  $\cup$  esta función   \* Se agrega a la lista de funciones disponibles
                                         *\
11:    else   \* Parámetros de función *\
12:      Separar la línea por espacios   \* Se separa la información *\
13:      l = l  $\cup$  (función indicada y parámetros)   \* Se asocia dicha información con
                                         la función creada anteriormente *\
14:    end if
15:  end if
16: end for
17: return l
```

---

---

**Pseudocódigo 5.2** step: seq = generateMatrix (functions, s)

---

```
1: seq =  $\Phi$ 
2: for (f, a) in functions do
3:   artic =  $\Phi$ 
4:   for j in 0...samples - 1 do
5:     params = a  $\cup$  j   \* Se agrega a los parámetros el punto de muestreo *\
6:     artic = artic  $\cup$  f(params)   \* Se realiza el cálculo del valor *\
7:   end for
8:   seq = seq  $\cup$  artic   \* Se agrega la serie de la articulación a la secuencia *\
9: end for
10: return seq
```

---

---

**Pseudocódigo 5.3** step: mrsOutput (seq, fd)

---

```

1: for module in seq do
2:   Convertir a cadenas de caracteres la serie de valores del módulo
3:   Imprimir a fd cada serie con espacios entre valores en una línea
4: end for

```

---



---

**Pseudocódigo 5.4** step: cOutput (seq, fd)

---

```

1: Imprimir a fd el inicio de cabecera
2: lines =  $\Phi$ 
3: for module in seq do
4:   Generar la línea en formato C entre llaves
5:   lines = lines  $\cup$  nueva línea   \ * Agregar la nueva línea a la lista de líneas final * \
6: end for
7: Imprimir a fd las lista de líneas final
8: Imprimir a fd la llave final de cierre

```

---



---

**Pseudocódigo 5.5** step: octaveOutput (seq, fd)

---

```

1: for i, module in enumerar (seq) do   \ * i es el índice, module cada serie * \
2:   Convertir a cadenas de caracteres la serie de valores del módulo
3:   Escribir a fd la serie de valores en formato Octave entre corchetes
4: end for
5: Imprimir el resto del código del script Octave con el formato adecuado

```

---

### 5.7.2. Módulo *bashcolor*

---

**Pseudocódigo 5.6** bashcolor: msg (L)

---

```

1: cadena =  $\Phi$ 
2: for (color, texto) in L do
3:   codigoColor = diccionarioColores.obtener (color)   \ * Se obtiene el código de color
   indicado * \
4:   cadena = cadena  $\cup$  (texto con codigoColor)   \ * Se añade a la cadena final el texto
   coloreado * \
5: end for
6: comando = "echo -e cadena"   \ * Se agrega la parte del comando a ejecutar * \
7: system (comando)   \ * Se ejecuta el comando * \

```

---

Esta función va recorriendo los pares (color, texto) para ir generando una cadena válida para ser ejecutada en un terminal tipo Bash. Finalmente se genera el comando propiamente dicho y se ejecuta haciendo una llamada al sistema operativo.

## 5.8. Pruebas

### 5.8.1. Ficheros de prueba

Para realizar las pruebas se han empleado los ficheros en formato .cpg listados a continuación. Para mayor claridad, las expresiones matemáticas se han dividido en varias líneas.

#### quad.cpg

```
1  # Fichero de prueba del generador 1. Robot cuadrúpedo de 9 articulaciones
2
3  # Definición de la función generadora
4  f: A, Phi, t: int (A * sin ((2 * pi * t) / samples + DEG_TO_RAD * Phi))
5
6  # Asignación de parámetros a cada función
7  # Función A    Phi
8  # ----- -    ---
9      f          20  105
10     f          20   5
11     f          20 340
12     f          20 165
13     f          20 250
14     f          20 210
15     f          20 280
16     f          20 120
17     f          20 150
```

#### rotating.cpg

```
1  # Fichero de prueba del generador 2. Robot lineal de 8 articulaciones
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- --    -    --- --
10     f          60  1     0  0
11     f          60  1    90  0
12     f          60  1     0  0
13     f          60  1    90  0
```

```

14    f      60  1    0  0
15    f      60  1   90  0
16    f      60  1    0  0
17    f      60  1   90  0

```

#### simple.cpg

```

1  # Fichero de prueba del generador 3. Robot lineal de 8 articulaciones
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Funcion A    T    Phi    O
9  # ----- -    -    ---  --
10 f      30  1     0  0
11 f      30  1   120  0
12 f      30  1   240  0
13 f      30  1     0  0
14 f      30  1   120  0
15 f      30  1   240  0
16 f      30  1     0  0
17 f      30  1   120  0

```

#### star.cpg

```

1  # Fichero de prueba del generador 4. Robot lineal de 8 articulaciones
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -    -    --- --
10 f      30  1     0  0
11 f      30  1    60  0
12 f      30  1   120  0
13 f      30  1   180  0
14 f      30  1   240  0

```

## 5.8.2. Pruebas unitarias

### 1.- Módulo *step*



1.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo. Utilizando todas sus funciones.

1.2.- **Llamada:** `./step.py`

1.3.- **Entrada:**  $\Phi$

1.4.- **Salida esperada:** El módulo deberá imprimir por salida estándar el siguiente texto.

```
1 Autotesting...
2 f = lambda x, y, t: x + y + t
3
4 * generateMatrix ([(f, (2, 3))], 10)
5 [[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]]
6
7 * MRS output
8 5 6 7 8 9 10 11 12 13 14
9
10 * C output
11 static secPP seq = {
12     {5, 6, 7, 8, 9, 10, 11, 12, 13, 14}
13 };
14
15 * Octave output
16 f1 = [5, 6, 7, 8, 9, 10, 11, 12, 13, 14];
17 t = [0 : 1 : 9];
18 plot (t, f1 )
19 grid on;
20 pause;
```

### 2.- Módulo *generator*



2.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo. Para ello es necesario alimentarlo con un fichero.

2.2.- **Llamada:** `./generator star.cpg`

2.3.- **Entrada:** El fichero *star.cpg* descrito en la página 66.

2.4.- **Salida esperada:**

- **Salida estándar:**

- 1 generator: Warning: No output file. Using default.

- ***sequence.mvm*:**

```

1  113 20 109 88 -179 -128 83 -117 -169 -123
2  -67 -141 -139 119 -61 78 102 -72 -159 -159
3   85 -18 -88 -102 -49 108 55 -128 -129 151
4  -76 -101 140 -74 18 164 -13 153 143 162
5   25 160 56 -99 29 43 65 -147 137 55

```

### 3.- Módulo bashcolor



3.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

3.2.- **Llamada:** `./bashcolor.py`

3.3.- **Entrada:**  $\Phi$

3.4.- **Salida esperada:** El módulo deberá imprimir por salida estándar el siguiente texto.

**Nota:** En realidad, cada línea debería estar impresa en el color indicado, pero se ha optado por imprimir todo en negro ya que este documento es en blanco y negro.

```

1  Autoprueba del módulo bashcolor.py
2  MORADO
3  morado
4  gris
5  blanco
6  rojo
7  marron
8  AZUL
9  negro
10 VERDE
11 ROJO
12 azul
13 GRIS
14 cyan
15 CYAN
16 verde
17 amarillo
18 NoCol

```

### 5.8.3. Pruebas de integración

La estrategia que se va a utilizar para probar la integración de los diferentes módulos será de arriba a abajo. Es decir, partiendo del módulo principal, se irán integrando los demás módulos. Para los módulos aún no integrados se utilizarán *stubs* que los sustituyan.

#### 1.- Interfaz bashcolor



1.1.- **Propósito:** Integración entre *generator* y *bashcolor*. Las funciones a integrar serán:

1.1.1.- *msg* (*L*)



1.2.- **Llamada:** *./generator simple.cpg*

1.3.- **Entrada:** El fichero *simple.cpg* descrito en la página 66.

1.4.- **Salida esperada:** El módulo deberá imprimir por salida estándar el siguiente texto.

```
1 generator: Warning: No output file. Using default.
```

2.- *Interfaz step*



2.1.- **Propósito:** Integración entre *generator* y *step*. Las funciones a integrar serán:

2.1.1.- *cpgs* = *ParseFile* (*file*)



2.1.2.- *seq* = *generateMatrix* (*cpgs*, *s*)



2.1.3.- *mrsOutput* (*seq*, *fd*)



2.1.4.- *cOutput* (*seq*, *fd*)



2.1.5.- *octaveOutput* (*seq*, *fd*)



2.2.- **Llamada:** *./generator quad.cpg*

2.3.- **Entrada:** El fichero *quad.cpg* descrito en la página 65.

2.4.- **Salida esperada:**

■ **Salida estándar:**

```
1 generator: Warning: No output file. Using default.
```

■ ***sequence.mvm*:**

```
1 19 12 1 -10 -18 -19 -12 -1 10 18
2 1 13 19 18 10 -1 -13 -19 -18 -10
3 -6 5 15 19 16 6 -5 -15 -19 -16
4 5 -7 -16 -19 -15 -5 7 16 19 15
5 -18 -19 -12 0 11 18 19 12 0 -11
6 -10 -18 -19 -13 -2 10 18 19 13 2
7 -19 -13 -2 9 17 19 13 2 -9 -17
8 17 8 -4 -14 -19 -17 -8 4 14 19
9 9 -2 -13 -19 -18 -9 2 13 19 18
```

## 5.8.4. Pruebas de sistema

En este apartado se presentan las pruebas de tipo **caja negra** que se han diseñado para cubrir los requisitos que se especificaron.

## 1.- Salida tipo C

1.1.- **Propósito:** Comprobar los siguientes requisitos:

1.1.1.- **RF** *Formato de entrada* (**Esencial**)



1.1.2.- **RF** *Generación de secuencias* (**Esencial**)



1.1.3.- **RF** *Formato de salida* (**Esencial**)



1.1.4.- **RU** *Interfaz de línea de comandos* (**Esencial**)



1.1.5.- **RI** *Interfaz para programadores (C)* (**Esencial**)



1.2.- **Llamada:** `./generator.py rotating.cpg rotating.mvm -c`

1.3.- **Entrada:** El fichero *rotating.cpg* descrito en la página 65.

1.4.- **Salida esperada:**

■ **Salida estándar:**

```
1 generator: Note: c output.
```

■ ***rotating.c:***

```
1 static secPP seq = {
2     {0, 35, 57, 57, 35, 0, -35, -57, -57, -35},
3     {60, 48, 18, -18, -48, -60, -48, -18, 18, 48},
4     {0, 35, 57, 57, 35, 0, -35, -57, -57, -35},
5     {60, 48, 18, -18, -48, -60, -48, -18, 18, 48},
6     {0, 35, 57, 57, 35, 0, -35, -57, -57, -35},
7     {60, 48, 18, -18, -48, -60, -48, -18, 18, 48},
8     {0, 35, 57, 57, 35, 0, -35, -57, -57, -35},
9     {60, 48, 18, -18, -48, -60, -48, -18, 18, 48}
10 };
```

## 2.- Salida tipo Octave

2.1.- **Propósito:** Comprobar los siguientes requisitos:

2.1.1.- **RF** *Formato de entrada* (**Esencial**)



2.1.2.- **RF** *Generación de secuencias* (**Esencial**)



2.1.3.- **RF** *Formato de salida* (**Esencial**)



2.1.4.- **RU** *Interfaz de línea de comandos* (**Esencial**)



2.1.5.- **RI** *Interfaz para programadores (Octave)* (**Esencial**)



2.2.- **Llamada:** `./generator.py quad.cpg quad.mvm -octave`

2.3.- **Entrada:** El fichero *quad.cpg* descrito en la página 65.

2.4.- **Salida esperada:**

■ **Salida estándar:**

```
1 generator: Note: octave output.
```

■ ***quad.s:***

```
1 f1 = [19, 12, 1, -10, -18, -19, -12, -1, 10, 18];
2 f2 = [1, 13, 19, 18, 10, -1, -13, -19, -18, -10];
3 f3 = [-6, 5, 15, 19, 16, 6, -5, -15, -19, -16];
4 f4 = [5, -7, -16, -19, -15, -5, 7, 16, 19, 15];
5 f5 = [-18, -19, -12, 0, 11, 18, 19, 12, 0, -11];
6 f6 = [-10, -18, -19, -13, -2, 10, 18, 19, 13, 2];
7 f7 = [-19, -13, -2, 9, 17, 19, 13, 2, -9, -17];
8 f8 = [17, 8, -4, -14, -19, -17, -8, 4, 14, 19];
9 f9 = [9, -2, -13, -19, -18, -9, 2, 13, 19, 18];
10 t = [0 : 1 : 9];
11 plot (t, f1, t, f2, t, f3, t, f4, t, f5, t, f6, t, f7, t, f8, t, f9);
12 grid on;
13 pause;
```

3.- Salida en formato nativo *.mvm*

3.1.- **Propósito:** Comprobar los siguientes requisitos:

3.1.1.- **RF** *Formato de entrada* (**Esencial**)



3.1.2.- **RF** *Generación de secuencias* (**Esencial**)



3.1.3.- **RF** *Formato de salida* (**Esencial**)



3.1.4.- **RU** *Interfaz de línea de comandos* (**Esencial**)



3.1.5.- **RI** *Interfaz para programadores (Simulator)* (**Esencial**)



3.2.- **Llamada:** *./generator.py simple.cpg simple.mvm -mrs*

3.3.- **Entrada:** El fichero *simple.cpg* descrito en la página 66.

3.4.- **Salida esperada:**

■ **Salida estándar:**

```
1 generator: Note: mrs output.
```

■ ***simple.mvm:***

```
1 0 17 28 28 17 0 -17 -28 -28 -17
2 25 12 -6 -22 -29 -25 -12 6 22 29
3 -25 -29 -22 -6 12 25 29 22 6 -12
4 0 17 28 28 17 0 -17 -28 -28 -17
```

|   |     |     |     |     |     |     |     |     |     |     |
|---|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 5 | 25  | 12  | -6  | -22 | -29 | -25 | -12 | 6   | 22  | 29  |
| 6 | -25 | -29 | -22 | -6  | 12  | 25  | 29  | 22  | 6   | -12 |
| 7 | 0   | 17  | 28  | 28  | 17  | 0   | -17 | -28 | -28 | -17 |
| 8 | 25  | 12  | -6  | -22 | -29 | -25 | -12 | 6   | 22  | 29  |

## 5.9. Resumen

La herramienta **generator** sintetiza las **secuencias de movimiento** a partir de las **funciones generadoras** y sus parámetros. La entrada es un fichero de texto en formato .cpg. Con este formato se consigue una gran flexibilidad en la especificación de **funciones generadoras**, pudiéndose emplear cualquier función matemática.

La salida es un fichero de texto con **secuencias de movimiento** en forma de matriz. El formato de este fichero puede ser el nativo del simulador, .mvm, tablas de lenguaje C o el de un *script* para Octave.

Se ha realizado una implementación modular para reutilizar los componentes. Los módulos principales son *step* y *generator*. El primero es una librería que implementa toda la funcionalidad del generador y que puede ser utilizada por programas del usuario. El segundo es una fachada con una interfaz para la línea de comandos.



---

Parte III

## Simulator: Simulación física

---



---

## Capítulo 6

# Simulación y evaluación

---

### 6.1. Introducción

El problema a resolver es cómo realizar la simulación de unas estructuras modulares que se mueven según unas secuencias de movimiento. Para ello es necesario la incorporación de un motor físico.

El primer paso es crear un modelo del robot y del mundo virtual. Se utilizarán cuerpos rígidos para modelar las dos partes de cada módulo y articulaciones de un grado de libertad para sus uniones. El mundo virtual estará formado por una superficie plana, rígida y con rozamiento. El motor físico se encarga de realizar todos los cálculos necesarios durante la simulación.

Existe un compromiso entre la exactitud en la simulación y el tiempo requerido para la misma. A mayor exactitud mayor tiempo se requerirá para finalizar la simulación y viceversa. Para analizar la locomoción de robots modulares es más importante la fluidez en el movimiento que obtener una gran exactitud.

En este capítulo se describirá cómo funciona un motor físico en general y después se explicará el motor utilizado: el Open Dynamics Engine. Se abordará el tema de la modelización de los módulos Y1, los parámetros internos empleados y los algoritmos empleados para conseguir la locomoción a partir de las secuencias de movimiento.

### 6.2. Motor de simulación

Existen dos fases en el uso de los motores físicos: modelado y simulación. Durante la fase de modelado se utiliza la API del motor para la creación del mundo virtual, establecer sus propiedades y definir los objetos a simular. En la fase de simulación, el motor se encarga de actualizar el estado de todos los objetos, determinado por sus posiciones, velocidades, aceleraciones y fuerzas.

Los cambios de estado no son continuos, sino que se producen discretamente, cada ciertos instantes separados por intervalos fijos de tiempo. La duración de estos intervalos se denomina resolución temporal. Existe un compromiso entre la resolución y la fluidez en la simulación. Cuanta mayor sea la resolución, mayor es la

exactitud, pero peor es la fluidez. Los movimientos del robot tardarán más tiempo en realizarse.

Los cambios de estado se denominan pasos de simulación. En cada paso, se recalculan los estados de todos los objetos. La resolución marca el tiempo que transcurre entre dos pasos consecutivos.

Es importante destacar la independencia entre el mundo físico y el gráfico. Es posible realizar simulaciones sin visualizar los robots. Esto es lo que se hace al evaluar robots para la implementación de algoritmos de búsqueda.

El bucle principal de la simulación es:

---

**Pseudocódigo 6.1** simulator: Algoritmo de simulación

---

- 1: Modificar los estímulos si los hay.
  - 2: Realizar un paso de simulación.
  - 3: Renderizar el mundo tridimensional a partir del estado del motor físico. (opcional)
- 

## 6.3. Motor físico ODE

El motor físico empleado en este proyecto es el ODE[ODE]. Es un proyecto de código abierto, ampliamente extendido entre las aplicaciones de simulación y videojuegos.

Su interfaz de programación está escrita en C, aunque se pueden emplear los *wrappers* existentes tanto para C++ como para Python[PyO]. Esto último es lo que se ha empleado en este proyecto. Para una introducción rápida al uso de ODE bajo Python se puede consultar [Man].

Como se comentó al final del apartado anterior, la simulación física y gráfica están separadas. El simulador se puede configurar para trabajar sólo con simulación, finalizando al cabo de un tiempo o bien con simulación más renderizado, deteniéndose al cerrar la ventana de presentación.

El ODE incorpora un sistema de detección de colisiones entre los cuerpos del mundo virtual. Sin embargo, en este proyecto se está utilizando un sistema propio, apoyado en ODE, que sólo tiene en cuenta las colisiones entre los cuerpos y el suelo. Con ello se consigue pasar de  $O(n^2)$  posibles colisiones a  $O(n)$ , donde  $n$  es el número de cuerpos. Consiguiéndose simulaciones más rápidas.

## 6.4. Módulos Y1

### 6.4.1. Modelado

Para modelar los módulos Y1 con el ODE se han empleado dos hexaedros unidos por una articulación de tipo *Hinge Joint*.

| Parámetro   | Valor  | Descripción                                           |
|-------------|--------|-------------------------------------------------------|
| ParamFMax   | 0,4    | Par máximo del servo                                  |
| ParamVel    | 0,0    | Velocidad del servo, inicialmente 0                   |
| ParamLoStop | $-\pi$ | Límite inferior del servo                             |
| ParamHiStop | $\pi$  | Límite superior del servo                             |
| X           | 0,36   | Tamaño en el eje X                                    |
| Y           | 0,52   | Tamaño en el eje Y                                    |
| Z           | 0,52   | Tamaño en el eje Z                                    |
| GAIN        | 5,0    | Ganancia del servo en el algoritmo de posicionamiento |

Cuadro 6.1: Parámetros empleados en la simulación de un módulo Y1

| Parámetro   | Valor  | Descripción                         |
|-------------|--------|-------------------------------------|
| ParamFMax   | 1000,0 | Par máximo del servo                |
| ParamVel    | 0,0    | Velocidad del servo, inicialmente 0 |
| ParamLoStop | $-\pi$ | Límite inferior del servo           |
| ParamHiStop | $\pi$  | Límite superior del servo           |

Cuadro 6.2: Parámetros empleados para la unión entre dos módulos Y1

La altura y anchura de esos hexaedros son la de los módulos Y1 y de la longitud, la mitad. Las dimensiones de cada uno son:  $36 \times 52 \times 52$ mm. Todos los parámetros empleados para la simulación se muestran en el cuadro 6.1. El parámetro GAIN es la ganancia que se emplea para determinar la velocidad del servo en el algoritmo de posicionamiento descrito en el apartado 6.4.2.

El último de los parámetros es la ganancia que se emplea para determinar la velocidad del servo en el algoritmo de posicionamiento de los servos, en la página 80.

La unión entre dos módulos Y1 se ha modelado como una articulación *Hinge Joint*, pero con un par extremadamente alto. El valor de sus parámetros se muestra en el cuadro 6.2. Se utiliza esta articulación frente a la *Fixed Joint* debido a que esta última es mucho más restrictiva y consume mucho más tiempo del motor físico. De hecho, el autor del motor desaconseja su uso para casos como éste.

Los parámetros empleados para modelar el mundo virtual se muestran en el cuadro 6.3.

El modelado de los robots para su simulación se hace mediante hexaedros unidos con articulaciones. En nuestro mundo virtual, los robots serán uniones de estos hexaedros. Sin embargo, a la hora de renderizar en pantalla se tienen dos opciones. Una es mostrar este modelo físico hexaédrico. La otra es hacer un renderizado en forma de módulos Y1. Con ello se consigue ver la apariencia que tendrá el robot una vez construido con módulos Y1. Pero lo que realmente se está simulando es el modelo de los hexaedros.

| Parámetro        | Valor  | Descripción                                          |
|------------------|--------|------------------------------------------------------|
| Gravedad         | 9,81   | Valor de fuerza de la gravedad                       |
| CFM              | $1e-5$ | Coefficiente utilizado para controlar el error       |
| AutoDisableFlag  | 1      | Límite inferior del servo                            |
| MaxCorrectingVel | 0,1    | Límite de cambio de velocidad por rozamiento         |
| SurfaceLayer     | 0,001  | Margen de los objetos para colisionar unos con otros |

Cuadro 6.3: Parámetros empleados en la simulación

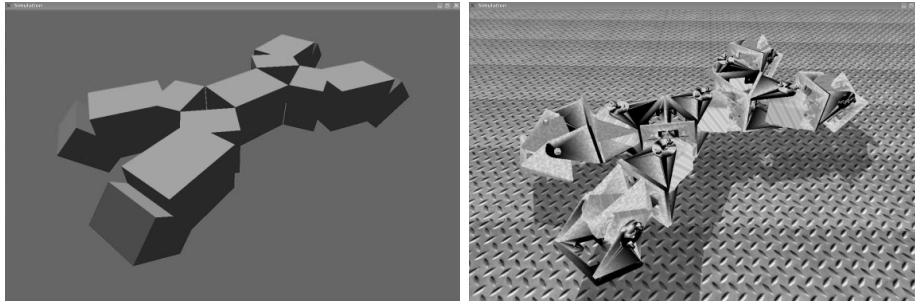


Figura 6.1: Comparativa de la simulación en el plano físico y gráfico

En la figura 6.1 se muestran estos dos tipos de renderizado para un robot cuadrúpedo de 9 módulos. En la izquierda está el renderizado hexaédrico y en la derecha el de módulos Y1.

### 6.4.2. Simulación

Para simular los módulos Y1 hay que modelar los servomecanismos que incorporan. El ODE no tiene implementados los servos como elementos de simulación. Una opción es programar el controlador y encontrar los valores de sus parámetros. Sin embargo se ha optado por la solución que ofrece el propio autor de ODE y que se describe a continuación:

El pseudocódigo del algoritmo de posicionamiento de servomecanismos es el siguiente:

---

#### Pseudocódigo 6.2 simulator: Algoritmo de posicionamiento de servos

---

```

1: errorGlobal = 0,0
2: for servo in servos do
3:   error = (anguloActual - anguloDeseado) × constanteAjuste   \ * Calcula el error
   en la posición *
4:   errorGlobal += |error|   \ * Agrega este error al error global *
5:   velocidadMotor = -error × GAIN   \ * Establece la velocidad final del motor *
6: end for

```

---

Se está empleando un controlador de tipo proporcional con ganancia dada por la constante GAIN. La velocidad que se asigna al servo en cada instante es el producto del error por esta ganancia. El error mide la distancia angular que separa al servo de su posición de referencia, *anguloDeseado*.

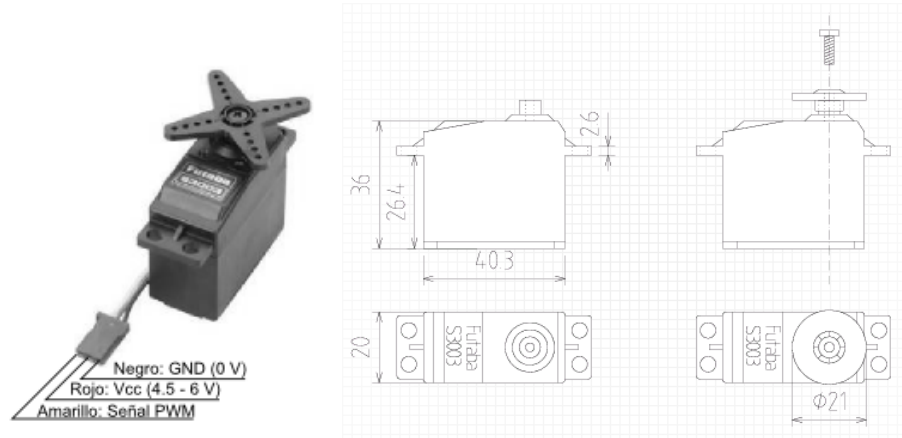


Figura 6.2: Conexión y dimensiones de un servo Futaba S3003

Los servos empleados en los módulos Y1 son del tipo Futaba S3003. El par máximo es de 4,0Kg/cm y la velocidad angular máxima es 0,23s/60grados. En la figura 6.2 se muestra el aspecto de uno de estos servos, junto con sus dimensiones y los tres cables de conexión: alimentación, masa y señal PWM.

## 6.5. Simulación de robots modulares

En este apartado se describe el algoritmo de simulación de un robot. Las posiciones iniciales de los servos se toman de la primera columna de la matriz de movimiento. En cada paso de simulación se ejecuta el algoritmo de posicionamiento de servos descrito en el apartado 6.4.2, calculándose las nuevas velocidades y el error global, *errorGlobal*.

Cuando todos los servos han alcanzado las posiciones deseadas, se leen las nuevas posiciones de referencia, que se obtienen de la siguiente columna de la matriz de movimiento. Esta condición se cumple cuando *errorGlobal* es menor que un cierto umbral, *umbralError*. El proceso se repite indefinidamente hasta que la simulación finalice, bien porque transcurra el tiempo máximo indicado o bien porque el usuario cierre la ventana de simulación.

El algoritmo se resume a continuación:

**Pseudocódigo 6.3** simulator: Algoritmo de actualización de estímulos

- 
- 1: Establecer las nuevas posiciones con el Algoritmo de posicionamiento.
  - 2: **if** errorGlobal < umbralError **then**
  - 3:   Avanza la secuencia de movimiento   \ *\* Sólo si se ha llegado a una posición adecuada se avanza en la secuencia de movimiento \** \
  - 4: **end if**
  - 5: Renderizar las nuevas posiciones
- 

Debido a que ODE almacena toda la información de los cuerpos y las uniones, para obtener toda esta información sólo hace falta almacenar las referencias a los objetos *Hinge Joint* junto con el número de orden que se leyó del fichero. Así se pueden actualizar las articulaciones de una manera sencilla.

## 6.6. Movimiento de robots reales

Una característica importante del MRSuite es que además de la simulación de robots, permite el movimiento de robots reales conectados a través del puerto serie. Los componentes involucrados son: el ordenador, la electrónica y el robot real, como se muestra en la figura 6.3.

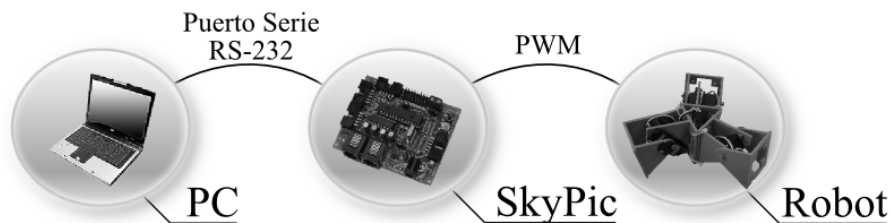


Figura 6.3: Comunicaciones con el robot

En el PC se ejecuta el simulador que se comunica mediante el puerto serie con el firmware servos8[ser], alojado en la tarjeta SkyPic[Skyb]. Este firmware funciona como un servidor de posicionamiento y ofrece unos servicios a los clientes que ejecutan en el PC. Forma parte del proyecyo Stargate[Sta].

El servidor recibe las posiciones para cada servo a través del protocolo serie y genera unas señales de modulación de pulsos (PWM) que permiten modificar las posiciones de los servos.

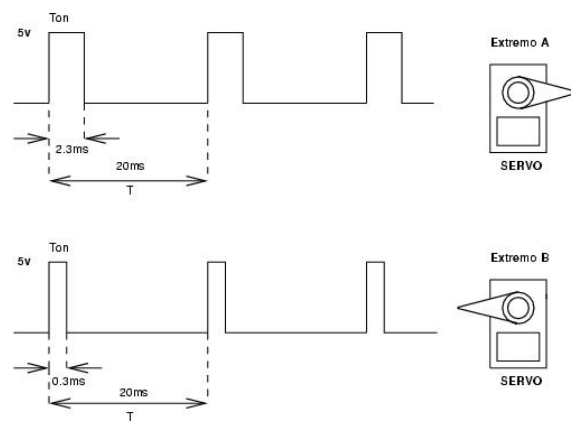


Figura 6.4: Señales PWM

Las señales PWM se muestran en la figura 6.4. El período es fijo, con un valor de 20ms. El ancho del pulso determina la posición a la que se moverá el servo. Los valores correspondientes a los extremos son de 0,3ms y 2,3ms.

## 6.7. Resumen

Para realizar la simulación de la locomoción de los robots modulares es necesario la utilización de un motor físico. Se ha elegido el ODE por ser libre, robusto y ampliamente usado.

Los módulos Y1 se modelan como dos cuerpos rígidos unidos mediante articulaciones. Para las uniones entre distintos módulos se emplean articulaciones rígidas.

Para el posicionamiento de los servos se utiliza un controlador de tipo proporcional. El algoritmo empleado es el propuesto por el creador del ODE y puede verse en la sección 6.4.2.

Por último, para la simulación del movimiento del robot se comprueba en cada paso de simulación si todos los módulos han alcanzado las posiciones deseadas, determinadas por la columna  $i$ -ésima de la matriz de movimiento. Cuando se cumple dicha condición, se lee la columna  $i + 1$ -ésima y se establecen las nuevas posiciones de referencia.

A la hora de simular hay que establecer un compromiso entre exactitud y fluidez. Se han configurado los parámetros del ODE para conseguir una simulación fluida.

---

## Capítulo 7

# Simulator

---

### 7.1. Introducción

*Simulator* es una aplicación de simulación para **robots modulares**. Además de la visualización en 3D, permite mover un robot real conectado al PC a través del puerto serie. La entrada de la aplicación serán los ficheros con la estructura del robot y la matriz de movimiento, obtenidos a partir de las aplicaciones designer y generator respectivamente. La salida será bien la simulación en la pantalla o el movimiento del robot real. La simulación física se realiza a partir del motor Open Dynamics Engine (ODE).

El objetivo de esta aplicación es permitir la simulación de robots modulares de una forma rápida y fácil para que los desarrolladores e investigadores puedan crear nuevas estructuras o generar nuevos patrones de movimiento.

Para enriquecer el objetivo anterior, la aplicación permite el movimiento libre de cámara por el escenario, la simulación de múltiples robots a la vez para realizar competiciones virtuales y la renderización mediante cubos, módulos Y1 o sin renderización.

Por último, para mejorar la extensibilidad de la aplicación, el simulador está pensado para ser muy fácilmente integrado en otros programas de usuario a través de su API o extender su API para añadir más funcionalidad.

### 7.2. Definición del problema

El problema que se plantea es el diseño de esta aplicación de simulación. A partir del grafo estructural y la matriz de movimiento se deberá realizar la simulación o el control del robot real, como se muestra en la figura 7.1. Ambas entradas serán ficheros de texto en los formatos correctos.

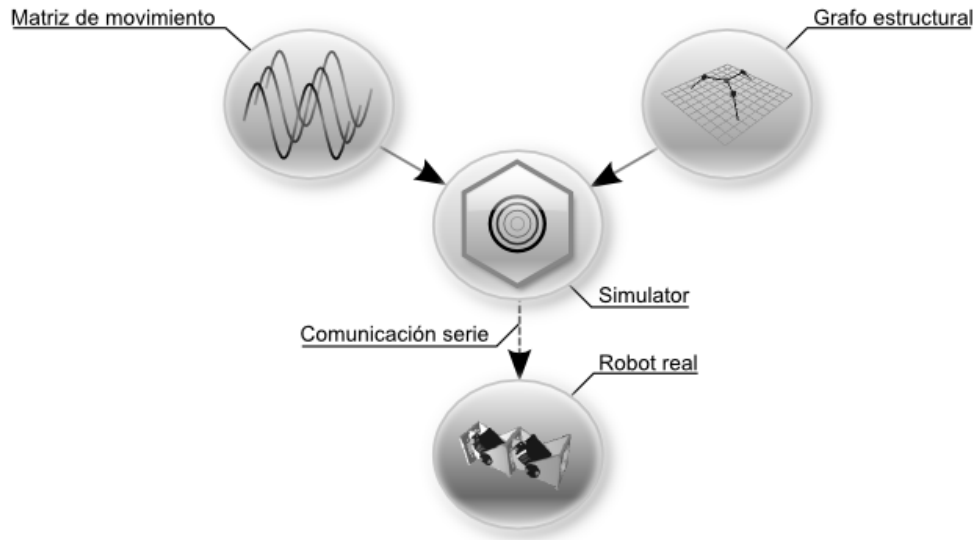


Figura 7.1: Esquema de funcionamiento del simulador

## 7.3. Especificación de requisitos

### 1.- RF *Formato de entrada* (Esencial)

La entrada a la aplicación se efectuará mediante el uso de ficheros de texto. El formato de fichero será el formato .mvm definido en el apartado 5.3.2 para los movimientos y el formato .rbt especificado en el apartado 3.3.1 para las estructuras.

### 2.- RF *Protocolo de comunicación serie* (Esencial)

Para la comunicación a través de la línea serie se debe utilizar el protocolo de comunicaciones definido en el apartado 7.4.1.

### 3.- RU *Interfaz de línea de comandos* (Esencial)

#### 3.1.- Argumentos

La interfaz con la línea de comando tendrá los siguientes argumentos en ese orden:

3.1.1.- Fichero de **grafo estructural** (Obligatorio)

3.1.2.- Fichero de **matriz de movimiento** (Opcional)

3.1.3.- Opciones

- *-t<seconds>*: Determina el tiempo durante el cual correrá la simulación. Si no se especifica será infinito.
- *-n<# robots>*: Define el número de robots que se van a simular, a modo de carrera.

- *-d<serial dev>*: Especifica el dispositivo serie que se usa para enviar los movimientos al robot real.
- *-m*: Flag que indica la renderización de **módulos Y1**.
- *-w*: Flag que determina el uso de cubos (por defecto).
- *-s*: Flag para realizar una simulación sin renderizado.

#### 4.- RU *Interfaz visual* (Esencial)

##### 4.1.- Apariencia general

El sistema se compondrá de una ventana tridimensional de visualización donde irá apareciendo la simulación.

##### 4.2.- Controles

Todos los comandos serán introducidos a través del teclado o del ratón.

##### 4.3.- Movimiento de cámara

Mediante el uso del ratón y sus botones será posible realizar los siguientes movimientos de cámara.

###### 4.3.1.- Movimiento de cabeceo y rotación sobre si misma

Mediante el movimiento del ratón y de la pulsación del botón izquierdo se realizarán estos movimientos.

###### 4.3.2.- Desplazamiento en la dirección de abscisas y de ordenadas

Mediante el movimiento del ratón y de la pulsación del botón derecho se realizarán estos movimientos.

###### 4.3.3.- Desplazamiento en altura

Mediante el movimiento del ratón y de la pulsación de ambos botones se realizarán estos movimientos.

#### 5.- RI *Interfaz para programadores* (Esencial)

##### 5.1.- Utilización

La librería tendrá una API para poder ser utilizada en otras aplicaciones. La interfaz gráfica no será más que una *fachada* para su manejo.

##### 5.2.- API

Los diferentes servicios que deberá ofrecer serán:

###### 5.2.1.- `initSim (robots, seqs, dModule, wire, serial)`

Función para inicializar una simulación. Recibe los diferentes robots y **secuencias de movimiento** a simular y otros parámetros de la simulación.

###### 5.2.2.- `simulationLoop (args, endTime, renderEnabled)`

Función responsable de realizar un bucle de simulación durante un tiempo. Una misma simulación puede tener tantos bucles de simulación como se desee.

- 5.2.3.- `getRobotInitPosition (index)`  
Devuelve la posición inicial de la que parte cada robot.
- 5.2.4.- `getRobotLastPosition (index)`  
Devuelve la posición tras un bucle de simulación.
- 5.2.5.- `getSimulationTime ()`  
Devuelve la última información del tiempo de simulación.

## 7.4. Diseño de la solución

### 7.4.1. Protocolo serie

La comunicación del simulador con el robot real se hace a través de comandos que se envían por el puerto serie al firmware que se ejecuta en el microcontrolador, ver figura 6.3 del apartado 6.6. Este firmware funciona a modo de servidor, ofreciendo unos servicios accesibles mediante comunicación serie.

Las comunicaciones se realizan a 9600bps con 8 bits de datos, sin paridad y 1 bit de stop, es la configuración 8N1. Ahora se exponen los servicios:

#### 1.- Servicio POS:

Servicio encargado de posicionar los diferentes servos del robot. Se envía una trama por la línea serie con el siguiente formato:

(1 byte)0x57(1 byte)<índice de servo>(1 byte)<posición>

Los índices de los servos están comprendidos en el rango [1 8]. La posición es un valor entero sin signo en el rango [0 232], obteniendo una precisión de 0,77 grados.

#### 2.- Servicio ENA:

Servicio para activar y desactivar los servos. Un servo desactivado no consume y no ejerce fuerza para mantener su posición. Se envía una trama con el siguiente formato:

(1 byte)0x45(1 byte)<máscara>

La máscara indica los servos activos e inactivos. El bit de menor peso se corresponde con el servo con índice 1, así hasta el de mayor peso, con índice 8. Un bit a 1 significa que el servo está activo.

## 7.5. Diseño de alto nivel

La arquitectura de la aplicación se muestra en la figura 7.2. Consta de diferentes módulos que se pasa a especificar:

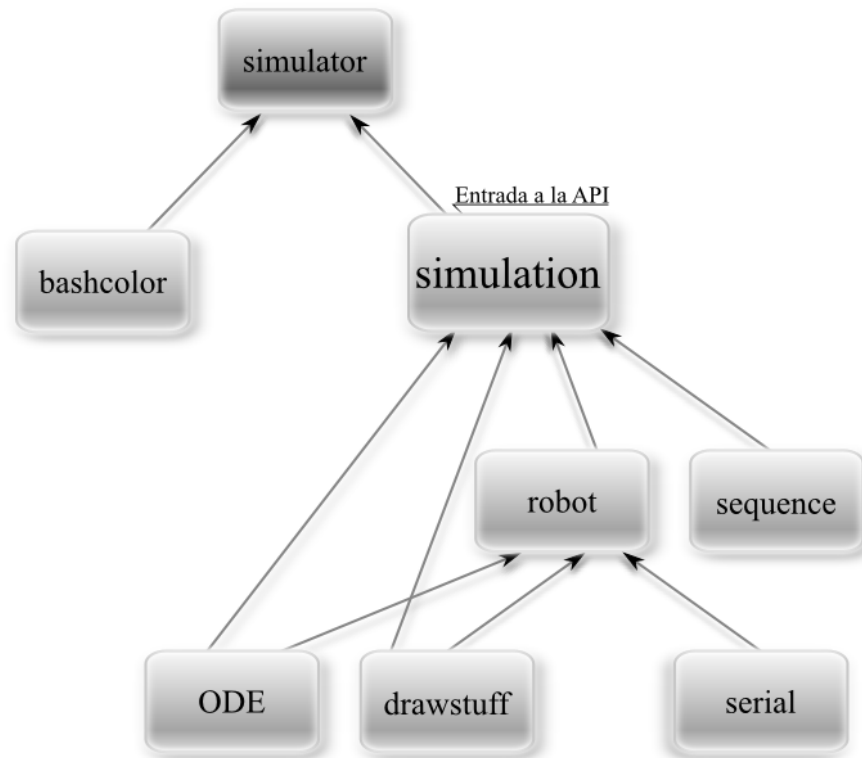


Figura 7.2: Arquitectura de alto nivel del simulador

Se observa la organización y la interacción modular de los componentes descritos. Las responsabilidades de cada uno de estos componentes quedarían repartidas de la siguiente forma:

**simulation:** Librería encargada de ser la API de programación, núcleo de la simulación. Dicha API está descrita en el apartado 7.3.

**simulator:** Se corresponde con la fachada del sistema mediante línea de comandos para invocar a la simulación.

**robot:** Módulo encargado de la carga de robots desde fichero y su posterior simulación.

**sequence:** Módulo responsable del tratamiento de ficheros de movimiento y su aplicación a los robots.

**serial:** Módulo de comunicación serie con los robots.

**drawStuff:** Módulo responsable de gran parte del trabajo de visualización a través de OpenGL. Librería parte del motor físico ODE que se utiliza en la simulación. Se ha adaptado de forma que pueda ser usado desde Python. Copyright (C) 2001-2003 Russell L. Smith. Bajo licencia GNU.

**bashcolor:** Se ocupa de la mensajería coloreada.

De esta forma se permite la reutilización de componentes para otros proyectos. El módulo *simulation* puede ser utilizado en otras aplicaciones de simulación. La fachada, *simulator*, puede ser sustituida por otra de tipo línea de comandos. El resto de librerías son componentes de trabajo con los formatos descritos y de ayuda en el uso de los motores OpenGL y ODE.

## 7.6. Descripción de interfaces entre módulos

### 7.6.1. Interfaz *simulation*

#### 1.- *initSim (robots, seqs, dModule, wire, serial)*

Se encarga de iniciar la simulación. El parámetro *robots* es una lista, bien con nombres de fichero o con los **grafos estructurales**. En *seqs* se almacena una lista de nombres de fichero o una lista de **secuencias de movimiento**. El parámetro *dModule* determina si se renderizan los **módulos Y1**, *wire* establece si dibujar los límites de los módulos en forma alámbrica y *serial* suministra un dispositivo serie para comunicarse con el robot real.

#### 2.- *simulationLoop (args, endTime, renderEnabled)*

Función responsable de realizar un bucle de simulación. Se deben suministrar los argumentos originales en *args*. *endTime* almacena los segundos que durará el bucle de simulación y *renderEnabled* determina si se va a realizar visualización.

#### 3.- *position = getRobotInitPosition (index)*

Devuelve la posición inicial de la que parte cada robot. *index* está en el rango  $[0 \text{ } \#robots - 1]$ .

#### 4.- *position = getRobotLastPosition (index)*

Devuelve la última posición conocida tras un bucle de simulación. *index* se mueve en el mismo rango.

#### 5.- *time = getSimulationTime ()*

Devuelve la última información del tiempo de simulación.

### 7.6.2. Interfaz *robot*

#### 1.- *init (source, world, space, pos, dModule, wire, ser)*

Método constructor de la clase que inicializa el contexto del robot en la simulación. *source* representa o bien el fichero o la secuencia de aristas del robot.

*world* y *space* son objetos del motor físico. *pos* es la posición inicial del primer módulo. Los parámetros *dModule* y *wire* determinan la visualización de los módulos. Por último, *ser* determina el objeto serie que se comunica con el robot real.

**2.- *createRotbot* ()**

Método para crear el robot tanto en el motor físico como en el motor 3D.

**3.- *drawRobot* ()**

Método encargado de dibujar el robot a instancias del simulador y basado en las posiciones determinadas por el motor físico.

**4.- *setServos* (*seq*)**

Método para establecer las posiciones de los servos del robot según la **secuencia de movimiento** almacenada en la lista *seq*.

### 7.6.3. Interfaz *sequence*

**1.- *init* (*seq*)**

Método constructor al que se suministra la **matriz de movimiento** en el parámetro *seq*.

**2.- *forward* ()**

Método para avanzar al siguiente paso de la **secuencia de movimiento**.

**3.- *position = desiredPos* ()**

Método que devuelve la posición de todos los servos para un punto concreto de la **secuencia de movimiento**.

**4.- *writeGait* (*file*)**

Método que escribe la actual **secuencia de movimiento** en el fichero *file*.

### 7.6.4. Interfaz *serial*

**1.- *init* (*term*)**

Método constructor encargado de inicializar el dispositivo proporcionado. El parámetro *term* contiene el nombre del dispositivo que se va a emplear.

**2.- *setServos* (*servos*)**

Método encargado de enviar las tramas de posicionamiento a partir de los datos recibidos. El parámetro *servos* está formado por la lista de valores de posición de cada servo.

### 3.- *setMask (mask)*

Método encargado de enmascarar ciertos servos para que no estén habilitados. El parámetro *mask* contiene el byte de enmascaramiento, con un bit por servo.

### 7.6.5. Interfaz *drawStuff*

La interfaz ya fue descrita en el apartado 3.6.2, página 38.

### 7.6.6. Interfaz *bashcolor*

Esta interfaz puede encontrarse en el apartado 5.6.2 de la página 62.

## 7.7. Diseño de bajo nivel

### 7.7.1. Módulo *simulation*

---

**Pseudocódigo 7.1** *simulation: initSim (robots, seqs, dModule, wire, serial)*

---

```

1: Crea el mundo físico y establece sus parámetros
2: models =  $\Phi$   \ * Inicializa los modelos * \
3: simulationTime = 0,0  \ * Inicializa el tiempo de simulación * \
4: for i, modeli in enumerate(modelfile) do  \ * Itera sobre los la lista de robots * \
5:   model = robot (modeli)  \ * Crea un robot a partir del fichero * \
6:   Comprueba que es un modelo válido (grafo acíclico simple)
7:   seq = sequence (seqfiles [i])  \ * Crea una secuencia a partir del fichero * \
8:   Comprueba que model y seq son compatibles
9:   models = models  $\cup$  (model, seq)  \ * Agrega el par modelo, secuencia a los modelos
   * \
10: end for
11: for (model, seq) in models do  \ * Itera sobre los modelos válidos * \
12:   model.createRobot ()  \ * Crea el modelo en el mundo físico virtual * \
13: end for

```

---

---

**Pseudocódigo 7.2** simulation: simulationLoop (args, endTime, renderEnabled)

---

```

1: totalTime = simulationTime + endTime  \ * Establece el tiempo total de simulación
   * \
2: if renderEnabled then
3:   Inicializa el vector de funciones de callback con los valores por defecto.
4:   Realiza un paso del simulador drawstuff
5: else  \ * No se requiere renderizado * \
6:   running = True  \ * Establece la bandera de parada * \
7:   while running do
8:     Realiza la detección de colisiones
9:     Invoca un paso del motor físico
10:  if simulationTime >= totalTime then
11:    running = False  \ * Detiene la ejecución del bucle * \
12:  end if
13:  Establece las nuevas posiciones de los servos
14:  for model, seq in models do
15:    Almacena la nueva posición del modelo
16:  end for
17: end while
18: end if

```

---



---

**Pseudocódigo 7.3** simulation: position = getRobotInitPosition (index)

---

```

1: if existe ese índice then
2:   return models [index][0].initCenter  \ * Devuelve el valor de la posición * \
3: else  \ * No existe el índice * \
4:   return None  \ * Devuelve vacío * \
5: end if

```

---



---

**Pseudocódigo 7.4** simulation: position = getRobotLastPosition (index)

---

```

1: if existe ese índice then
2:   return models [index][0].lastCenter  \ * Devuelve el valor de la posición * \
3: else  \ * No existe el índice * \
4:   return None  \ * Devuelve vacío * \
5: end if

```

---



---

**Pseudocódigo 7.5** simulation: time = getSimulationTime ()

---

```

1: return simulationTime  \ * Devuelve el tiempo de simulación * \

```

---

### 7.7.2. Módulo *robot*

---

**Pseudocódigo 7.6** robot: init (source, world, space, pos, dModule, wire, ser)
 

---

```

1: Almacena la información de los parámetros en la instancia
2: Inicializa el dispositivo serie si existe
3: if source es un fichero then
4:   Parsea el fichero para recopilar su información
5: else   \ * Es una secuencia * \
6:   Almacena la secuencia
7: end if

```

---



---

**Pseudocódigo 7.7** robot: createRotbot ()
 

---

```

1: Busca una hoja del grafo estructural
2: while nodo in búsqueda en profundidad do
3:   Busca los módulos a crear conectados al nodo actual
4:   Calcula la posición del módulo a crear
5:   Crea el módulo con la información anterior
6: end while

```

---



---

**Pseudocódigo 7.8** robot: drawRobot ()
 

---

```

1: for modulo in robot do
2:   Recupera la información del motor físico de la posición
3:   Dibuja el módulo con la información anterior
4: end for
5: Actualiza la posición actual del robot

```

---



---

**Pseudocódigo 7.9** robot: setServos (seq)
 

---

```

1: Solicita las posiciones deseadas al módulo sequence
2: if existe conexión serie y hay cambios en los servos then
3:   Envía la información de los servos por la línea serie
4: end if
5: for servo in servos do
6:   Calcula la diferencia entre la posición actual y la deseada
7:   Indica al motor físico que modifique la velocidad del servo en consecuencia
8: end for
9: if No se comete mucho error entre las posiciones reales y deseadas then
10:   Avanzar la secuencia de movimiento
11: end if

```

---

### 7.7.3. Módulo *sequence*

---

#### Pseudocódigo 7.10 sequence: init (seq)

---

```

1: Almacena la información e inicializa la instancia
2: seqPosition = 0  \ * Inicializa el paso de la secuencia * \
3: if seq es un fichero then
4:   Parsea el fichero para obtener la secuencia
5:   Almacena la secuencia leída del fichero
6: else  \ * Es una secuencia * \
7:   Almacena la secuencia recibida
8: end if
9: Almacena el número de pasos de la secuencia

```

---



---

#### Pseudocódigo 7.11 sequence: forward ()

---

```

1: seqPosition = seqPosition+1

```

---



---

#### Pseudocódigo 7.12 sequence: position = desiredPos ()

---

```

1: return gait [seqPosition]  \ * Devuelve las posiciones deseadas * \

```

---



---

#### Pseudocódigo 7.13 sequence: writeGait (file)

---

```

1: Abre el fichero file para escritura
2: Almacena la información en forma de matriz por filas
3: Cierra el fichero

```

---

### 7.7.4. Módulo *serial*

---

#### Pseudocódigo 7.14 serial: init (term)

---

```

1: Almacena los parámetros
2: Inicializa el dispositivo serie

```

---



---

#### Pseudocódigo 7.15 serial: setServos (servos)

---

```

1: for servo in servos do
2:   if Ha cambiado la posición desde la última vez then
3:     Envío de una trama al robot para ese servo
4:   end if
5: end for

```

---



---

#### Pseudocódigo 7.16 serial: setMask (mask)

---

```

1: Envío de una trama al robot con la máscara

```

---

### 7.7.5. Módulo *bashcolor*

La descripción de bajo nivel de este módulo se puede encontrar en el apartado 5.7.2 de la página 64.

## 7.8. Descripción de interfaces de usuario

La interfaz de usuario es la misma que la del designer. Los movimientos de la cámara se controlan con el ratón como se describe en el capítulo 5. En la figura 7.3 se muestra la simulación de un robot hexápodo de 14 módulos. Se han dibujado unas líneas ficticias para apreciar mejor la separación entre módulos.

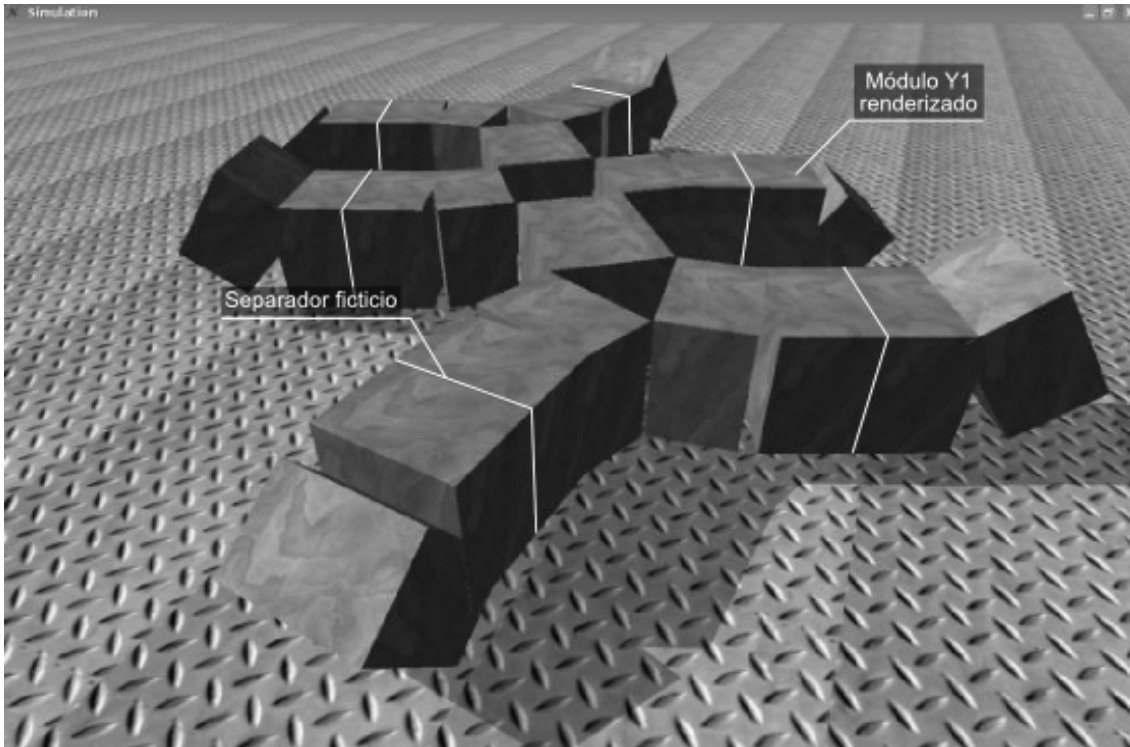


Figura 7.3: Interfaz del simulador

## 7.9. Pruebas

### 7.9.1. Ficheros de prueba

Para la realización de las pruebas se emplean tanto ficheros en formato .mvm como en formato .rbt. Todos los ficheros a utilizar ya han sido listados como resul-

tados de las pruebas de las anteriores aplicaciones por lo que se hará referencia a ellas.

## 7.9.2. Pruebas unitarias

### 1.- Módulo simulation



1.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

1.2.- **Llamada:** `./simulation.py`

1.3.- **Entrada:** Los ficheros *hypercube.rbt* y *rotating.mvm* internamente.

1.4.- **Salida esperada:** El módulo imprimirá por salida estándar:

```

1  Autotesting...
2  5.05 0.203493577547
3  10.05 0.299709421696
4  15.05 0.463482877706
5  20.05 0.447765206345
6  25.05 0.412541061941
7  30.05 0.447688946037
8  35.1 0.268706948008
9  40.15 0.227228167488
10 45.2 0.175223539937
11 50.25 0.468458795917
12 55.3 0.87858939197
13 60.35 1.0091944437
14 65.4 1.45731045979
15 70.45 1.95843632787
16 75.5 2.18417978413
17 80.55 2.7182326597
18 85.6 3.32983927428
19 90.65 3.71152815708
20 95.7 4.29022521582
21 100.75 4.9315545638

```

### 2.- Módulo robot



2.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

2.2.- **Llamada:** `./robot.py`

2.3.- **Entrada:** El fichero *quad.rbt* internamente. Se puede encontrar en la página 47.

2.4.- **Salida esperada:** El módulo imprimirá por salida estándar:

```
1 Autotesting...
2 OK
```

### 3.- Módulo sequence



3.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

3.2.- **Llamada:** `./sequence.py`

3.3.- **Entrada:** El fichero `quad.mvm` internamente. Se puede encontrar en la página 69.

3.4.- **Salida esperada:**

■ **Salida estándar:**

```
1 Autotesting...
2 [0, 20, 17, 20, -17, 20, 0, 20]
3 [11, 20, 8, 20, -19, 20, 11, 20]
4 [19, 20, -4, 20, -14, 20, 19, 20]
5 [19, 20, -14, 20, -4, 20, 19, 20]
```

■ ***gait.txt:***

```
1 0 20 17 20 -17 20 0 20
2 11 20 8 20 -19 20 11 20
3 19 20 -4 20 -14 20 19 20
4 19 20 -14 20 -4 20 19 20
5 11 20 -19 20 8 20 11 20
6 0 20 -17 20 17 20 0 20
7 -11 20 -8 20 19 20 -11 20
8 -19 20 4 20 14 20 -19 20
9 -19 20 14 20 4 20 -19 20
10 -11 20 19 20 -8 20 -11 20
```

### 4.- Módulo serial



4.1.- **Propósito:** Comprobar el funcionamiento correcto del módulo.

4.2.- **Llamada:** `./serial.py`

4.3.- **Entrada:**  $\Phi$

4.4.- **Salida esperada:** El módulo imprimirá por salida estándar:

```
1 Autotest of "serial.py"
2 Using "/dev/ttyUSB0" as dev
```

### 5.- Módulo bashcolor



Esta prueba unitaria ya se describió en el apartado 3 de la página 68.

### 7.9.3. Pruebas de integración

La estrategia que se va a utilizar para probar la integración de los diferentes módulos será de arriba a abajo. Es decir, partiendo del módulo principal, se irán integrando los demás módulos. Para los módulos aún no integrados se utilizarán *stubs* que los sustituyan.

#### 1.- Interfaz simulation



1.1.- **Propósito:** Integración entre *simulator* y *simulation* junto a *drawstuff*. Las funciones a integrar serán:

1.1.1.- *initSim (robots, seqs, dModule, wire, serial)*



1.1.2.- *simulationLoop (args, endTime, renderEnabled)*



1.1.3.- *position = getRobotInitPosition (index)*



1.1.4.- *position = getRobotLastPosition (index)*



1.1.5.- *time = getSimulationTime ()*



1.2.- **Llamada:** *./simulator*

1.3.- **Entrada:**  $\Phi$

1.4.- **Salida esperada:** En esta prueba la única salida esperada es la visualización correcta del entorno de simulación.

#### 2.- Interfaz robot



2.1.- **Propósito:** Integración entre *simulation* y *robot*. Las funciones a integrar serán:

2.1.1.- *init (source, world, space, pos, dModule, wire, ser)*



2.1.2.- *createRobot ()*



2.1.3.- *drawRobot ()*



2.1.4.- *setServos (seq)*



2.2.- **Llamada:** *./simulator hypercube.rbt*

2.3.- **Entrada:** El fichero *hypercube.rbt* descrito en la página 45.

2.4.- **Salida esperada:** Añadido al entorno de simulación, se debe observar la renderización estática del robot en el escenario de simulación.

#### 3.- Interfaz sequence



3.1.- **Propósito:** Integración entre *simulation* y *sequence*. Las funciones a integrar serán:

- 3.1.1.- `init (seq)`
- 3.1.2.- `forward ()`
- 3.1.3.- `position = desiredPos ()`
- 3.1.4.- `writeGait (file)`

3.2.- **Llamada:** `./simulator quad.rbt quad.mvm`

3.3.- **Entrada:** Los ficheros `quad.rbt` y `quad.mvm` descritos en las páginas 47 y 69 respectivamente.

3.4.- **Salida esperada:** Esta prueba debe mostrar un robot cuadrúpedo que se mueve mediante ondas sinusoidales.

#### 4.- Interfaz serial



4.1.- **Propósito:** Integración entre *robot* y *serial*. Las funciones a integrar serán:

4.1.1.- `init (term)`



4.1.2.- `setServos (servos)`



4.1.3.- `setMask (mask)`



4.2.- **Llamada:** `./simulator hypercube.rbt rotating.mvm -d/dev/ttyUSB0 -s`

4.3.- **Entrada:** Los ficheros `hypercube.rbt` y `rotating.mvm`.

4.4.- **Salida esperada:** Esta prueba se realiza para comprobar sin visualización que el robot real conectado al puerto serie indicado responde a las consignas de la **secuencia de movimiento**, en este caso el robot debe rotar sobre sí mismo.

### 7.9.4. Pruebas de sistema

Para estas pruebas, el modelo seguido ha sido el de **caja negra**. Por lo que se realizan diferentes pruebas de forma que se cubran todos los requisitos especificados.

#### 1.- Configuración cuadrúpeda

1.1.- **Propósito:** Comprobar los siguientes requisitos:

1.1.1.- **RF** *Formato de entrada* (**Esencial**)



1.1.2.- **RU** *Interfaz de línea de comandos* (**Esencial**)



1.1.3.- **RU** *Interfaz visual* (**Esencial**)



1.1.4.- **RI** *Interfaz para programadores* (**Esencial**)



1.2.- **Llamada:** `./simulator.py quad.rbt quad.mvm`

1.3.- **Entrada:** Los ficheros `quad.rbt` y `quad.mvm` definidos en las páginas 47 y 69 respectivamente.

1.4.- **Salida esperada:** Visualización del robot cuadrúpedo en movimiento.

## 2.- Configuración lineal de 8 segmentos

2.1.- **Propósito:** Comprobar los siguientes requisitos:

2.1.1.- **RF** *Formato de entrada* (**Esencial**)



2.1.2.- **RU** *Interfaz de línea de comandos* (**Esencial**)



2.1.3.- **RU** *Interfaz visual* (**Esencial**)



2.1.4.- **RI** *Interfaz para programadores* (**Esencial**)



2.2.- **Llamada:** `./simulator.py hypercube.rbt simple.mvm`

2.3.- **Entrada:** Los ficheros `hypercube.rbt` y `simple.mvm` definidos en las páginas 45 y 71 respectivamente.

2.4.- **Salida esperada:** Visualización del robot “hypercube” avanzando mediante oscilaciones de su cuerpo.

## 3.- Configuración estrella de 5 segmentos

3.1.- **Propósito:** Comprobar los siguientes requisitos:

3.1.1.- **RF** *Formato de entrada* (**Esencial**)



3.1.2.- **RU** *Interfaz de línea de comandos* (**Esencial**)



3.1.3.- **RU** *Interfaz visual* (**Esencial**)



3.1.4.- **RI** *Interfaz para programadores* (**Esencial**)



3.2.- **Llamada:** `./simulator.py star.rbt star.mvm`

3.3.- **Entrada:** Los ficheros `star.rbt` y `star.mvm` definidos en las páginas 46 y 67 respectivamente.

3.4.- **Salida esperada:** Robot en forma de estrella pentagonal moviéndose.

## 4.- Configuración lineal de 8 segmentos real

4.1.- **Propósito:** Comprobar los siguientes requisitos:

4.1.1.- **RF** *Formato de entrada* (**Esencial**)



4.1.2.- **RF** *Protocolo de comunicación serie* (**Esencial**)



- 4.1.3.- **RU** *Interfaz de línea de comandos* (**Esencial**) ☒
- 4.1.4.- **RU** *Interfaz visual* (**Esencial**) ☒
- 4.1.5.- **RI** *Interfaz para programadores* (**Esencial**) ☒
- 4.2.- **Llamada:** `./simulator.py hypercube.rbt simple.mvm -d/dev/ttyUSB0`
- 4.3.- **Entrada:** Los ficheros *hypercube.rbt* y *simple.mvm* definidos en las páginas 45 y 71 respectivamente.
- 4.4.- **Salida esperada:** Simulación en pantalla del robot hypercube y movimiento del robot real conectado al PC por el cable serie.

## 7.10. Resumen

La aplicación *simulator* realiza la simulación de los diferentes componentes creados con las herramientas presentadas en las partes anteriores. Utiliza las **estructuras modulares** del diseñador y las **secuencias de movimiento** del generador.

La entrada son los ficheros generados por las otras herramientas y como salida obtenemos una simulación de la secuencia de movimiento aplicada a la estructura modular. Además, se pueden aplicar las secuencias a un robot real conectado al PC por el puerto serie.

La entrada a la aplicación son los ficheros generados por las otras herramientas, gracias a la interoperabilidad de formatos.

Además, se han creado otras librerías de soporte de los formatos definidos .rbt y .mvm y una librería de comunicación serie. Todas ellas pueden ser muy útiles en otros proyectos y fácilmente integrables. Para la renderización de los robots 3D se han utilizado las librerías drawstuff y OpenGL, y para la simulación de la física el Open Dynamics Engine (ODE).



# Extensión del simulador: Algoritmos genéticos

---

## 8.1. Introducción

Una de las características del MRSuite es que dispone de una API para que los usuarios puedan crear sus propias aplicaciones. En este capítulo se presenta la extensión *optimizer* que permite encontrar las secuencias de movimiento óptimas mediante la aplicación de algoritmos genéticos. Como ejemplo de aplicación, se va a calcular la secuencia óptima para que la estructura en estrella de cinco módulos vista en el apartado 3.9.3 pueda desplazarse en línea recta.

La extensión *optimizer*, además de servir de ejemplo para mostrar cómo aplicar la funcionalidad del MRSuite, es una herramienta muy útil para la búsqueda de soluciones. A partir de una estructura modular dada, se calculan las secuencias de movimiento óptimas.

## 8.2. El problema de la coordinación

El problema a resolver es el que se conoce como problema de la coordinación: cómo coordinar el movimiento de todos los módulos para conseguir que el robot se mueva de la mejor forma. Utilizando nuestro modelo bioinspirado, este problema se traduce en encontrar los parámetros de las funciones generadoras que hacen que el robot se mueva de forma óptima.

Se trata de un problema de búsqueda en un espacio de soluciones. Para su resolución se pueden emplear algoritmos de búsqueda como, por ejemplo, los algoritmos genéticos o el “simulated annealing” entre otros.

El criterio para realizar las búsquedas es la función de evaluación. Ésta podría ser la velocidad del robot, el paso, el consumo, ... Para evaluar una solución se simula el robot y se obtienen los valores de las variables de interés.

### 8.3. Espacio de soluciones

El espacio de soluciones está formado por las matrices de movimiento de  $M$  filas  $S$  columnas, siendo  $M$  el número de módulos y  $S$  las muestras empleadas para las funciones generadoras. Tiene por tanto una dimensión igual a  $M \times S$ .

Si se utilizan números enteros para representar las posiciones de los servos y teniendo en cuenta que el rango de giro es de 180 grados, el número total de elementos que hay en este espacio es de  $180^{M \times S}$ . No sólo se trata de un espacio muy grande, sino que además crece exponencialmente con el número de módulos y el número de muestras.

En el ejemplo de aplicación que se muestra en el apartado 8.5.2, el robot está constituido por 5 módulos. Si se utilizan 10 muestras para las funciones generadoras se obtiene que en total hay  $180^{50} = 5,810^{112}$  matrices diferentes. ¡Este número es decenas de órdenes de magnitud mayor que la cantidad estimada de átomos en el universo!

La dimensión de este espacio de búsqueda se puede reducir si en vez de usarse las matrices de movimiento se emplease otro espacio formado por los parámetros de las funciones generadoras. Por ejemplo, si en el problema anterior se usasen funciones generadoras sinusoidales, vistas en el apartado 4.4, cada una de las cuales tiene 4 parámetros diferentes, el espacio tendría ahora  $4 \times 5 = 20$  dimensiones.

Sin embargo, el objetivo del *optimizer* es mostrar cómo implementar un algoritmo de búsqueda en un espacio de soluciones. Será tarea del investigador el buscar el espacio de búsqueda que mejor se adapte a sus intereses. En esta implementación se utilizará el espacio de búsqueda de las matrices de movimiento.

### 8.4. Algoritmos genéticos.

El optimizer realiza la búsqueda en el espacio de soluciones de las matrices de movimiento utilizando algoritmos genéticos. La función de evaluación empleada es:

$$f(x_{inicial}, y_{inicial}, x_{final}, y_{final}) = \sqrt{(x_{final} - x_{inicial})^2 + (y_{final} - y_{inicial})^2} \quad (8.1)$$

Con dicha función se buscan las secuencias de movimiento que consiguen un mayor desplazamiento del robot.

Cada individuo tiene como código genético una lista de  $M \times S$  números en el rango  $[-90, 90]$ . Existen 3 operadores para que la población avance a la siguiente generación. El primer operador es la copia directa de un individuo, con probabilidad del 10 %. El segundo es el entrecruzamiento de dos individuos con un sólo punto de corte, con probabilidad del 50 %. Por último, la mutación de un individuo en un punto, con probabilidad del 40 %.

Para la construcción de la generación  $i + 1$ -ésima se toman los dos mejores individuos de la generación  $i$ -ésima, de forma que se asegura su supervivencia. Para

completar los individuos de la generación  $i + 1$ -ésima se utilizan los operadores vistos con anterioridad.

La condición de parada que se exige para que se detenga la evolución es un límite de generaciones, típicamente 1000.

## 8.5. La extensión optimizer

### 8.5.1. Pseudocódigo

Para evaluar un individuo definido por su matriz de movimiento se realiza una simulación de 10 segundos del motor físico ODE. Se toma la posición del centro de masas al comienzo y al final de la simulación. Con esto se calcula su desplazamiento mediante la ecuación 8.1 y se determina su evaluación.

Se realiza la evaluación de todos los individuos de la generación  $i$ -ésima y se van seleccionando de mejor a peor para producir la generación  $i + 1$ -ésima utilizándose los operadores descritos en el apartado anterior.

El pseudocódigo de lo que hace el optimizador es el siguiente:

---

**Pseudocódigo 8.1** optimizer: optimizer <archivo de estructura>

---

- 1: Inicializa unas secuencias de movimiento al azar
  - 2: **while** condición de parada **do**
  - 3:   Crea una nueva simulación   \ \* API del simulador \*\
  - 4:   Se obtiene el centro de masas del robot en su posición inicial   \ \* API del simulador \*\
  - 5:   Genera un nuevo bucle de simulación durante 10 segundos con la estructura requerida y las secuencias.   \ \* API del simulador \*\
  - 6:   Se obtiene el centro de masas del robot al final de la simulación   \ \* API del simulador \*\
  - 7:   Se calcula el desplazamiento a partir de los centros de masas
  - 8:   Los algoritmos genéticos seleccionan los nuevos individuos para conformar la generación siguiente
  - 9: **end while**
  - 10: Se escribe la **secuencia de movimiento** más prometedora a un archivo.
- 

Se pretende mostrar con esto que un usuario avanzado en el uso de Python y con la información recogida en este documento podría diseñar sus propias soluciones a medida, gracias a las API's desarrolladas.

### 8.5.2. Experimento de prueba

Como experimento de prueba del *optimizer* se va a generar la secuencia de movimiento óptima para la configuración en estrella de 5 módulos vista en el apartado

3.9.3. Se ha empleado una población de 10 individuos. El experimento finaliza a las 1000 generaciones.

#### 1.- Configuración estrella de 5 segmentos

1.1.- **Llamada:** `./optimizer.py star.rbt starOptimus.mvm`

1.2.- **Entrada:** El fichero `star.rbt` definido en la página 46.

1.3.- **Salida:**

- ***starOptimus.mvm*:** Almacena la matriz de movimiento obtenida como solución.
- ***historic.m*:** Script Octave donde se visualiza la evolución de la población.

Este experimento se ha repetido 3 veces. El tiempo medio de ejecución ha sido de 655 segundos. Con esto tenemos que:

$$655 / (1000 \text{ generaciones} \times 10 \text{ individuos} \times 10 \text{ segundos de simulación}) = 6,510^{-3}$$

Con lo que resulta que por cada segundo de simulación, se necesitan 6,5 milésimas de segundo. Por supuesto, este tiempo variará dependiendo de la complejidad del robot simulado.

## 8.6. Resultados

Ahora se exponen los resultados obtenidos para cada una de las repeticiones del experimento anterior, para cada caso se muestra la matriz resultante y la gráfica de evolución:

#### 1.- Primera prueba:

1.1.- **Matriz:**

```

1  -54 27 -79 22 67 24 -25 12 -3 18
2  -89 33 23 52 -37 25 -31 84 54 -50
3  -44 -7 35 72 88 -39 62 67 59 -10
4   39 -13 18 -72 -31 -50 34 82 -35 -56
5  -79 1 32 -53 -63 62 34 89 -8 50
```

1.2.- **Gráfico:**

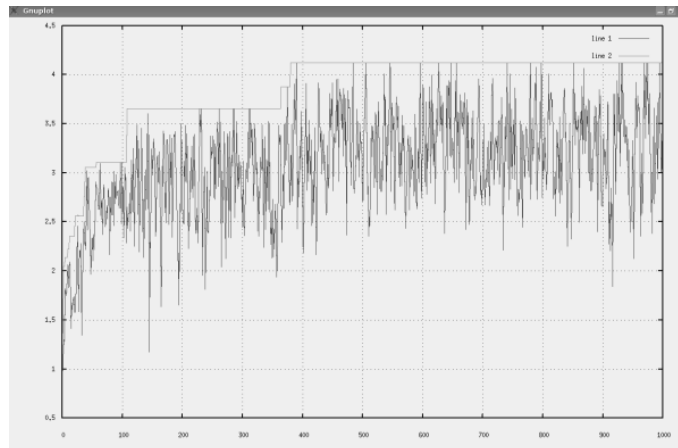


Figura 8.1: Gráfico de evolución de los algoritmos genéticos (I)

## 2.- Segunda prueba:

### 2.1.- Matriz:

|   |     |     |     |     |     |    |     |     |     |     |
|---|-----|-----|-----|-----|-----|----|-----|-----|-----|-----|
| 1 | -82 | 66  | 20  | 60  | -23 | 41 | -80 | 45  | -80 | -49 |
| 2 | -4  | 81  | -82 | 56  | -11 | -9 | 81  | -16 | -66 | -53 |
| 3 | 35  | -61 | -89 | 56  | -58 | 66 | 52  | -49 | -86 | -63 |
| 4 | 56  | 57  | 84  | 21  | -13 | 71 | -19 | -17 | 27  | 54  |
| 5 | 45  | 40  | 27  | -89 | 14  | 16 | 62  | -89 | 88  | -57 |

### 2.2.- Gráfico:

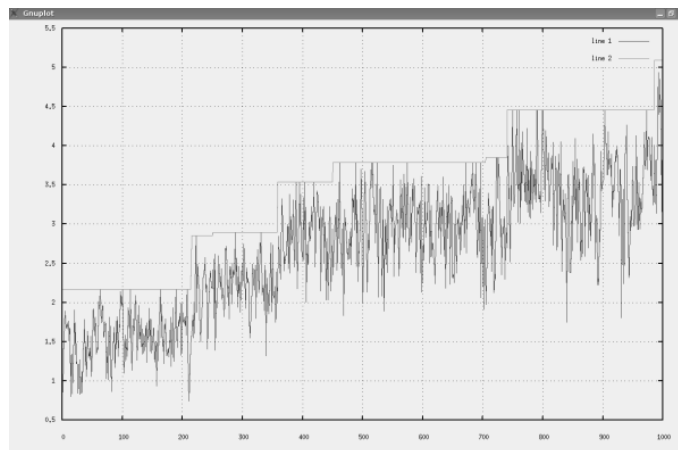


Figura 8.2: Gráfico de evolución de los algoritmos genéticos (II)

## 3.- Tercera prueba:

### 3.1.- Matriz:

```

1  -46 75 39 -51 83 12 -2 -50 47 -80
2  -30 -72 -10 -72 80 -67 -63 -39 -4 -40
3  -18 8 34 -49 47 -37 -19 -46 -71 27
4   52 -74 64 -70 88 -60 -30 49 10 10
5  -26 88 44 -27 -55 -53 -33 -40 11 17

```

### 3.2.- Gráfico:

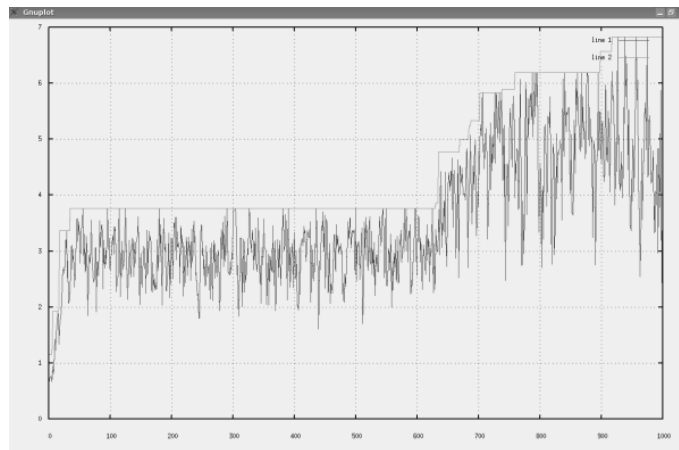


Figura 8.3: Gráfico de evolución de los algoritmos genéticos (III)

## 8.7. Resumen

La herramienta de simulación incorpora una API que permite a los programadores desarrollar sus propias aplicaciones. Para comprobar su viabilidad se ha creado un *plugin* para encontrar las mejores secuencias de movimiento de una estructura modular. Este *plugin* se llama *optimizer*. Realiza las búsquedas mediante un algoritmo genético en el espacio de soluciones de las matrices de movimiento.

Como ejemplo de prueba, se ha realizado un experimento con la configuración en estrella de 5 módulos. Tras 3 experimentos de 1000 generaciones cada uno, se ha conseguido un desplazamiento máximo de 6,83 unidades del motor físico. Que representan aproximadamente 6,83 cm del mundo real. La velocidad media estimada sería de  $6,83/10 = 0,683$  cm/s.

Utilizando esta idea de extensiones, el MRSuite se convierte en una herramienta muy personalizable para que los investigadores en el campo de la robótica modular puedan realizar sus propios experimentos de manera rápida y sencilla. Además, dado el carácter libre de este software, cada nuevo *plugin* creado se puede ser incorporar al repositorio oficial, haciendo que la herramienta crezca.



---

Parte IV

## **MRSuite: Entorno completo de simulación**

---



---

## Capítulo 9

# El entorno MRSuite

---

### 9.1. Introducción

El objetivo de este proyecto es crear el entorno MRSuite para el diseño y simulación de robots modulares con topologías de 1 y 2 dimensiones. Esta aplicación está compuesta de las tres herramientas descritas en los capítulos anteriores: *designer*, *generator* y *simulator*.

El MRSuite es un entorno de prototipado rápido de robots modulares. Permite la creación de las estructuras modulares, la generación de las secuencias de movimiento, la simulación del robot y, opcionalmente, el movimiento de robots reales conectados a través del puerto serie. Además, ofrece una API para que los programadores puedan extender su funcionalidad, adaptándola a sus necesidades.

En este capítulo se presenta la arquitectura global del entorno, las pruebas de aceptación, un breve manual de usuario y, lo más importante, las pruebas de validación del simulador. Para ello se han tomado los resultados publicados del movimiento de los robots PP e Hypercube y se han comparado cualitativamente con los obtenidos con el MRSuite. Los resultados validan el funcionamiento del simulador.

### 9.2. Arquitectura

En la figura 9.1 se muestra el diagrama de bloques del MRSuite. Está compuesto por las tres aplicaciones *generator*, *simulator* y *designer*, y por los tres formatos de ficheros necesarios para describir las funciones generadoras (.cpg), el grafo estructural (.rbt) y la matriz de movimiento (.mvm) respectivamente. Las flechas indican cómo es el flujo de diseño.

El usuario define la estructura modular del robot a través del *designer*. La secuencia de movimiento se obtiene a partir de las funciones generadoras usando el *generator*. Finalmente la simulación se realiza a partir de la matriz de movimiento y el grafo estructural. Opcionalmente, el simulador puede también mover un robot real, conectado a través del puerto serie del PC.

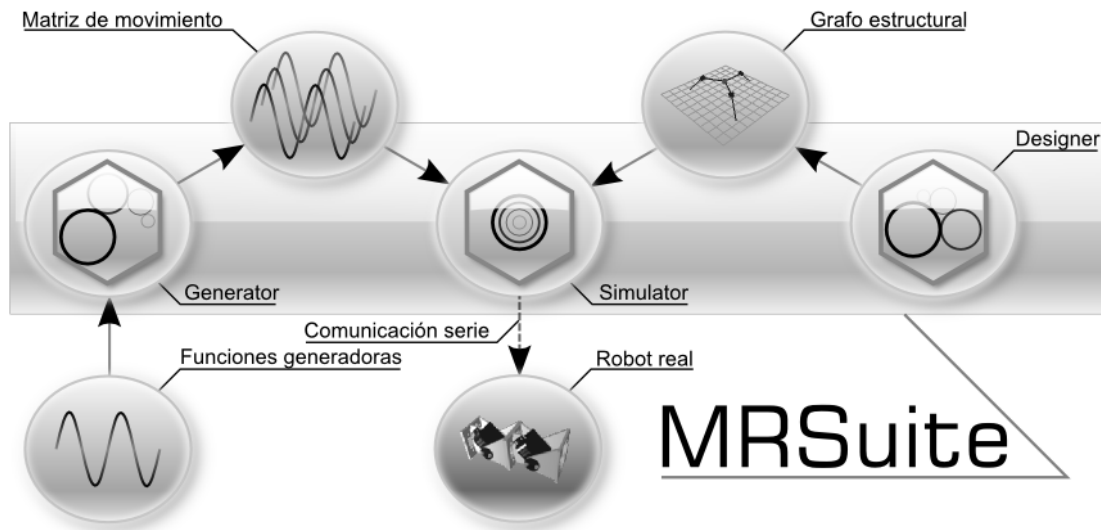


Figura 9.1: Diagrama de bloques del MRSuite

### 9.3. Pruebas de aceptación

Para comprobar el MRSuite se han diseñado y simulado tres robots diferentes: cube revolutions (topología 1D), un robot cuadrúpedo de 9 módulos (topología 2D) y una estrella de cinco módulos (topología 2D). Para cada una de ellas se ha creado su estructura modular, una matriz de movimiento y finalmente se ha simulado. Adicionalmente, se ha utilizado la extensión optimizer para la generación de una secuencia de movimiento óptima para la estrella de 5 módulos.

#### ■ Configuración lineal

##### 1.- Llamadas:

- `./designer.py lineal.rbt`
- `./generator.py simple.cpg simple.mvm`
- `./simulator.py lineal.rbt simple.mvm`

##### 2.- Entrada:

- La estructura robótica visualizada en la figura 3.8.
- El fichero `simple.cpg` descrito en la página 66.

##### 3.- Pantallazo:

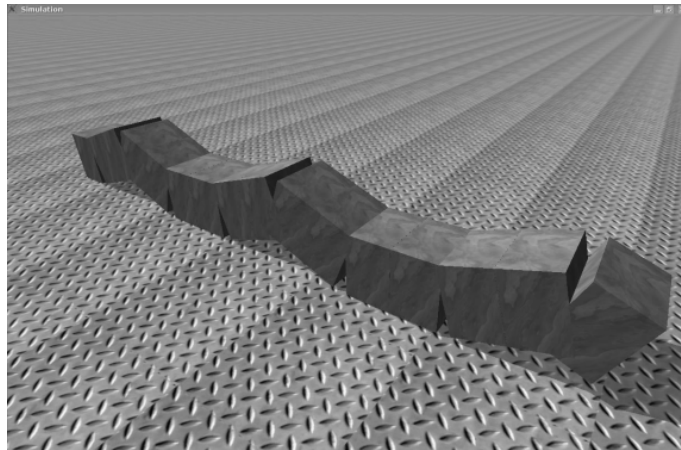


Figura 9.2: Simulación de la configuración lineal de 8 módulos

■ Configuración cuadrúpeda

1.- Llamadas:

- `./designer.py quad.rbt`
- `./generator.py quad.cpg quad.mvm`
- `./simulator.py quad.rbt quad.mvm`

2.- Entrada:

- La estructura robótica visualizada en la figura 3.10.
- El fichero `quad.cpg` descrito en la página 65.

3.- Pantallazo:

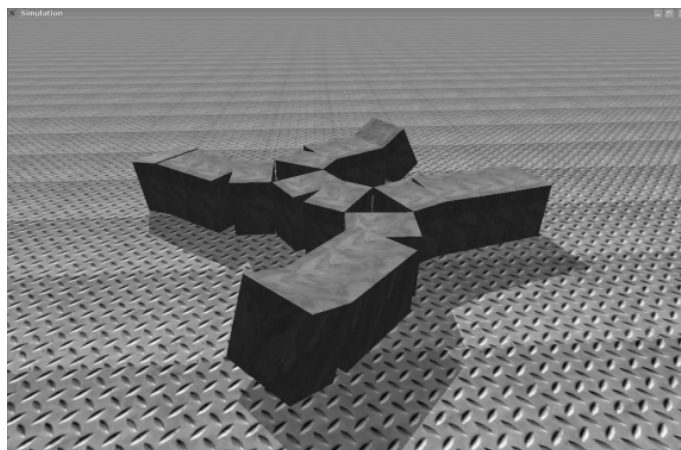


Figura 9.3: Simulación de la configuración cuadrúpeda

■ Configuración en estrella

### 1.- Llamadas:

- `./designer.py star.rbt`
- `./optimizer.py star.rbt starOptimus.mvm`
- `./simulator.py star.rbt starOptimus.mvm`

### 2.- Entrada:

- La estructura robótica visualizada en la figura 3.9.

### 3.- Pantallazo:

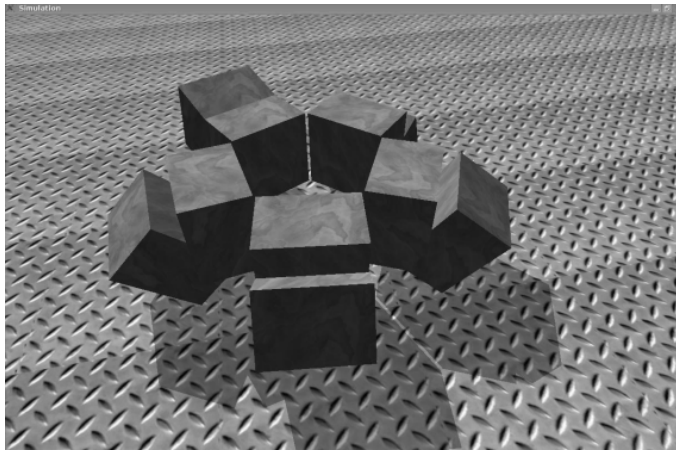


Figura 9.4: Simulación de la configuración en estrella

## 9.4. Validación del simulador

### 9.4.1. Pruebas con el robot PP

Para la realización de estas pruebas se ha tenido en cuenta el artículo [JGGa] presentado en el **Clawar 2005**.

La definición del robot será la siguiente:

```
1 0 VNN 1
2 1 VNN 2
```

A continuación se describen las diferentes pruebas realizadas:

- Desfase de 0 grados, el robot PP no se mueve.

### 1.- Llamadas:

- `./generator.py PP-test1.cpg PP-test1.mvm`
- `./simulator.py PP.rbt PP-test1.mvm`

**2.- Entrada:** El fichero *PP-test1.cpg*.

```

1  # Fichero de validación del robot PP, prueba 1.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -    -    --- --
10     f          30  1      0  0
11     f          30  1      0  0

```

**3.- Pantallazo:**

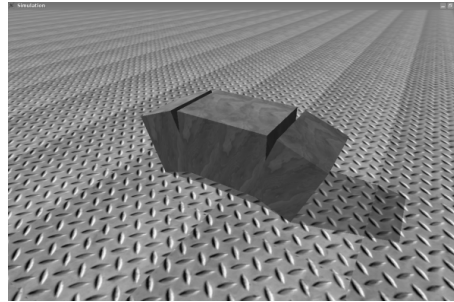


Figura 9.5: Simulación del robot PP (I)

- Desfase de 180 grados, el robot PP no se mueve.

**1.- Llamadas:**

- `./generator.py PP-test2.cpg PP-test2.mvm`
- `./simulator.py PP.rbt PP-test2.mvm`

**2.- Entrada:** El fichero *PP-test2.cpg*.

```

1  # Fichero de validación del robot PP, prueba 2.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -    -    --- --
10     f          30  1      0  0
11     f          30  1     180  0

```

### 3.- Pantallazo:

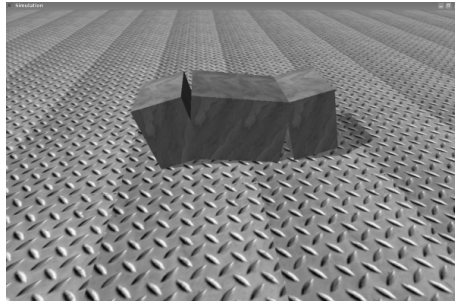


Figura 9.6: Simulación del robot PP (II)

- Desfase de 120 grados, amplitud 30, el robot se mueve en línea recta, avanzando.

#### 1.- Llamadas:

- `./generator.py PP-test3.cpg PP-test3.mvm`
- `./simulator.py PP.rbt PP-test3.mvm`

#### 2.- Entrada: El fichero *PP-test3.cpg*.

```

1  # Fichero de validación del robot PP, prueba 3.
2
3  # Definición de la función generadora
4  f: A, T, Phi, 0, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + 0
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi 0
9  # ----- -    -    --- --
10     f        30   1     0  0
11     f        30   1    120  0

```

### 3.- Pantallazo:

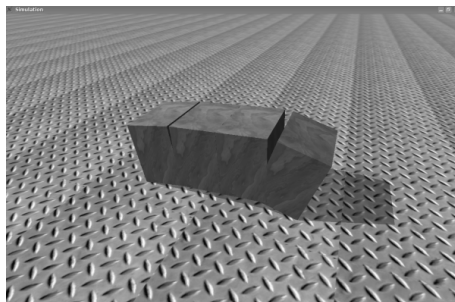


Figura 9.7: Simulación del robot PP (III)

- Desfase de  $-120$  grados, amplitud 30, el robot se mueve en línea recta, retrocediendo.

**1.- Llamadas:**

- `./generator.py PP-test4.cpg PP-test4.mvm`
- `./simulator.py PP.rbt PP-test4.mvm`

**2.- Entrada:** El fichero *PP-test4.cpg*.

```

1  # Fichero de validación del robot PP, prueba 4.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -    -    --- --
10   f           30  1     0  0
11   f           30  1  -120  0

```

**3.- Pantallazo:**

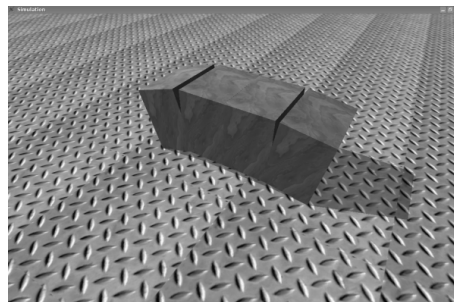


Figura 9.8: Simulación del robot PP (IV)

- Desfase de 120 grados, amplitud 45, el robot avanza en línea recta, más rápido cuanto mayor es la amplitud.

**1.- Llamadas:**

- `./generator.py PP-test5.cpg PP-test5.mvm`
- `./simulator.py PP.rbt PP-test3.mvm PP-test5.mvm`

**2.- Entrada:** El fichero *PP-test5.cpg*.

```

1  # Fichero de validación del robot PP, prueba 5.
2
3  # Definición de la función generadora

```

```

4  f: A, T, Phi, 0, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + 0
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi 0
9  # ----- -    -    --- --
10   f          45  1     0  0
11   f          45  1    120  0

```

### 3.- Pantallazo:

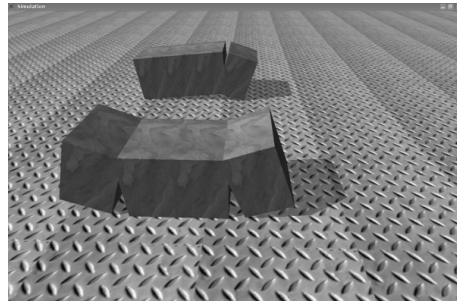


Figura 9.9: Simulación del robot PP (V)

En esta última prueba se ha realizado una carrera entre el robot de la prueba 3, al fondo, y el robot de esta prueba, en primera instancia. Se observa como el robot de esta prueba tiene un avance mayor.

Todas las pruebas realizadas han sido satisfactorias, el robot PP es simulado como se expone en el artículo.

### 9.4.2. Pruebas con el robot Hypercube

Estas pruebas tomarán como referencia los resultados del artículo [JGGb] presentado en el **Clawar 2006**.

La definición del robot será la siguiente:

```

1  0 VNN 1
2  1 HNN 2
3  2 VNN 3
4  3 HNN 4
5  4 VNN 5
6  5 HNN 6
7  6 VNN 7
8  7 HNN 8

```

A continuación se describen las diferentes pruebas realizadas:

- Movimiento en línea recta.

#### 1.- Llamadas:

- `./generator.py hypercube-test1.cpg hypercube-test1.mvm`
- `./simulator.py hypercube.rbt PP-test1.mvm`

#### 2.- Entrada: El fichero *hypercube-test1.cpg*.

```

1  # Fichero de validación del robot hypercube, prueba 1.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -- - --- --
10   f          40  1   120 0
11   f           0  1     0 0
12   f          40  1   240 0
13   f           0  1     0 0
14   f          40  1     0 0
15   f           0  1     0 0
16   f          40  1   120 0
17   f           0  1     0 0

```

#### 3.- Pantallazo:

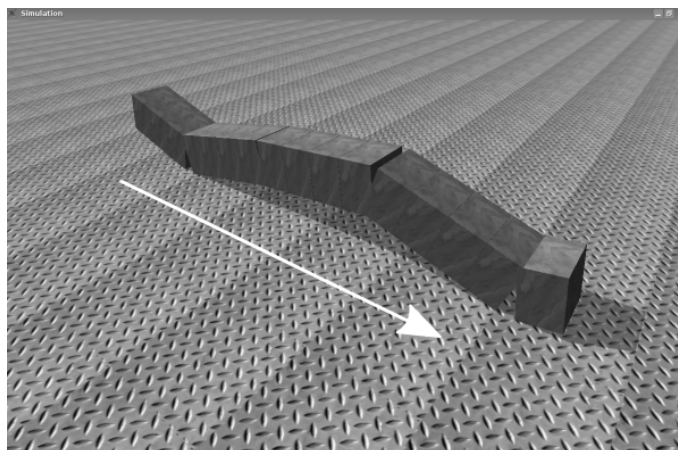


Figura 9.10: Simulación del robot hypercube (I)

- Giro.

#### 1.- Llamadas:

- `./generator.py hypercube-test2.cpg hypercube-test2.mvm`
- `./simulator.py hypercube.rbt PP-test2.mvm`

**2.- Entrada:** El fichero *hypercube-test2.cpg*.

```

1  # Fichero de validación del robot hypercube, prueba 2.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -- - --- --
10 f          40  1   120  0
11 f           0  1     0 20
12 f          40  1   240  0
13 f           0  1     0 20
14 f          40  1     0  0
15 f           0  1     0 20
16 f          40  1   120  0
17 f           0  1     0 20

```

**3.- Pantallazo:**

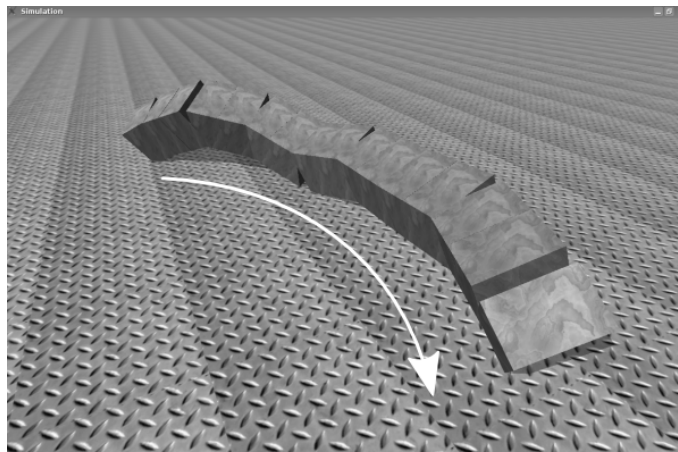


Figura 9.11: Simulación del robot hypercube (II)

■ Rodar.

**1.- Llamadas:**

- `./generator.py hypercube-test3.cpg hypercube-test3.mvm`
- `./simulator.py hypercube.rbt PP-test3.mvm`

**2.- Entrada:** El fichero *hypercube-test3.cpg*.

```

1  # Fichero de validación del robot hypercube, prueba 3.
2
3  # Definición de la función generadora
4  f: A, T, Phi, O, t: int (A * sin ((2 * pi * t) / (T * samples)
5  + DEG_TO_RAD * Phi)) + O
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi O
9  # ----- -    -    --- --
10 f          40  1    0    0
11 f          40  1   90    0
12 f          40  1    0    0
13 f          40  1   90    0
14 f          40  1    0    0
15 f          40  1   90    0
16 f          40  1    0    0
17 f          40  1   90    0

```

**3.- Pantallazo:**

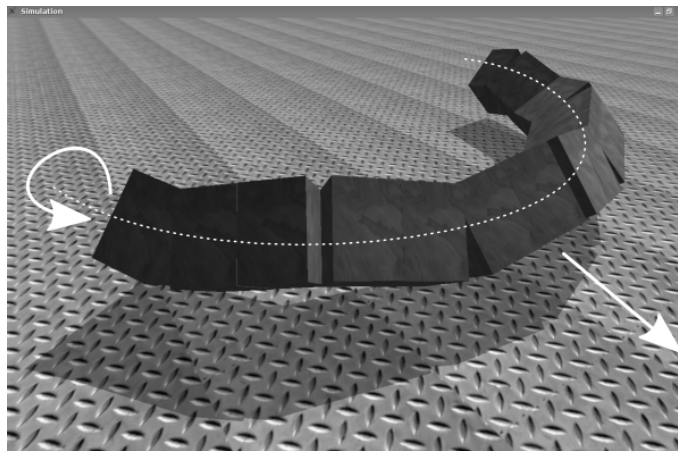


Figura 9.12: Simulación del robot hypercube (III)

■ Desplazamiento lateral.

**1.- Llamadas:**

- `./generator.py hypercube-test4.cpg hypercube-test4.mvm`
- `./simulator.py hypercube.rbt PP-test4.mvm`

**2.- Entrada:** El fichero *hypercube-test4.cpg*.

```

1  # Fichero de validación del robot hypercube, prueba 4.

```

```

2
3 # Definición de la función generadora
4 f: A, T, Phi, 0, t: int (A * sin ((2 * pi * t) / (T * samples)
5 + DEG_TO_RAD * Phi)) + 0
6
7 # Asignación de parámetros a cada función
8 # Función A    T    Phi 0
9 # ----- -    -    --- --
10 f      40  1     0  0
11 f      40  1     0  0
12 f      40  1    100  0
13 f      40  1    100  0
14 f      40  1    200  0
15 f      40  1    200  0
16 f      40  1    300  0
17 f      40  1    300  0

```

### 3.- Pantallazo:

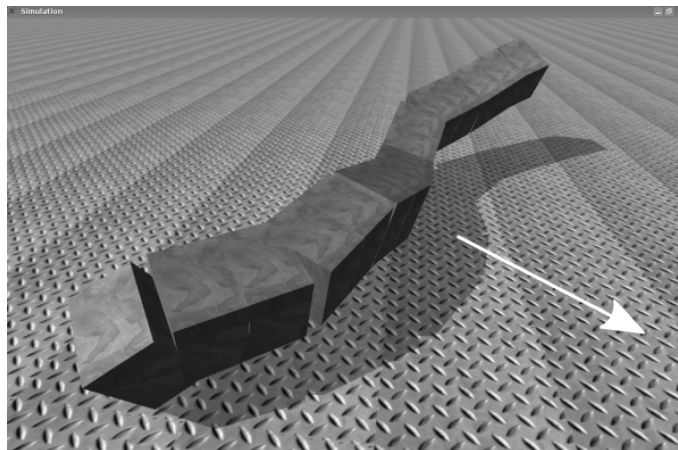


Figura 9.13: Simulación del robot hypercube (IV)

#### ■ Rotación.

##### 1.- Llamadas:

- `./generator.py hypercube-test5.cpg hypercube-test5.mvm`
- `./simulator.py hypercube.rbt PP-test5.mvm`

##### 2.- Entrada: El fichero *hypercube-test5.cpg*.

```

1 # Fichero de validación del robot hypercube, prueba 5.
2
3 # Definición de la función generadora
4 f: A, T, Phi, 0, t: int (A * sin ((2 * pi * t) / (T * samples)

```

```

5  + DEG_TO_RAD * Phi)) + 0
6
7  # Asignación de parámetros a cada función
8  # Función A    T    Phi 0
9  # ----- -    -    --- --
10 f            40  1     0  0
11 f            40  1     0  0
12 f            40  1    120  0
13 f            40  1     50  0
14 f            40  1    240  0
15 f            40  1    100  0
16 f            40  1     0  0
17 f            40  1    150  0

```

### 3.- Pantallazo:

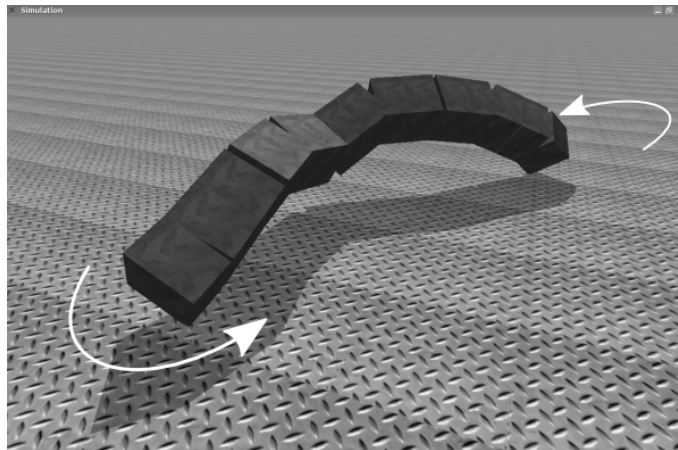


Figura 9.14: Simulación del robot hypercube (V)

Al igual que en caso anterior, los robots simulados se han comportado de acuerdo al artículo de referencia.

## 9.5. Manual de usuario

### 9.5.1. Instalación

Una vez descomprimido el paquete de MRSuite es necesario realizar la compilación de la librería DrawStuff. Para ello basta con entrar en el directorio *drawstuff* y hacer *make*.

Además tienen que estar instaladas las siguientes librerías:

- Librería gráfica OpenGL y los *wrappers* de Python (*freeglut3* o *libglut3* y *python-opengl* desde apt-get)
- Motor físico ODE y los *wrappers* de Python PyODE (*python-pyode* y sus dependencias desde apt-get)

### 9.5.2. Manejo

Para facilitar al usuario la utilización de las herramientas se resumen a continuación las llamadas a las aplicaciones junto con sus parámetros:

- designer [fichero de salida]:

1.- Fichero de salida (Opcional)

- generator <fichero de entrada> [fichero de salida] [opciones]:

1.- Fichero de entrada (Obligatorio)

2.- Fichero de salida (Opcional)

3.- Opciones

- *-s<samples>*: Determina las muestras que se van a tomar. Si no se especifica, serán 10 muestras.
  - *-c*: Activa el formato de salida para el lenguaje C.
  - *-octave*: Activa el formato de salida para Octave.
  - *-mrs*: Activa el formato de salida para el simulador, definido en el apartado 5.3.2.
- simulator <fichero de **grafo estructural**> [fichero de **matriz de movimiento**] [opciones]:

1.- Fichero de **grafo estructural** (Obligatorio)

2.- Fichero de **matriz de movimiento** (Opcional)

3.- Opciones

- *-t<seconds>*: Determina el tiempo durante el cual correrá la simulación. Si no se especifica será infinito.
  - *-n<# robots>*: Define el número de robots que se van a simular, a modo de carrera.
  - *-d<serial dev>*: Especifica el dispositivo serie que se usa para enviar los movimientos al posible robot físico.
  - *-m*: Flag que indica la renderización de **módulos Y1**.

- *-w*: Flag que determina el uso de cubos (por defecto).
- *-s*: Flag para realizar una simulación sin renderizado.

## 9.6. Resumen

Con la unión de las herramientas *designer*, *generator* y *simulator* se da lugar al entorno de desarrollo para robótica modular **MRSuite**. Con este entorno completo se puede crear un nuevo robot modular basado en **módulos Y1** desde cero, creando tanto la parte estructural como las **secuencias de movimiento**. Así mismo, permite probarlo sobre un robot real gemelo.

El entorno permite a los investigadores de robótica modular tener una herramienta de prototipado rápido para probar sus nuevas ideas.

---

## Capítulo 10

# Resultados y conclusiones

---

Los objetivos iniciales de este proyecto eran el desarrollo de software para:

- 1.- Simular la locomoción de robots modulares.
- 2.- Generar secuencias de movimiento.
- 3.- Crear estructuras modulares a partir de unos módulos básicos.

Inicialmente se creó un prototipo del simulador que permitía la visualización de los robots modulares en forma de alambre. Con ello se aprendió el manejo de las librerías de representación en 3D y se evaluó su viabilidad.

Como los resultados obtenidos fueron satisfactorios, se pasó a la siguiente fase del proyecto: incorporar el motor físico. Se eligió el Open Dynamics Engine (ODE) por su licencia libre y su amplio uso en el mundo de la robótica de investigación. Tras una etapa de estudio y aprendizaje se integró en el prototipo del simulador, obteniéndose la primera versión funcional y estable. Primer objetivo cumplido.

A continuación se abordó el tema de la generación automática de secuencias de movimiento. Siguiendo los enfoques empleados en varios robots de investigación se optó por implementar un modelo bioinspirado basado en generadores centrales de patrones (CPG's) simplificados. Se diseñó el formato de los ficheros de movimiento (.mvm) y se desarrolló la herramienta generator. Segundo objetivo cumplido.

Además de cumplir con los objetivos iniciales se han realizado una serie de mejoras adicionales que amplían las posibilidades del MRSuite:

- 1.- Modularización del MRSuite y creación de una API para permitir a los usuarios la creación de sus propias herramientas o extender el funcionamiento mediante *plugins*. Para cada una de las tres aplicaciones se ha creado su correspondiente API. Las librerías creadas son: Design, Step, Robot, Sequence y Simulation.
- 2.- Desarrollo de la extensión *optimizer* para buscar soluciones de locomoción mediante algoritmos genéticos. A partir de la nueva API es posible extender o crear nuevas aplicaciones. Un ejemplo es el *optimizer*.

- 3.- Modificación del simulador de forma que sea capaz de controlar robots reales conectados a través del puerto serie del PC. Esto permite comprobar en robots físicos las diferentes secuencias de movimiento generadas.
- 4.- El entorno MRSuite ha sido validado por medio de la simulación de robots modulares reales conocidos, obteniéndose resultados muy satisfactorios. Podemos decir que es una herramienta que está madura para su uso tanto en investigación como en la educación.

Como conclusión final, destacar que con este proyecto no sólo se han cubierto las expectativas iniciales, sino que se han superado, haciendo del entorno MRSuite una herramienta muy útil para los investigadores en el campo de la robótica modular.

---

## Capítulo 11

# Líneas futuras

---

Este proyecto no está cerrado, sino que puede seguir mejorando y ampliando. Las mejoras propuestas son:

- 1.- Formato de los ficheros en XML. Utilización del formato estándar XML para la representación de las matrices de movimiento, grafos estructurales y funciones generadoras. Además de la mayor estandarización, con este formato se logra una fácil extensión. Permitiendo que en el futuro se pudiese incorporar más información en estos ficheros, manteniendo la compatibilidad hacia atrás. Por ejemplo se podría extender el formato de representación de grafos estructurales para incluir a las topologías 3D.
- 2.- Convertir el designer en un entorno completo de creación y edición de robots modulares. La aplicación actual permite su creación y la generación del fichero .rbt, pero no incluye capacidades avanzadas de diseño como el copy/paste, undo/redo o el borrado.
- 3.- Integración de las tres herramientas actuales en una única aplicación que permita acceder a cada una de ellas mediante una interfaz gráfica. Con eso se mejoraría la usabilidad.
- 4.- Ampliación del simulador para soportar robots modulares con topologías 3D. Esto permitiría diseñar y simular nuevos tipos de robots como por ejemplo esferas o “erizos” y estudiar sus propiedades.
- 5.- Posibilidad de incluir sensores virtuales en los módulos para simular comportamientos y conductas en los robots, permitiendo trabajar en algoritmos que hagan al robot reaccionar frente al entorno.
- 6.- Creación de obstáculos y de diferentes escenarios para realizar pruebas de locomoción en entornos más complejos.

Al tratarse de un proyecto abierto, se permite que el MRSuite evolucione hacia donde sus usuarios quieran. El autor sólo ha sembrado la semilla para que la aplicación vaya creciendo y pueda algún día convertirse en una herramienta de referencia en el mundo de la robótica modular.

---

## Bibliografía

- [Ada] *Simulador Adam*. <http://birg.epfl.ch/page56513-en.html>.
- [Alc] *Concurso Alcabot*. <http://www.depeca.uah.es/alcabot>.
- [Ang86] ANGULO, JOSE MARÍA: *Robótica práctica*. Ed. Paraninfo, 1986.
- [Asi] *Robot Asimo*. <http://world.honda.com/ASIMO>.
- [Cuba] *Robot ápedo Cube Revolutions*. <http://www.iearobotics.com/personal/juan/doctorado/cube-revolutions>.
- [Cubb] *Robot modular Cube*. <http://www.iearobotics.com/personal/juan/doctorado/cube-revolutions>.
- [Dar] *Entorno Darwin2K*. <http://darwin2k.sourceforge.net>.
- [Hyp] *Videos de movimiento del robot ápedo Hypercube*. <http://www.iearobotics.com/personal/juan/doctorado/hypercube/download>.
- [JGGa] JUAN GONZÁLEZ GÓMEZ, EDUARDO BOEMO: *Motion of Minimal Configurations of a Modular Robot: Sinusoidal, Lateral Rolling and Lateral Shift*. 8th International Conference on Climbing and Walking Robots (CLAWAR).
- [JGGb] JUAN GONZÁLEZ GÓMEZ, HOUXIANG ZHANG, EDUARDO BOEMO JIANWEI ZHANG: *Locomotion Capabilities of a Modular Robot with Eight Pitch-Yaw-Connecting Modules*. 9th International Conference on Climbing and Walking Robots (CLAWAR).
- [lam] *Movimiento de las lampreas*. <http://es.wikipedia.org/wiki/Lamprea>.
- [Man] *Cuaderno técnico 10. Motor físico ODE en Python*. <http://www.iearobotics.com/proyectos/cuadernos/ct10/ct10.html>.
- [Mic] *Microsoft Robotics Studio*. <http://msdn2.microsoft.com/en-us/robotics/bb625969.aspx>.
- [MPG89] MIKELL P. GROOVER, MITCHELL WEISS: *Robótica industrial. Tecnología, programación y aplicaciones*. Ed. Mc Graw Hill, 1989.
- [MTr] *Robot modular M-Tran (Japón)*. <http://unit.aist.go.jp/is/dsysd/mtran/English/experimentE.htm>.
- [ODE] *Motor físico ODE*. <http://www.ode.org>.
- [PC83] PHILLIPPE COIFFET, MICHAEL CHIROUZE: *An introduction to ROBOT technology*. Ed. Kogan Page, 1983.

- 
- [PI] *Robot PI*. <http://www.rbzrobotdesign.com/index.php?module=3&lang=en&cID=12>.
- [Pol] *Robot modular PolyBot (USA)*. <http://www2.parc.com/spl/projects/modrobots/polybot/polybot.html>.
- [PyO] *Motor físico ODE, wrappers para Python*. <http://pyode.sourceforge.net>.
- [Pyr] *Entorno Pyro*. <http://www.pyrorobotics.org>.
- [Pyt] *Lenguaje de programación Python*. <http://www.python.org>.
- [Rob] *El término robot, según la Wikipedia*. <http://es.wikipedia.org/wiki/Robot>.
- [ser] *Servidor de 8 servos para SkyPic*. <http://www.iearobotics.com/proyectos/stargate/servidores/sg-servos8/sg-servos8.html>.
- [Skya] *Robot SkyBot*. <http://www.iearobotics.com/proyectos/skybot/skybot.html>.
- [Skyb] *Tarjeta controladora SkyPic*. <http://www.iearobotics.com/proyectos/skypic/skypic.html>.
- [Sta] *Proyecto Stargate para comunicaciones PC-SkyPic*. <http://www.iearobotics.com/proyectos/stargate>.
- [Tar] *Robot Tarribot*. <http://www.tarribot.com>.
- [Tri] *Robot Tritt*. <http://www.iearobotics.com/proyectos/tritt/tritt.html>.
- [Web] *Entorno Webots*. <http://www.cyberbotics.com>.
- [Y1] *Módulos Y1*. <http://www.iearobotics.com/personal/juan/doctorado/Modulos-Y1/modulos-y1.html>.